
Oracle Database 11g: PL/SQL Fundamentals

Electronic Presentation

D49990GC11
Edition 1.1
April 2009

ORACLE®

Authors

Tulika Srivastava
Lauran K. Serhal

Technical Contributors and Reviewers

Tom Best
Christoph Burandt
Yanti Chang
Ashita Dhir
Peter Driver
Gerlinde Frenzen
Nancy Greenberg
Chaitanya Kortamaddi
Tim Leblanc
Wendy Lo
Bryan Roberts
Abhishek X Singh
Puja Singh
Lex.Van.Der Werff

Editors

Aju Kumar
Raj Kumar

Graphic Designer

Priya Saxena

Publishers

Syed Ali
Giri Venugopal

Copyright © 2009, Oracle. All rights reserved.

Disclaimer

This course provides an overview of features and enhancements planned in release 11g. It is intended solely to help you assess the business benefits of upgrading to 11g and to plan your IT projects.

This course in any form, including its course labs and printed matter, contains proprietary information that is the exclusive property of Oracle. This course and the information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This course and its contents are not part of your license agreement nor can they be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

This course is for informational purposes only and is intended solely to assist you in planning for the implementation and upgrade of the product features described. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remain at the sole discretion of Oracle.

This document contains proprietary information and is protected by copyright and other intellectual property laws. You may copy and print this document solely for your own use in an Oracle training course. The document may not be modified or altered in any way. Except where your use constitutes "fair use" under copyright law, you may not use, share, download, upload, copy, print, display, perform, reproduce, publish, license, post, transmit, or distribute this document in whole or in part without the express authorization of Oracle.

The information contained in this document is subject to change without notice. If you find any problems in the document, please report them in writing to: Oracle University, 500 Oracle Parkway, Redwood Shores, California 94065 USA. This document is not warranted to be error-free.

Restricted Rights Notice

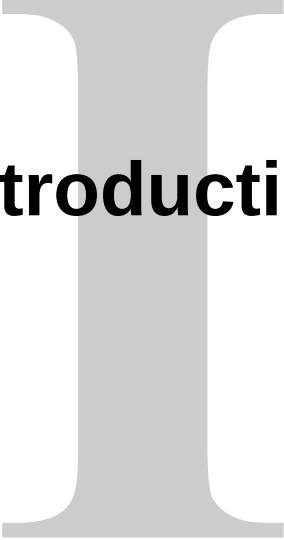
If this documentation is delivered to the United States Government or anyone using the documentation on behalf of the United States Government, the following notice is applicable:

U.S. GOVERNMENT RIGHTS

The U.S. Government's rights to use, modify, reproduce, release, perform, display, or disclose these training materials are restricted by the terms of the applicable Oracle license agreement and/or the applicable U.S. Government contract.

Trademark Notice

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

A large, light gray, serif capital letter 'I' is centered on the page. The word 'Introduction' is written in a bold, black, sans-serif font across the middle of the 'I'.

Introduction

Lesson Objectives

After completing this lesson, you should be able to do the following:

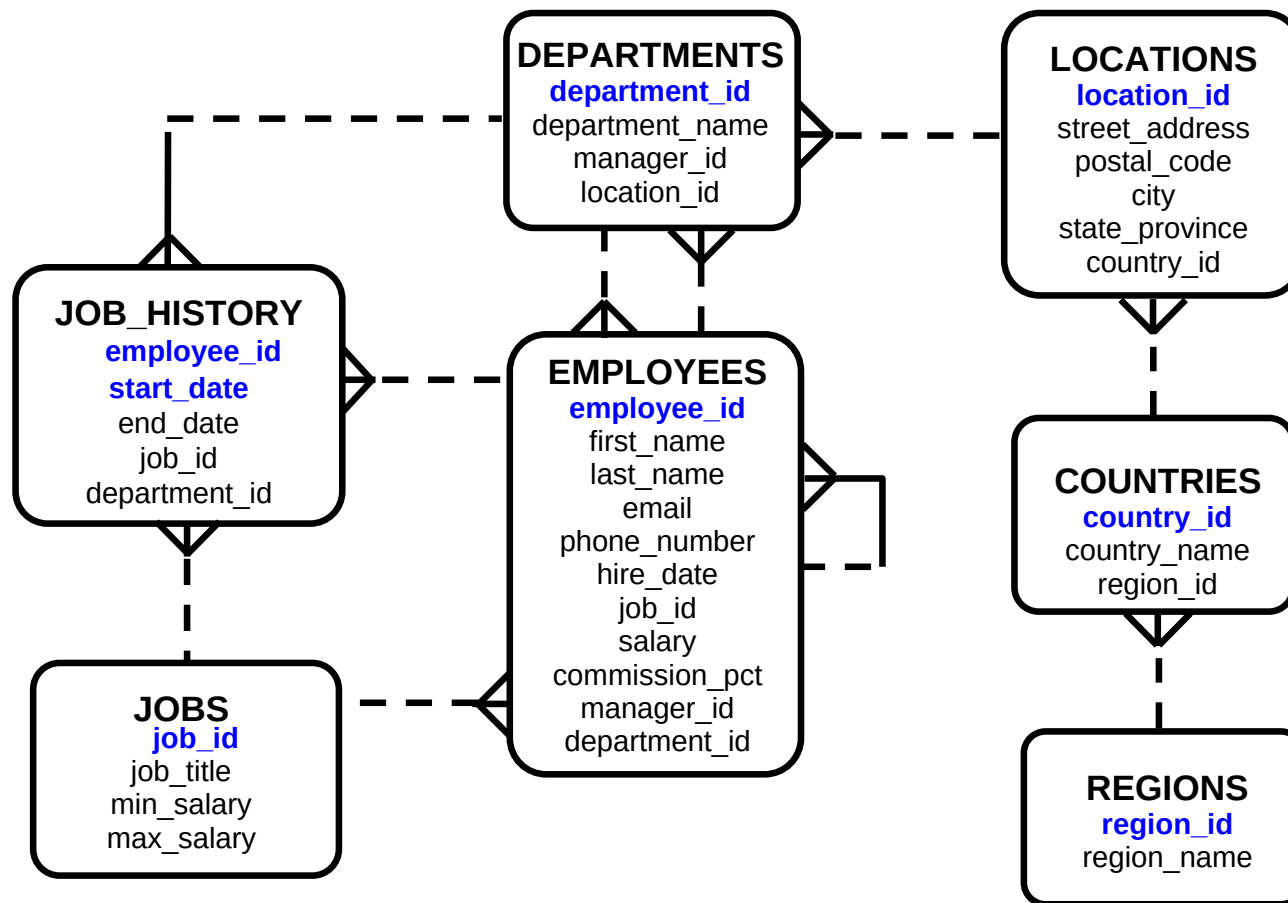
- Discuss the goals of the course
- Identify the available environments that can be used in this course
- Describe the HR database schema that is used in the course
- Review the basic features of SQL Developer
- List the available appendices, documentation, and other resources

Course Objectives

After completing this course, you should be able to do the following:

- Identify the programming extensions that PL/SQL provides to SQL
- Write PL/SQL code to interface with the database
- Design PL/SQL anonymous blocks that execute efficiently
- Use PL/SQL programming constructs and conditional control statements
- Handle run-time errors
- Describe stored procedures and functions

Human Resources (HR) Schema for This Course



Course Agenda

Day 1:

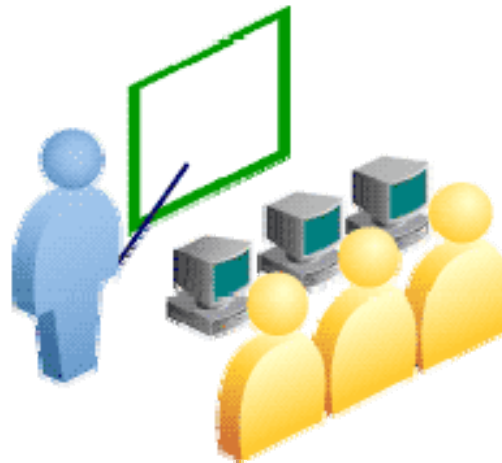
- I. Introduction
 1. Introduction to PL/SQL
 2. Declaring PL/SQL Variables
 3. Writing Executable Statements
 4. Interacting with the Oracle Database Server
 5. Writing Control Structures

Day 2:

6. Working with Composite Data Types
7. Using Explicit Cursors
8. Handling Exceptions
9. Creating Stored Procedures and Functions

Class Account Information

- Cloned HR account IDs are set up for you.
- Your account IDs are ora41–ora60.
- The password matches your account ID.
- Each machine is assigned one account.
- The instructor has a separate ID.



Appendixes Used in This Course

- Appendix A: Practice Solutions
- Appendix B: Table Descriptions and Data
- Appendix C: Using SQL Developer
- Appendix D: Using SQL*Plus
- Appendix E: Using JDeveloper
- Appendix F: REF Cursors
- Additional Practices
- Additional Practice Solutions

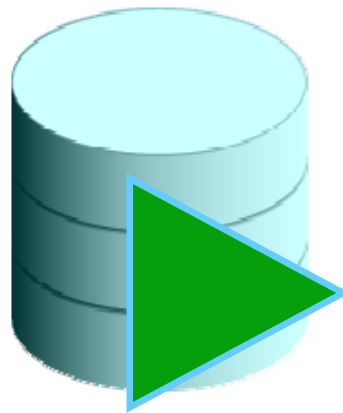
PL/SQL Development Environments

This course setup provides the following tools for developing PL/SQL code:

- Oracle SQL Developer (used in this course)
- Oracle SQL*Plus
- Oracle JDeveloper IDE

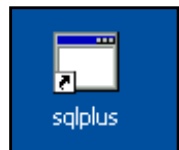
What Is Oracle SQL Developer?

- Oracle SQL Developer is a free graphical tool that enhances productivity and simplifies database development tasks.
- You can connect to any target Oracle database schema using standard Oracle database authentication.
- You will use SQL Developer in this course.



SQL Developer

Coding PL/SQL in SQL*Plus



```
C:\ sqlplus

SQL*Plus: Release 11.1.0.4.0 - Beta on Mon Apr 30 08:01:20 2007
Copyright (c) 1982, 2007, Oracle. All rights reserved.

SQL> connect ora61/ora61@orcl
Connected.
SQL> set serveroutput on
SQL> create or replace procedure hello is
  2  begin
  3    dbms_output.put_line('Hello Class!');
  4  end;
  5  /

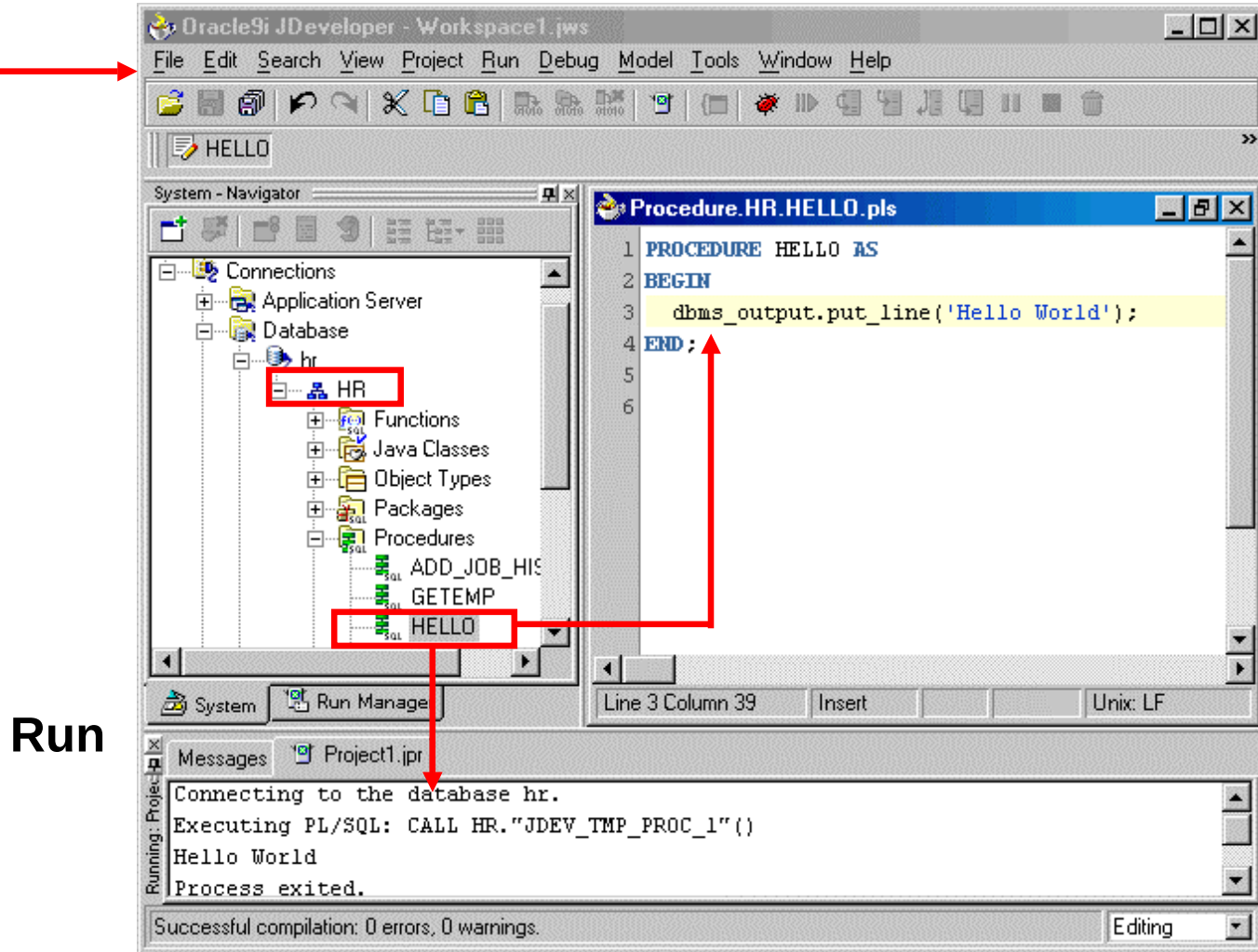
Procedure created.

SQL> execute hello
Hello Class!

PL/SQL procedure successfully completed.

SQL> _
```

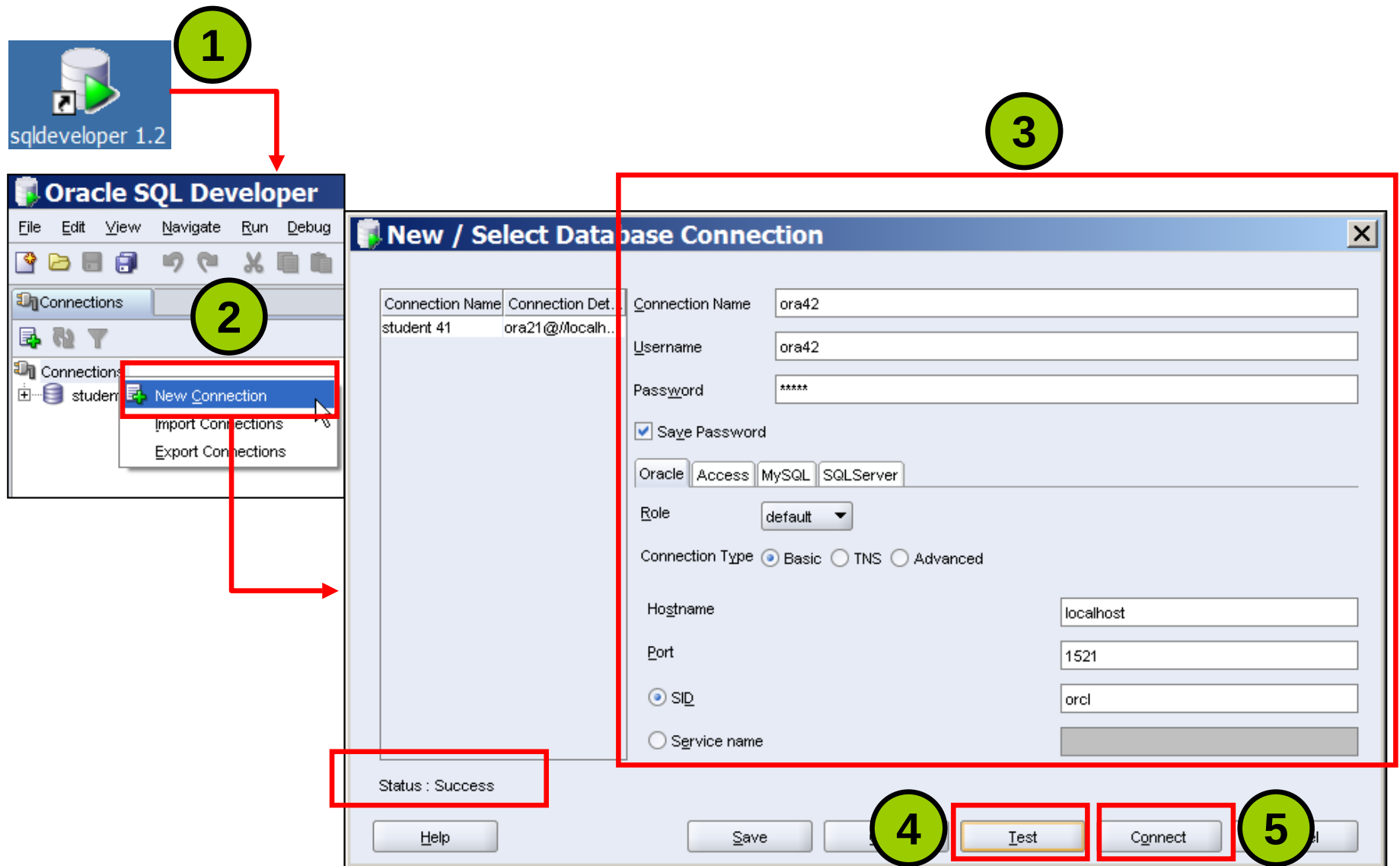
Coding PL/SQL in Oracle JDeveloper



Edit

Run

Starting SQL Developer and Creating a Database Connection

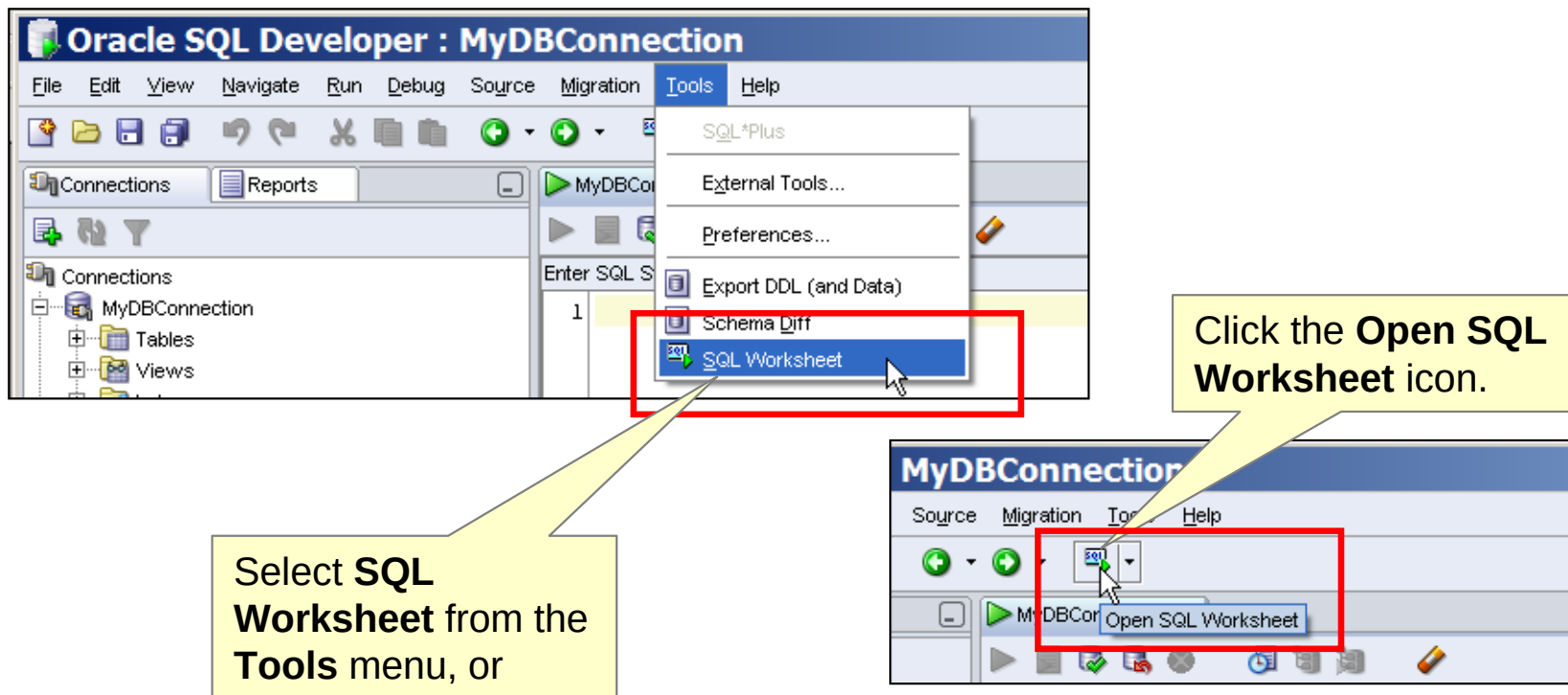


Creating Schema Objects

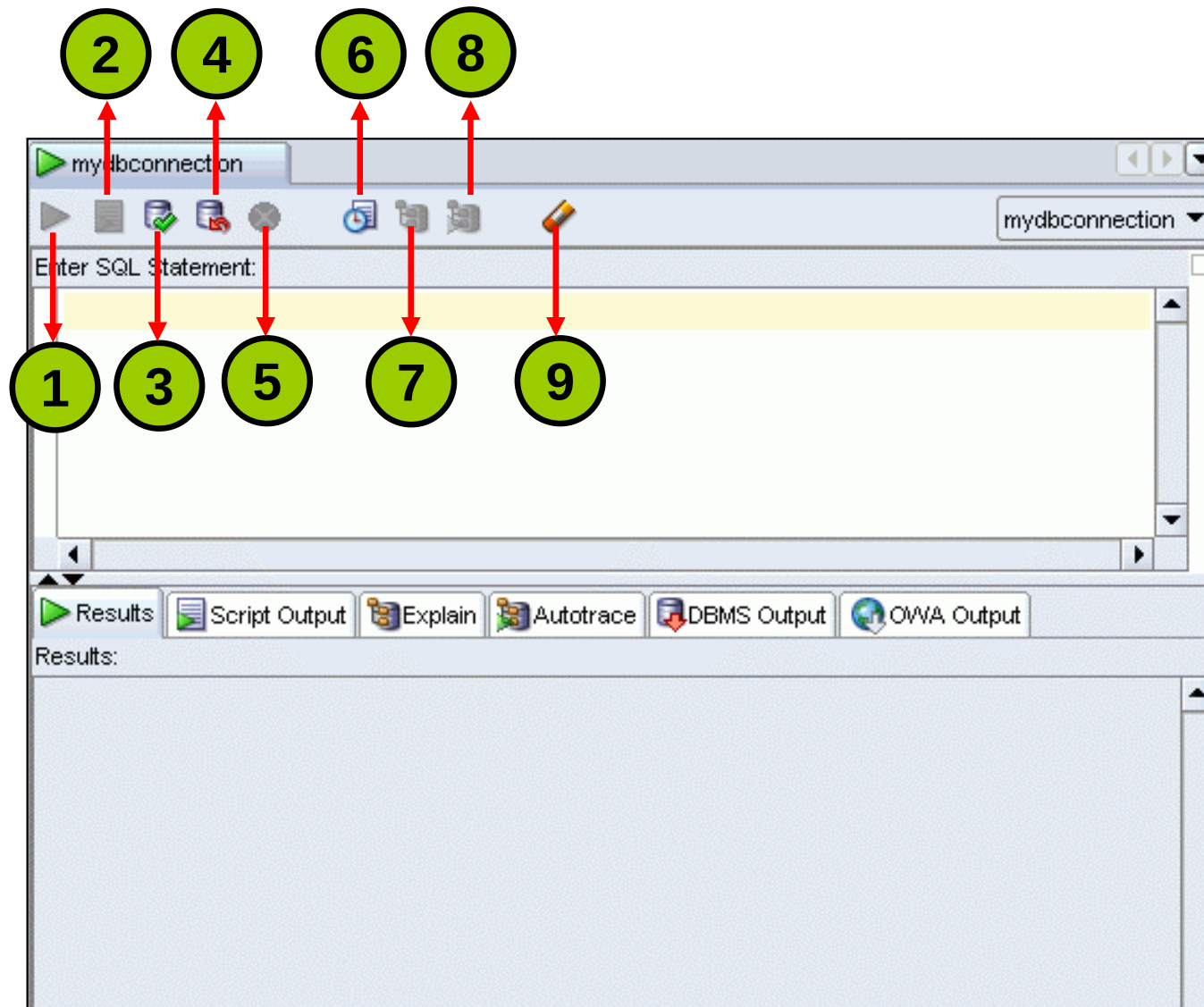
- You can create any schema object in SQL Developer using one of the following methods:
 - Executing a SQL statement in the SQL Worksheet
 - Using the context menu
- Edit the objects using an edit dialog box or one of the many context-sensitive menus.
- View the DDL for adjustments such as creating a new object or editing an existing schema object.

Using the SQL Worksheet

- Use the SQL Worksheet to enter and execute SQL, PL/SQL, and SQL *Plus statements.
- Specify any actions that can be processed by the database connection associated with the worksheet.



Using the SQL Worksheet



Executing SQL Statements

Use the Enter SQL Statement box to enter single or multiple SQL statements.

Use the **Enter SQL Statement** box to enter single or multiple SQL statements.

View the results on the **Script Output** tabbed page.

MyDBConnection

2.03304029 seconds

Enter SQL Statement:

```
1 SELECT last_name, salary
2 FROM employees
3 WHERE salary > 10000;
4
5 SELECT last_name "Name", salary*12 "Annual Salary"
6 FROM employees;
```

Results Script Output Explain Autotrace DBMS Output OWA Output

Ozer	11500
Abel	11000

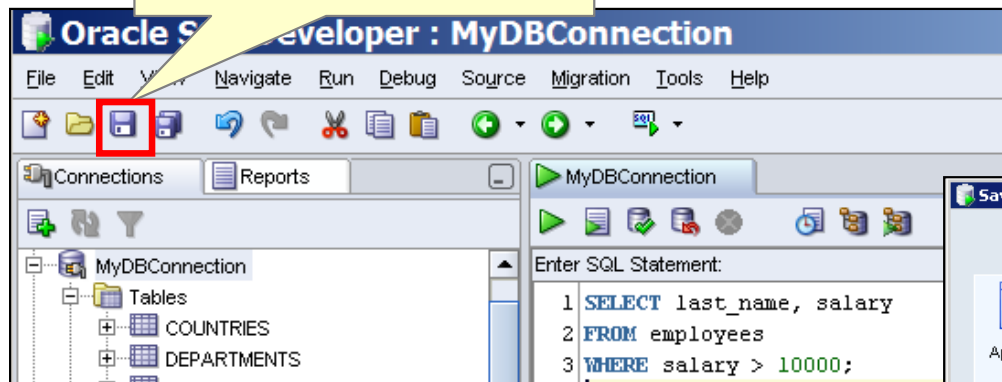
15 rows selected

Name	Annual Salary
OConnell	31200
Grant	31200
Whalen	52800

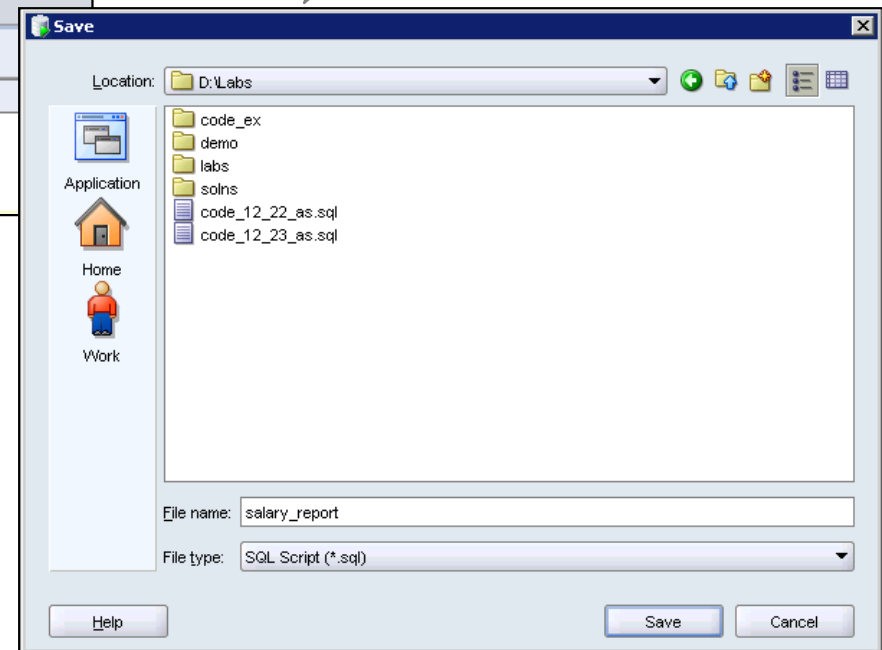
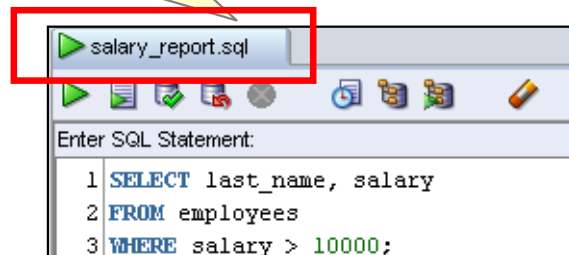
Saving SQL Scripts

Click the **Save** icon to save your SQL statement to a file.

Enter a file name and identify a location to save the file, and then click **Save**.



The contents of the saved file are visible and editable in your SQL Worksheet window.



Executing Saved Script Files: Method 1

Right-click in the SQL Worksheet area, and then select **Open File** from the shortcut menu.

Select (or navigate to) the script file that you want to open.

Click **Open**.

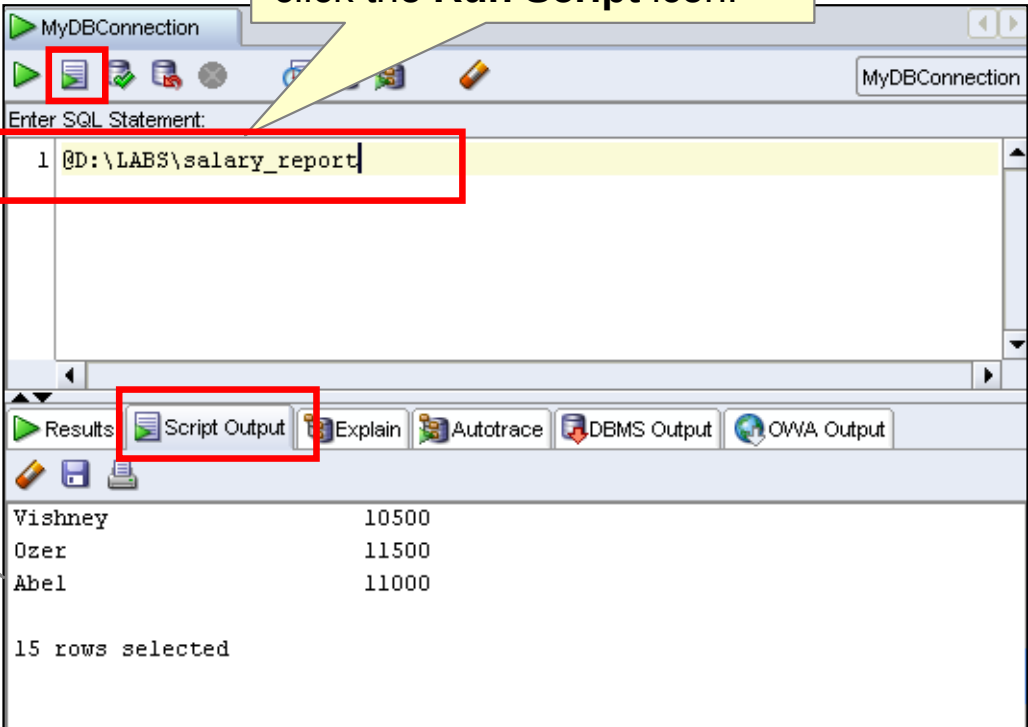
To run the code, click the **Run Script (F5)** icon.

```
CREATE PROCEDURE getemp IS -- header
emp_id employees.employee_id%type;
lname employees.last_name%type;
BEGIN
emp_id := 100;
SELECT last_name INTO lname
FROM EMPLOYEES
WHERE employee_id = emp_id;
DBMS_OUTPUT.PUT_LINE('Last name: '||lname);
END;
```

Executing Saved SQL Scripts: Method 2

Use the @ command followed by the location and name of the file that you want to execute, and then click the **Run Script** icon.

The output from the script is displayed on the Script Output tabbed page.



The screenshot shows the Oracle SQL Developer interface. At the top, a toolbar contains several icons, including a document icon with a green checkmark, which is highlighted by a red box. Below the toolbar, the 'Enter SQL Statement:' text box contains the text '1 @D:\LABS\salary_report', which is also highlighted by a red box. Below the text box, a tabbed interface shows several tabs: 'Results', 'Script Output', 'Explain', 'Autotrace', 'DBMS Output', and 'OWA Output'. The 'Script Output' tab is selected and highlighted by a red box. Below the tabs, the output of the script is displayed as a table with two columns: names and salaries. The table contains three rows of data: Vishney (10500), Ozer (11500), and Abel (11000). Below the table, it states '15 rows selected'.

Vishney	10500
Ozer	11500
Abel	11000

15 rows selected

Oracle 11g SQL and PL/SQL Documentation

- *Oracle Database New Features Guide 11g Release 1 (11.1)*
- *Oracle Database Advanced Application Developer's Guide 11g Release 1 (11.1)*
- *Oracle Database PL/SQL Language Reference 11g Release 1 (11.1)*
- *Oracle Database Reference 11g Release 1 (11.1)*
- *Oracle Database SQL Language Reference 11g Release 1 (11.1)*
- *Oracle Database Concepts 11g Release 1 (11.1)*
- *Oracle Database PL/SQL Packages and Types Reference 11g Release 1 (11.1)*
- *Oracle Database SQL Developer User's Guide Release 1.1.2*

Summary

In this lesson, you should have learned how to:

- Discuss the goals of the course
- Identify the available environments that can be used in this course
- Describe the HR database schema that is used in the course
- Review the basic features of SQL Developer
- List the available appendices, documentation, and other resources

Practice I Overview: Getting Started

This practice covers the following topics:

- Starting SQL Developer
- Creating a new database connection
- Browsing the HR schema tables
- Setting some SQL Developer preference

1

Introduction to PL/SQL

Objectives

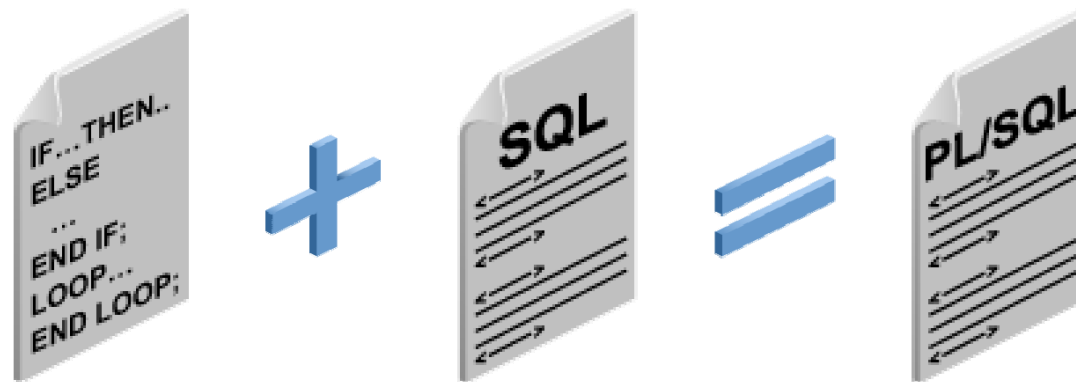
After completing this lesson, you should be able to do the following:

- Explain the need for PL/SQL
- Explain the benefits of PL/SQL
- Identify the different types of PL/SQL blocks
- Output messages in PL/SQL

About PL/SQL

PL/SQL:

- Stands for “Procedural Language extension to SQL”
- Is Oracle Corporation’s standard data access language for relational databases
- Seamlessly integrates procedural constructs with SQL

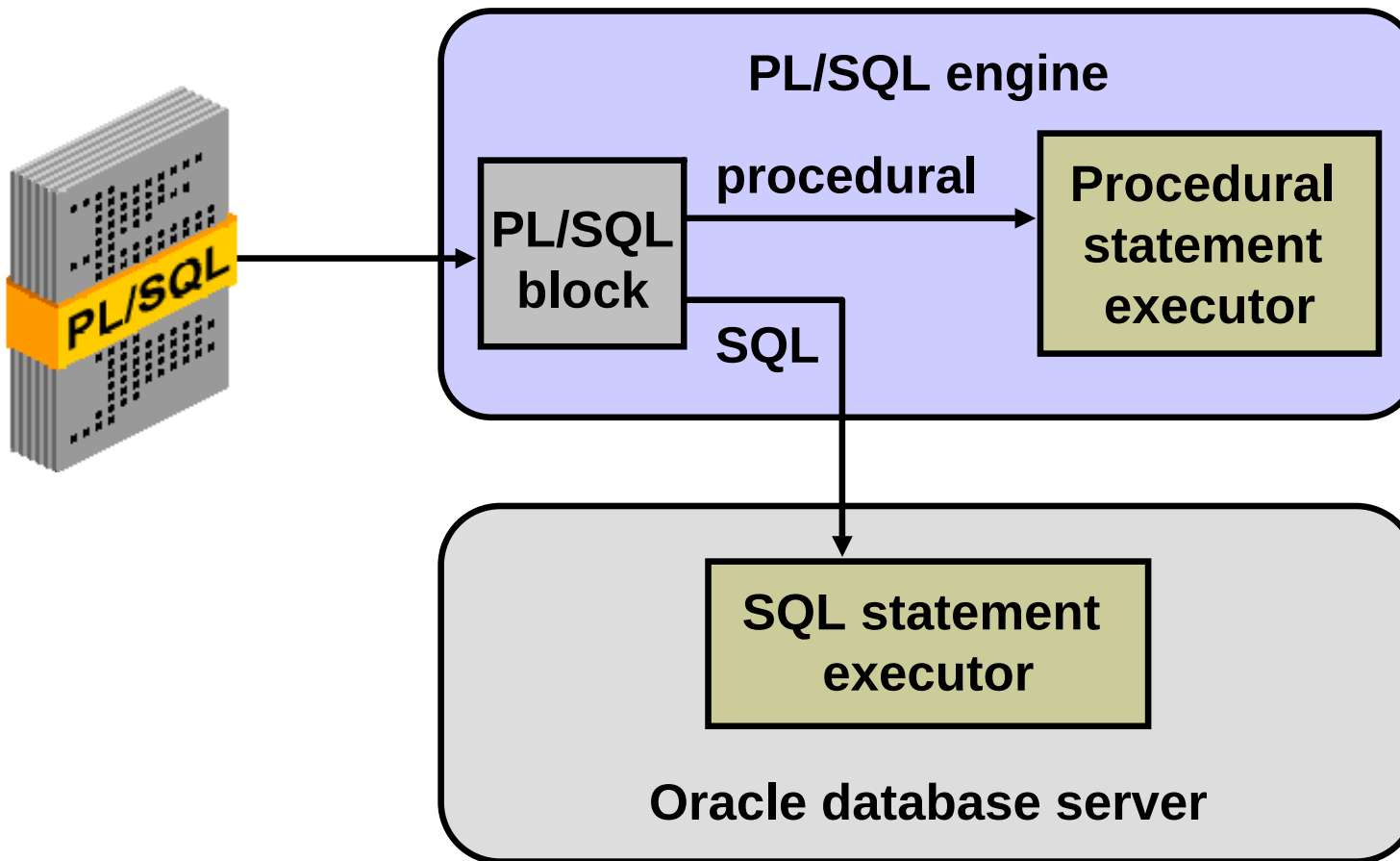


About PL/SQL

PL/SQL:

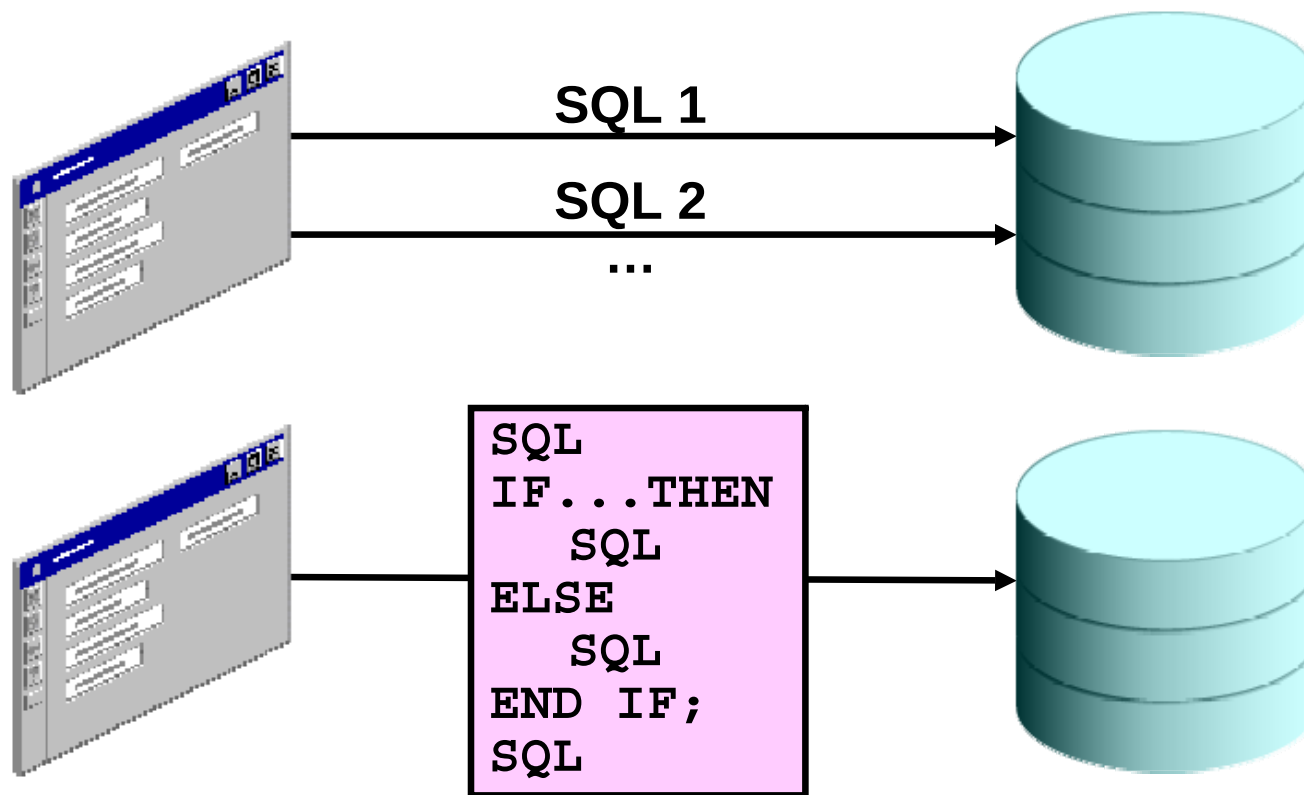
- Provides a block structure for executable units of code. Maintenance of code is made easier with such a well-defined structure.
- Provides procedural constructs such as:
 - Variables, constants, and data types
 - Control structures such as conditional statements and loops
 - Reusable program units that are written once and executed many times

PL/SQL Environment



Benefits of PL/SQL

- Integration of procedural constructs with SQL
- Improved performance

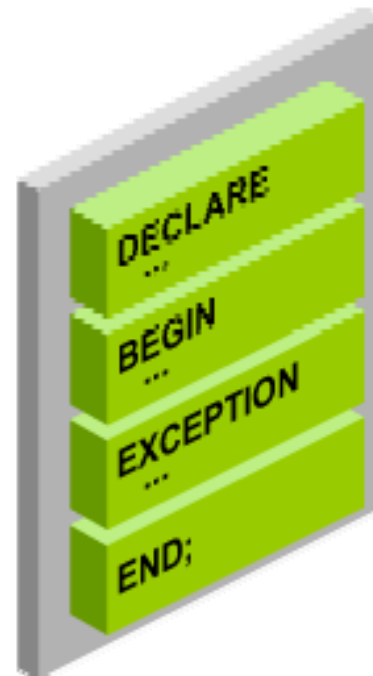


Benefits of PL/SQL

- Modularized program development
- Integration with Oracle tools
- Portability
- Exception handling

PL/SQL Block Structure

- DECLARE (optional)
 - Variables, cursors, user-defined exceptions
- BEGIN (mandatory)
 - SQL statements
 - PL/SQL statements
- EXCEPTION (optional)
 - Actions to perform when errors occur
- END; (mandatory)



Block Types

Anonymous

```
[DECLARE]

BEGIN
  --statements

[EXCEPTION]

END;
```

Procedure

```
PROCEDURE name
IS

BEGIN
  --statements

[EXCEPTION]

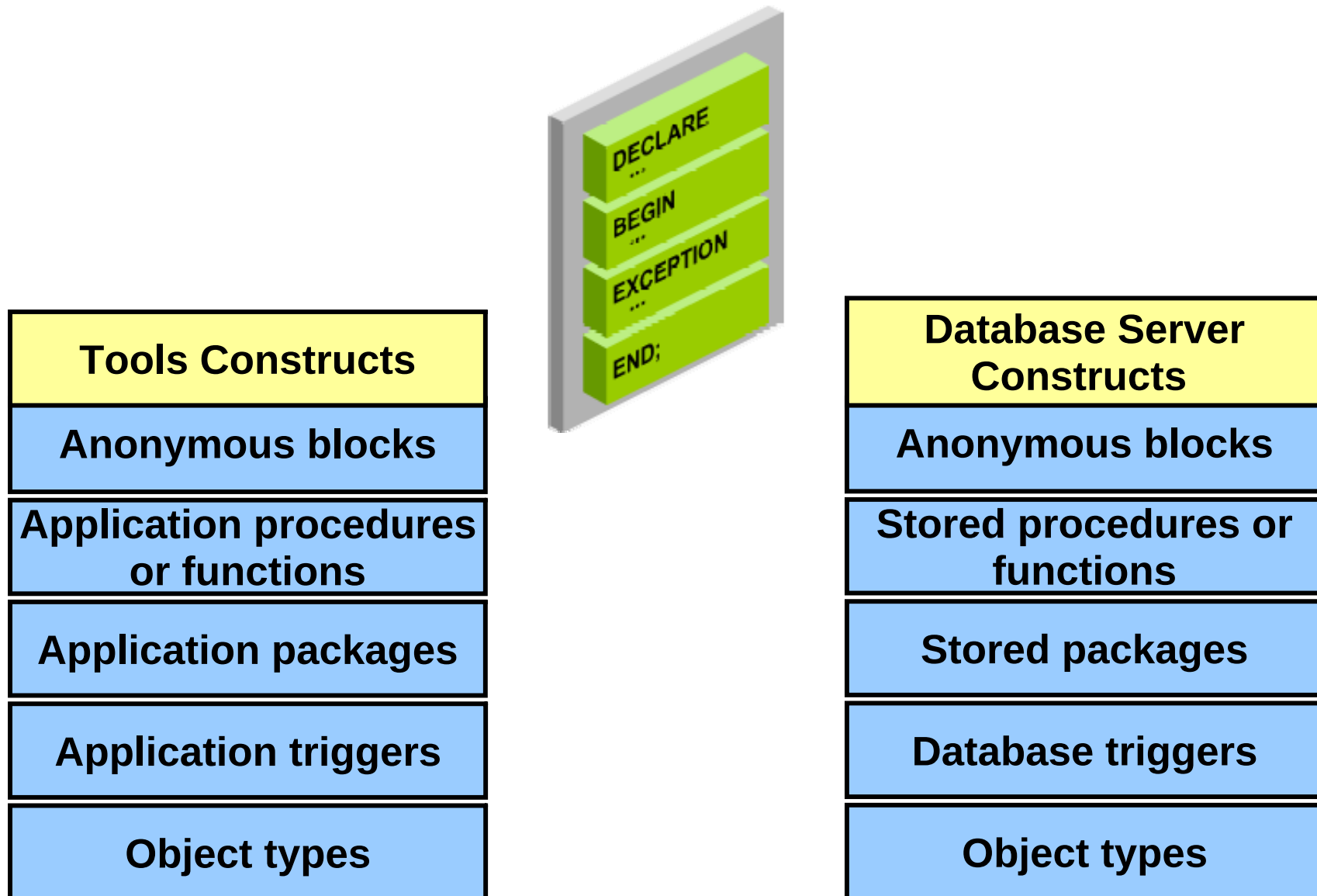
END;
```

Function

```
FUNCTION name
RETURN datatype
IS
BEGIN
  --statements
  RETURN value;
[EXCEPTION]

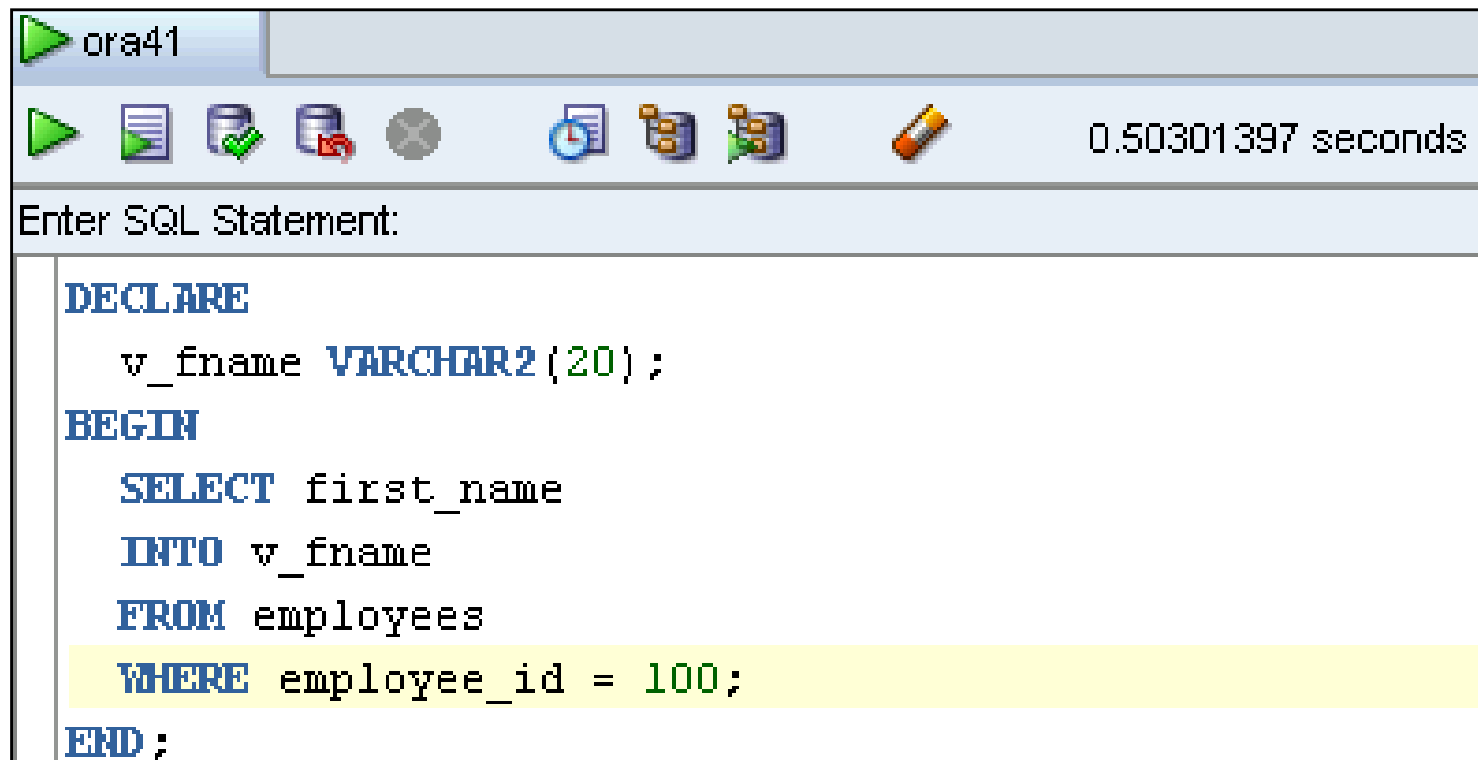
END;
```

Program Constructs



Create an Anonymous Block

Enter the anonymous block in the SQL Developer workspace:

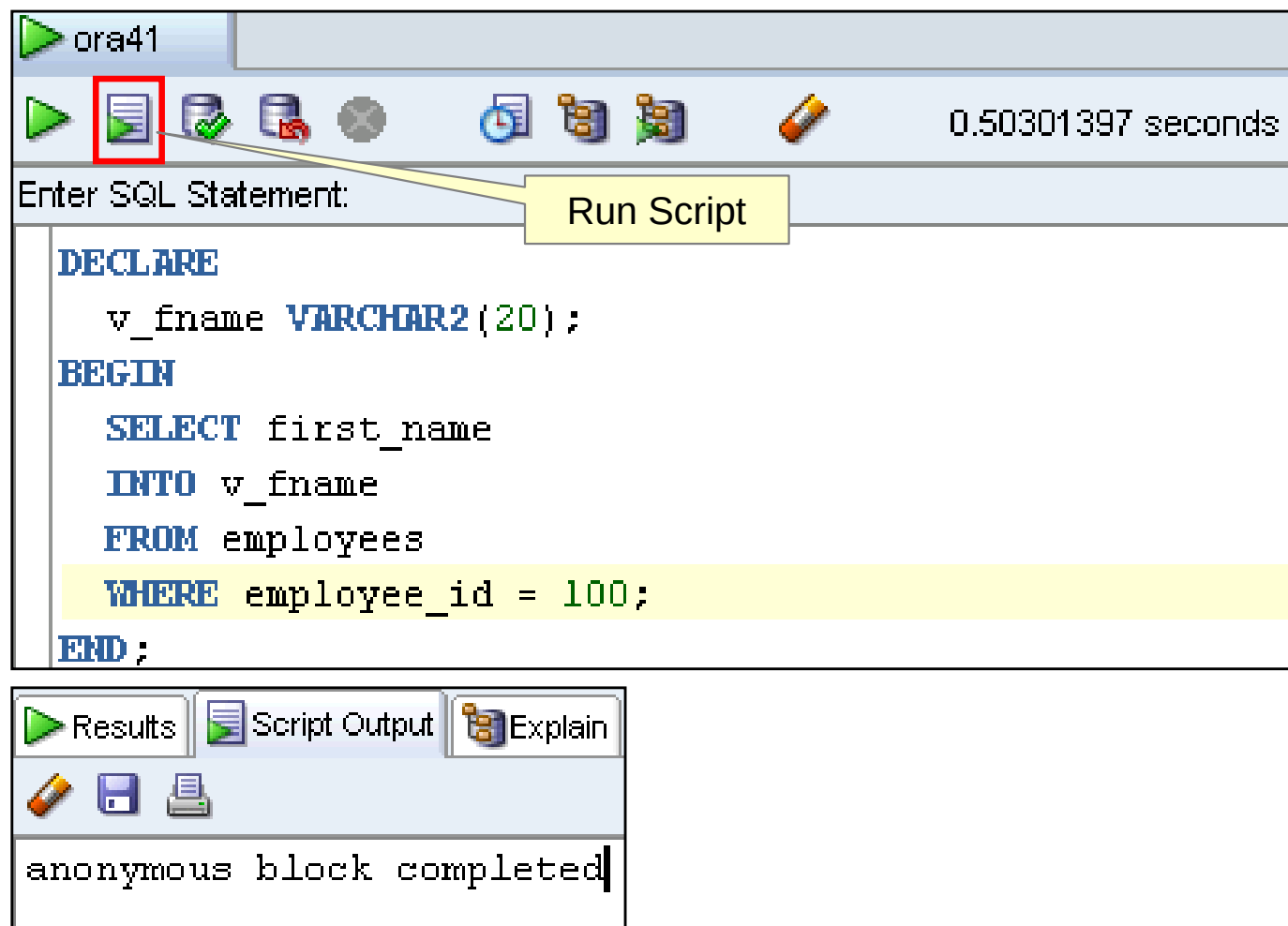


The screenshot shows the SQL Developer workspace with a single tab labeled 'ora41'. The toolbar includes icons for running, saving, and other database operations. The execution time is displayed as 0.50301397 seconds. The SQL statement entered is as follows:

```
DECLARE
  v_fname VARCHAR2(20);
BEGIN
  SELECT first_name
  INTO v_fname
  FROM employees
  WHERE employee_id = 100;
END;
```

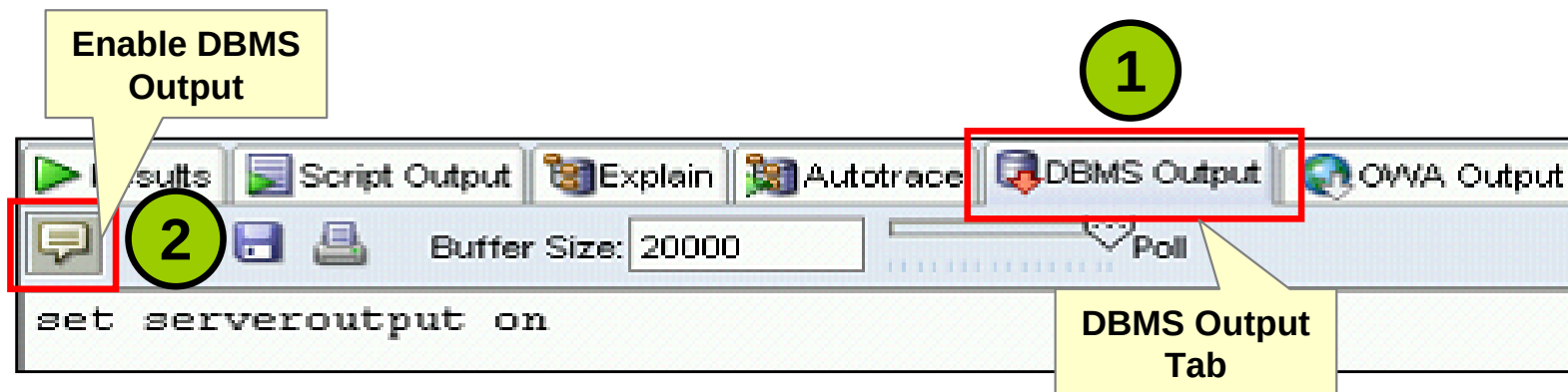
Execute an Anonymous Block

Click the Run Script button to execute the anonymous block:



Test the Output of a PL/SQL Block

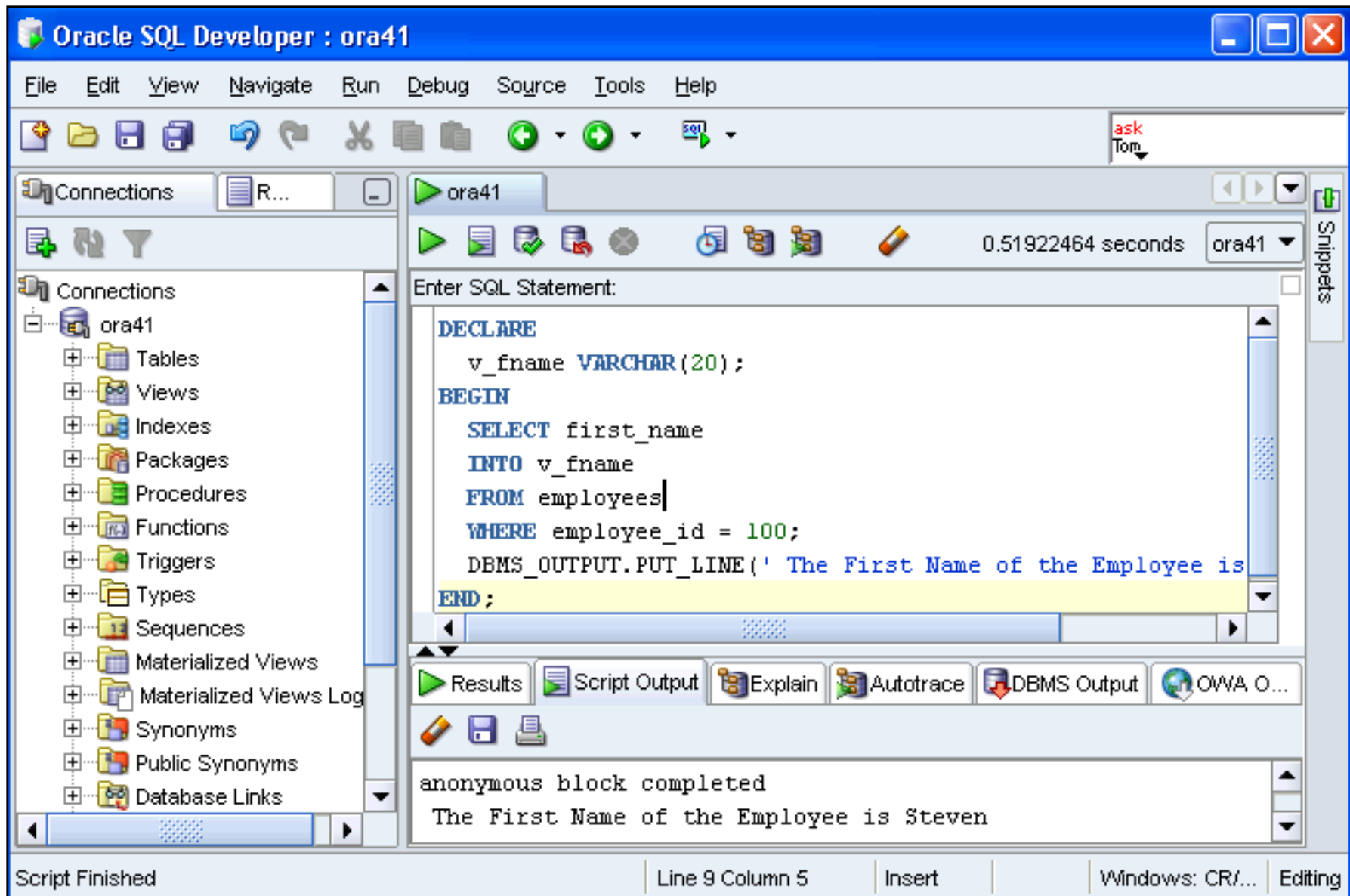
- Enable output in SQL Developer by clicking the Enable DBMS Output button on the DBMS Output tab:



- Use a predefined Oracle package and its procedure:
 - `DBMS_OUTPUT.PUT_LINE`

```
DBMS_OUTPUT.PUT_LINE(' The First Name of the  
Employee is ' || v_fname);  
...
```

Test the Output of a PL/SQL Block



Quiz

A PL/SQL block *must* consist of the following three sections:

- A Declarative section which begins with the keyword `DECLARE` and ends when the executable section starts.
- An Executable section which begins with the keyword `BEGIN` and ends with `END`.
- An Exception handling section which begins with the keyword `EXCEPTION` and is nested within the executable section.

1. True
2. False

Summary

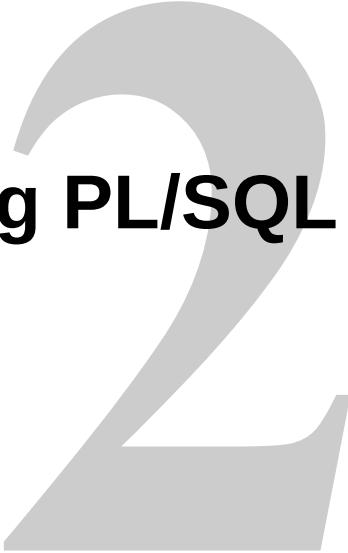
In this lesson, you should have learned how to:

- Integrate SQL statements with PL/SQL program constructs
- Describe the benefits of PL/SQL
- Differentiate between PL/SQL block types
- Output messages in PL/SQL

Practice 1: Overview

This practice covers the following topics:

- Identifying the PL/SQL blocks that execute successfully
- Creating and executing a simple PL/SQL block



Declaring PL/SQL Variables

Objectives

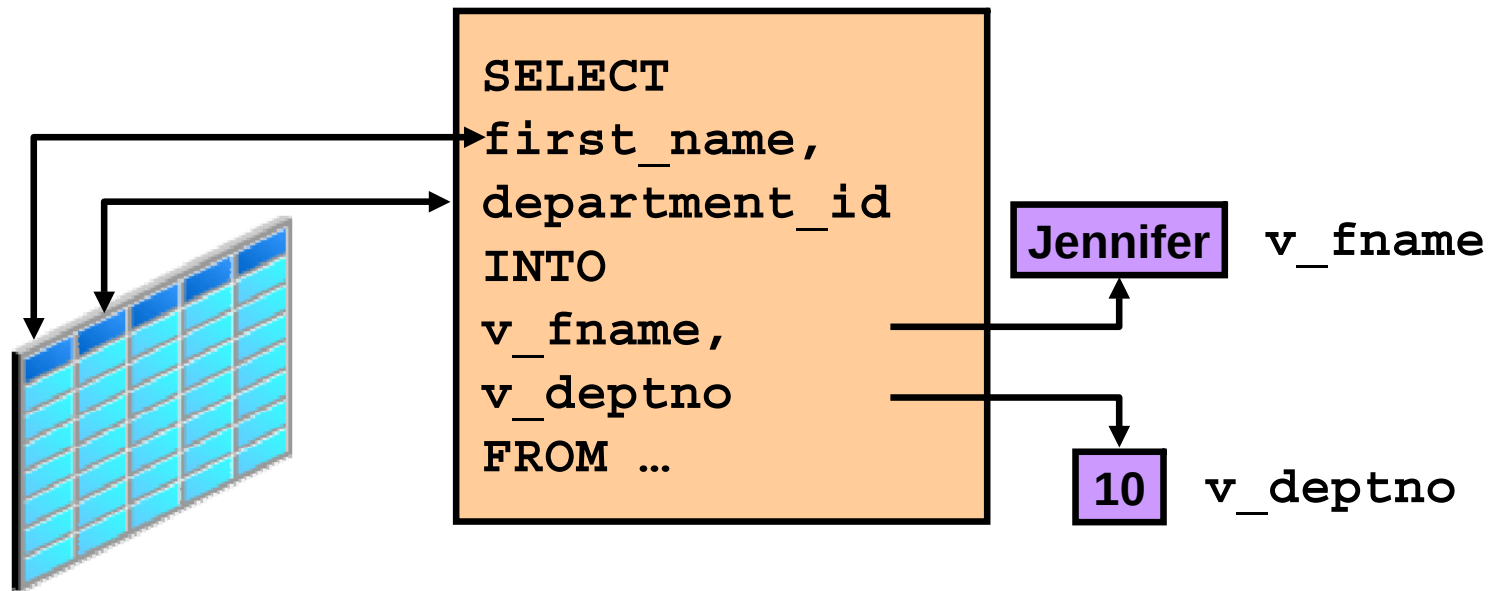
After completing this lesson, you should be able to do the following:

- Recognize valid and invalid identifiers
- List the uses of variables
- Declare and initialize variables
- List and describe various data types
- Identify the benefits of using the `%TYPE` attribute
- Declare, use, and print bind variables

Use of Variables

Variables can be used for:

- Temporary storage of data
- Manipulation of stored values
- Reusability



Requirements for Variable Names

A variable name:

- Must start with a letter
- Can include letters or numbers
- Can include special characters (such as \$, _, and #)
- Must contain no more than 30 characters
- Must not include reserved words



Handling Variables in PL/SQL

Variables are:

- Declared and initialized in the declarative section
- Used and assigned new values in the executable section
- Passed as parameters to PL/SQL subprograms
- Used to hold the output of a PL/SQL subprogram

Declaring and Initializing PL/SQL Variables

Syntax:

```
identifier [CONSTANT] datatype [NOT NULL]  
    [ := | DEFAULT expr ] ;
```

Examples:

```
DECLARE  
    v_hiredate      DATE;  
    v_deptno        NUMBER(2) NOT NULL := 10;  
    v_location      VARCHAR2(13) := 'Atlanta';  
    c_comm          CONSTANT NUMBER := 1400;
```

Declaring and Initializing PL/SQL Variables

1

```
DECLARE
    v_myName VARCHAR2(20);
BEGIN
    DBMS_OUTPUT.PUT_LINE('My name is: ' || v_myName);
    v_myName := 'John';
    DBMS_OUTPUT.PUT_LINE('My name is: ' || v_myName);
END;
/
```

2

```
DECLARE
    v_myName VARCHAR2(20) := 'John';
BEGIN
    v_myName := 'Steven';
    DBMS_OUTPUT.PUT_LINE('My name is: ' || v_myName);
END;
/
```


Delimiters in String Literals

```
DECLARE
    v_event VARCHAR2(15);
BEGIN
    v_event := q'!Father's day!';
    DBMS_OUTPUT.PUT_LINE('3rd Sunday in June is :
    ' || v_event );
    v_event := q'[Mother's day]';
    DBMS_OUTPUT.PUT_LINE('2nd Sunday in May is :
    ' || v_event );
END;
/
```

```
anonymous block completed
3rd Sunday in June is : Father's day
2nd Sunday in May is : Mother's day
```

Types of Variables

- PL/SQL variables:
 - Scalar
 - Composite
 - Reference
 - Large object (LOB)
- Non-PL/SQL variables: Bind variables

Types of Variables

TRUE



25-JAN-01

Snow White

Long, long ago,
in a land far, far away,
there lived a princess called
Snow White. . .

256120.08



Atlanta

Guidelines for Declaring and Initializing PL/SQL Variables

- Follow naming conventions.
- Use meaningful identifiers for variables.
- Initialize variables designated as NOT NULL and CONSTANT.
- Initialize variables with the assignment operator (:=) or the DEFAULT keyword:

```
v_myName VARCHAR2(20) := 'John';
```

```
v_myName VARCHAR2(20) DEFAULT 'John';
```

- Declare one identifier per line for better readability and code maintenance.

Guidelines for Declaring PL/SQL Variables

- Avoid using column names as identifiers.

```
DECLARE
  employee_id NUMBER(6);
BEGIN
  SELECT  employee_id
  INTO    employee_id
  FROM    employees
  WHERE   last_name = 'Kochhar';
END;
/
```



- Use the NOT NULL constraint when the variable must hold a value.

Scalar Data Types

- Hold a single value
- Have no internal components

TRUE

25-JAN-01

The soul of the lazy man
desires, and he has nothing;
but the soul of the diligent
shall be made rich.

256120.08

Atlanta

Base Scalar Data Types

- `CHAR [(maximum_length)]`
- `VARCHAR2 (maximum_length)`
- `NUMBER [(precision, scale)]`
- `BINARY_INTEGER`
- `PLS_INTEGER`
- `BOOLEAN`
- `BINARY_FLOAT`
- `BINARY_DOUBLE`

Base Scalar Data Types

- DATE
- TIMESTAMP
- TIMESTAMP WITH TIME ZONE
- TIMESTAMP WITH LOCAL TIME ZONE
- INTERVAL YEAR TO MONTH
- INTERVAL DAY TO SECOND

Declaring Scalar Variables

Examples:

```
DECLARE
  v_emp_job          VARCHAR2 (9) ;
  v_count_loop       BINARY_INTEGER := 0;
  v_dept_total_sal   NUMBER (9,2) := 0;
  v_orderdate        DATE := SYSDATE + 7;
  c_tax_rate         CONSTANT NUMBER (3,2) := 8.25;
  v_valid            BOOLEAN NOT NULL := TRUE;
  ...
```

%TYPE Attribute

- Is used to declare a variable according to:
 - A database column definition
 - Another declared variable
- Is prefixed with:
 - The database table and column names
 - The name of the declared variable

Declaring Variables with the %TYPE Attribute

Syntax

```
identifier      table.column_name%TYPE;
```

Examples

```
...  
  emp_lname      employees.last_name%TYPE;  
...
```

```
...  
  balance        NUMBER(7,2);  
  min_balance    balance%TYPE := 1000;  
...
```

Declaring Boolean Variables

- Only the `TRUE`, `FALSE`, and `NULL` values can be assigned to a Boolean variable.
- Conditional expressions use the logical operators `AND` and `OR` and the unary operator `NOT` to check the variable values.
- The variables always yield `TRUE`, `FALSE`, or `NULL`.
- Arithmetic, character, and date expressions can be used to return a Boolean value.

Bind Variables

Bind variables are:

- Created in the environment
- Also called *host variables*
- Created with the `VARIABLE` keyword
- Used in SQL statements and PL/SQL blocks
- Accessed even after the PL/SQL block is executed
- Referenced with a preceding colon

Printing Bind Variables

Example:

```
VARIABLE b_emp_salary NUMBER
BEGIN
    SELECT salary INTO :b_emp_salary
    FROM employees WHERE employee_id = 178;
END;
/
PRINT b_emp_salary
SELECT first_name, last_name FROM employees
WHERE salary=:b_emp_salary;
```

Printing Bind Variables

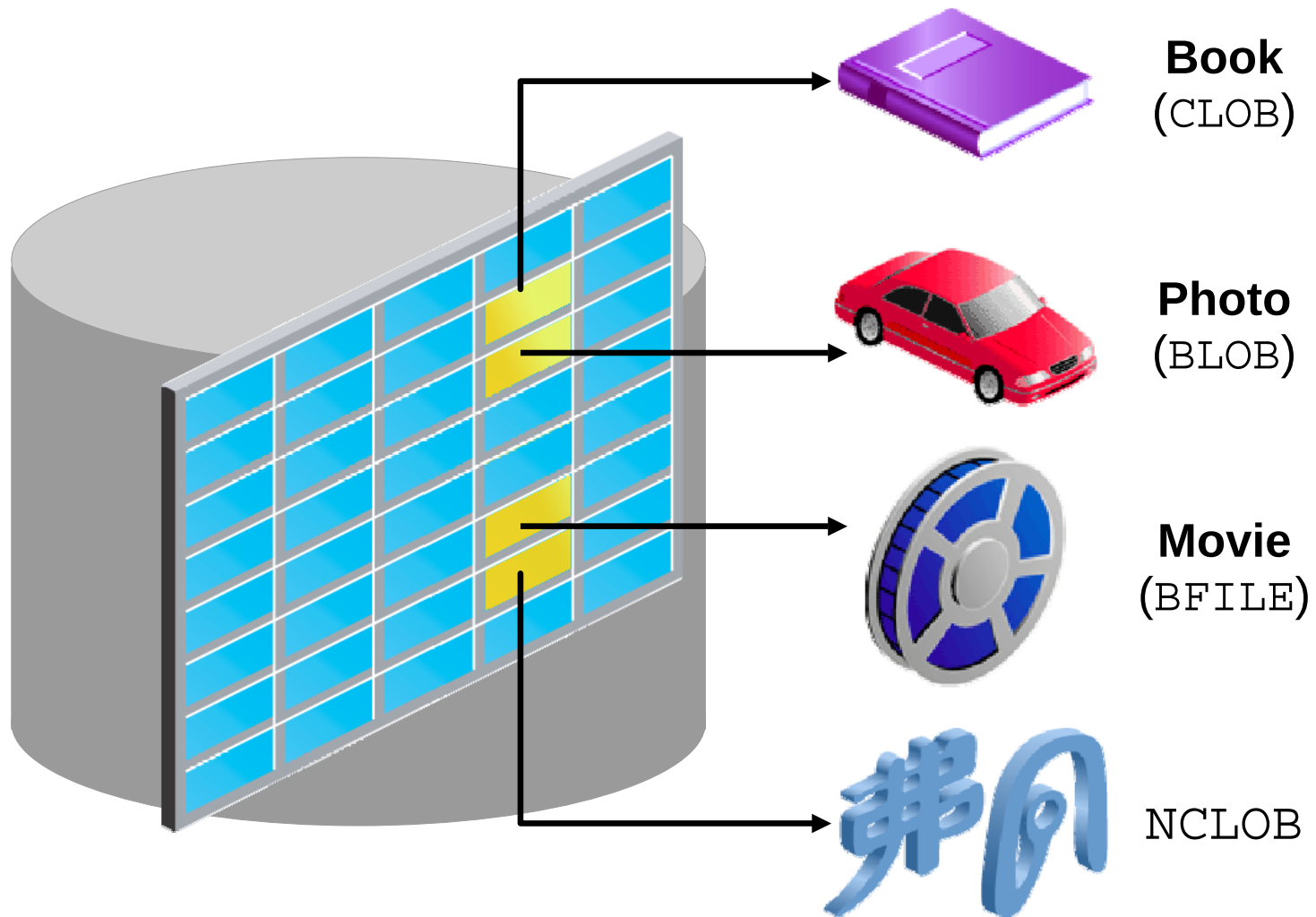
Example:

```
VARIABLE b_emp_salary NUMBER
SET AUTOPRINT ON
DECLARE
  v_empno NUMBER(6) := &empno;
BEGIN
  SELECT salary INTO :b_emp_salary
  FROM employees WHERE employee_id = v_empno;
END;
```

Output:

```
7000
```

LOB Data Type Variables



Composite Data Types

TRUE	23-DEC-98	ATLANTA	
------	-----------	---------	---

PL/SQL table structure

1	SMITH
2	JONES
3	NANCY
4	TIM

↑
PLS_INTEGER

↑
VARCHAR2

PL/SQL table structure

1	5000
2	2345
3	12
4	3456

↑
PLS_INTEGER

↑
NUMBER

Quiz

The %TYPE attribute:

1. Is used to declare a variable according to a database column definition
2. Is used to declare a variable according to a collection of columns in a database table or view
3. Is used to declare a variable according the definition of another declared variable
4. Is prefixed with the database table and column names or the name of the declared variable

Summary

In this lesson, you should have learned how to:

- Recognize valid and invalid identifiers
- Declare variables in the declarative section of a PL/SQL block
- Initialize variables and use them in the executable section
- Differentiate between scalar and composite data types
- Use the %TYPE attribute
- Use bind variables

Practice 2: Overview

This practice covers the following topics:

- Determining valid identifiers
- Determining valid variable declarations
- Declaring variables within an anonymous block
- Using the %TYPE attribute to declare variables
- Declaring and printing a bind variable
- Executing a PL/SQL block



Writing Executable Statements

Objectives

After completing this lesson, you should be able to do the following:

- Identify lexical units in a PL/SQL block
- Use built-in SQL functions in PL/SQL
- Describe when implicit conversions take place and when explicit conversions have to be dealt with
- Write nested blocks and qualify variables with labels
- Write readable code with appropriate indentation
- Use sequences in PL/SQL expressions

Lexical Units in a PL/SQL Block

Lexical units:

- Are building blocks of any PL/SQL block
- Are sequences of characters including letters, numerals, tabs, spaces, returns, and symbols
- Can be classified as:
 - Identifiers: `v_fname`, `c_percent`
 - Delimiters: `;`, `,`, `+`, `-`
 - Literals: `John`, `428`, `True`
 - Comments: `--`, `/* */`

PL/SQL Block Syntax and Guidelines

- Literals
 - Character and date literals must be enclosed in single quotation marks.
 - Numbers can be simple values or in scientific notation.

```
name := 'Henderson';
```

- Statements can span several lines.

The screenshot illustrates the process of formatting a PL/SQL block in SQL Developer. It is divided into three numbered sections:

- 1**: The initial PL/SQL code is entered into the editor:

```
DECLARE
v_fname VARCHAR2(20);
BEGIN
select first_name into v_fname
WHERE employee_id=100;
END;
```
- 2**: The 'Format SQL' menu option is selected from the 'Tools' menu. The menu items shown are: Clear (Ctrl-D), Cancel (Ctrl-Q), SQL History (F8), Cut (Ctrl-X), Copy (Ctrl-C), Paste (Ctrl-V), Select All (Ctrl-A), Query Builder, Describe (F4), and Format SQL... (Ctrl-B, highlighted with a red box).
- 3**: The result of the formatting, showing the code with proper indentation and line wrapping:

```
DECLARE
  v_fname VARCHAR2(20);
BEGIN
  SELECT first_name
  INTO v_fname
  FROM employees
  WHERE employee_id = 100;
END;
```


Commenting Code

- Prefix single-line comments with two hyphens (--).
- Place multiple-line comments between the symbols /* and */.

Example:

```
DECLARE
...
v_annual_sal NUMBER (9,2);
BEGIN
/* Compute the annual salary based on the
   monthly salary input from the user */
v_annual_sal := monthly_sal * 12;
--The following line displays the annual salary
DBMS_OUTPUT.PUT_LINE(v_annual_sal);
END;
/
```

SQL Functions in PL/SQL

- Available in procedural statements:
 - Single-row functions
- Not available in procedural statements:
 - DECODE
 - Group functions

SQL Functions in PL/SQL: Examples

- Get the length of a string:

```
v_desc_size INTEGER(5);  
v_prod_description VARCHAR2(70):='You can use this  
product with your radios for higher frequency';  
  
-- get the length of the string in prod description  
v_desc_size:= LENGTH(v_prod_description);
```

- Get the number of months an employee has worked:

```
v_tenure:= MONTHS_BETWEEN (CURRENT_DATE, v_hiredate);
```

Using Sequences in PL/SQL Expressions

Starting in 11g:

```
DECLARE
  v_new_id NUMBER;
BEGIN
  v_new_id := my_seq.NEXTVAL;
END;
/
```

Before 11g:

```
DECLARE
  v_new_id NUMBER;
BEGIN
  SELECT my_seq.NEXTVAL INTO v_new_id FROM Dual;
END;
/
```

Data Type Conversion

- Converts data to comparable data types
- Is of two types:
 - Implicit conversion
 - Explicit conversion
- Functions:
 - TO_CHAR
 - TO_DATE
 - TO_NUMBER
 - TO_TIMESTAMP

Data Type Conversion

1

```
date_of_joining DATE:= '02-Feb-2000';
```

2

```
date_of_joining DATE:= 'February 02,2000';
```

3

```
date_of_joining DATE:= TO_DATE('February  
02,2000','Month DD, YYYY');
```

Nested Blocks

PL/SQL blocks can be nested.

- An executable section (BEGIN ... END) can contain nested blocks.
- An exception section can contain nested blocks.



Nested Blocks: Example

```
DECLARE
  v_outer_variable VARCHAR2(20):='GLOBAL VARIABLE';
BEGIN
  DECLARE
    v_inner_variable VARCHAR2(20):='LOCAL VARIABLE';
  BEGIN
    DBMS_OUTPUT.PUT_LINE(v_inner_variable);
    DBMS_OUTPUT.PUT_LINE(v_outer_variable);
  END;
  DBMS_OUTPUT.PUT_LINE(v_outer_variable);
END;
```

```
anonymous block completed
LOCAL VARIABLE
GLOBAL VARIABLE
GLOBAL VARIABLE
```


Variable Scope and Visibility

```
DECLARE
  v_father_name VARCHAR2(20):='Patrick';
  v_date_of_birth DATE:='20-Apr-1972';
BEGIN
  DECLARE
    v_child_name VARCHAR2(20):='Mike';
    v_date_of_birth DATE:='12-Dec-2002';
  BEGIN
    DBMS_OUTPUT.PUT_LINE('Father's Name: ' || v_father_name);
    DBMS_OUTPUT.PUT_LINE('Date of Birth: ' || v_date_of_birth);
    DBMS_OUTPUT.PUT_LINE('Child's Name: ' || v_child_name);
  END;
  DBMS_OUTPUT.PUT_LINE('Date of Birth: ' || v_date_of_birth);
END;
/
```

1

2

Qualify an Identifier

```
BEGIN <<outer>>
DECLARE
  v_father_name VARCHAR2(20):='Patrick';
  v_date_of_birth DATE:='20-Apr-1972';
BEGIN
  DECLARE
    v_child_name VARCHAR2(20):='Mike';
    v_date_of_birth DATE:='12-Dec-2002';
  BEGIN
    DBMS_OUTPUT.PUT_LINE('Father's Name: ' || v_father_name);
    DBMS_OUTPUT.PUT_LINE('Date of Birth: '
                        || outer.v_date_of_birth);
    DBMS_OUTPUT.PUT_LINE('Child's Name: ' || v_child_name);
    DBMS_OUTPUT.PUT_LINE('Date of Birth: ' || v_date_of_birth);
  END;
END;
END outer;
```

Determining Variable Scope: Example

```
BEGIN <<outer>>
DECLARE
  v_sal      NUMBER(7,2) := 60000;
  v_comm     NUMBER(7,2) := v_sal * 0.20;
  v_message  VARCHAR2(255) := ' eligible for commission';
BEGIN
  DECLARE
    v_sal      NUMBER(7,2) := 50000;
    v_comm     NUMBER(7,2) := 0;
    v_total_comp NUMBER(7,2) := v_sal + v_comm;
  BEGIN
    v_message := 'CLERK not' || v_message;
    ① → outer.v_comm := v_sal * 0.30;
  END;
  v_message := 'SALESMAN' || v_message;
  ② → END;
END outer;
/
```

Operators in PL/SQL

- Logical
- Arithmetic
- Concatenation
- Parentheses to control order of operations

Same as in SQL

- Exponential operator (**)

Operators in PL/SQL: Examples

- Increment the counter for a loop.

```
loop_count := loop_count + 1;
```

- Set the value of a Boolean flag.

```
good_sal := sal BETWEEN 50000 AND 150000;
```

- Validate whether an employee number contains a value.

```
valid := (empno IS NOT NULL);
```

Programming Guidelines

Make code maintenance easier by:

- Documenting code with comments
- Developing a case convention for the code
- Developing naming conventions for identifiers and other objects
- Enhancing readability by indenting

Indenting Code

For clarity, indent each level of code.

```
BEGIN
  IF x=0 THEN
    y:=1;
  END IF;
END;
/
```

```
DECLARE
  deptno          NUMBER(4);
  location_id     NUMBER(4);
BEGIN
  SELECT  department_id,
          location_id
  INTO    deptno,
          location_id
  FROM    departments
  WHERE   department_name
          = 'Sales';

  ...
END;
/
```

Quiz

You can use most SQL single-row functions such as number, character, conversion, and date single-row functions in PL/SQL expressions.

1. True
2. False

Summary

In this lesson, you should have learned how to:

- Identify lexical units in a PL/SQL block
- Use built-in SQL functions in PL/SQL
- Write nested blocks to break logically related functionalities
- Decide when to perform explicit conversions
- Qualify variables in nested blocks
- Use sequences in PL/SQL expressions

Practice 3: Overview

This practice covers the following topics:

- Reviewing scoping and nesting rules
- Writing and testing PL/SQL blocks

4

Interacting with the Oracle Database Server

Objectives

After completing this lesson, you should be able to do the following:

- Determine the SQL statements that can be directly included in a PL/SQL executable block
- Manipulate data with DML statements in PL/SQL
- Use transaction control statements in PL/SQL
- Make use of the `INTO` clause to hold the values returned by a SQL statement
- Differentiate between implicit cursors and explicit cursors
- Use SQL cursor attributes

SQL Statements in PL/SQL

- Retrieve a row from the database by using the `SELECT` command.
- Make changes to rows in the database by using DML commands.
- Control a transaction with the `COMMIT`, `ROLLBACK`, or `SAVEPOINT` command.

SELECT Statements in PL/SQL

Retrieve data from the database with a `SELECT` statement.

Syntax:

```
SELECT  select_list
INTO    {variable_name[, variable_name]...
        | record_name}
FROM    table
[WHERE  condition];
```

SELECT Statements in PL/SQL

- The INTO clause is required.
- Queries must return only one row.

```
DECLARE
  v_fname VARCHAR2(25);
BEGIN
  SELECT first_name INTO v_fname
  FROM employees WHERE employee_id=200;
  DBMS_OUTPUT.PUT_LINE(' First Name is : ' || v_fname);
END;
/
```

```
anonymous block completed
First Name is : Jennifer
```

Retrieving Data in PL/SQL: Example

Retrieve `hire_date` and `salary` for the specified employee.

```
DECLARE
  v_emp_hiredate    employees.hire_date%TYPE;
  v_emp_salary      employees.salary%TYPE;
BEGIN
  SELECT    hire_date, salary
  INTO      v_emp_hiredate, v_emp_salary
  FROM      employees
  WHERE     employee_id = 100;
END;
/
```


Retrieving Data in PL/SQL

Return the sum of the salaries for all the employees in the specified department.

Example:

```
DECLARE
    v_sum_sal    NUMBER(10,2);
    v_deptno     NUMBER NOT NULL := 60;
BEGIN
    SELECT  SUM(salary)  -- group function
    INTO v_sum_sal  FROM employees
    WHERE    department_id = v_deptno;
    DBMS_OUTPUT.PUT_LINE ('The sum of salary is ' || v_sum_sal);
END;
```

```
anonymous block completed
The sum of salary is 28800
```

Naming Conventions

```
DECLARE
    hire_date      employees.hire_date%TYPE;
    sysdate        hire_date%TYPE;
    employee_id    employees.employee_id%TYPE := 176;
BEGIN
    SELECT          hire_date, sysdate
    INTO            hire_date, sysdate
    FROM            employees
    WHERE           employee_id = employee_id;
END;
/
```

Error report:

ORA-01422: exact fetch returns more than requested number of rows

ORA-06512: at line 6

01422. 00000 - "exact fetch returns more than requested number of rows"

*Cause: The number specified in exact fetch is less than the rows returned.

*Action: Rewrite the query or change number of rows requested

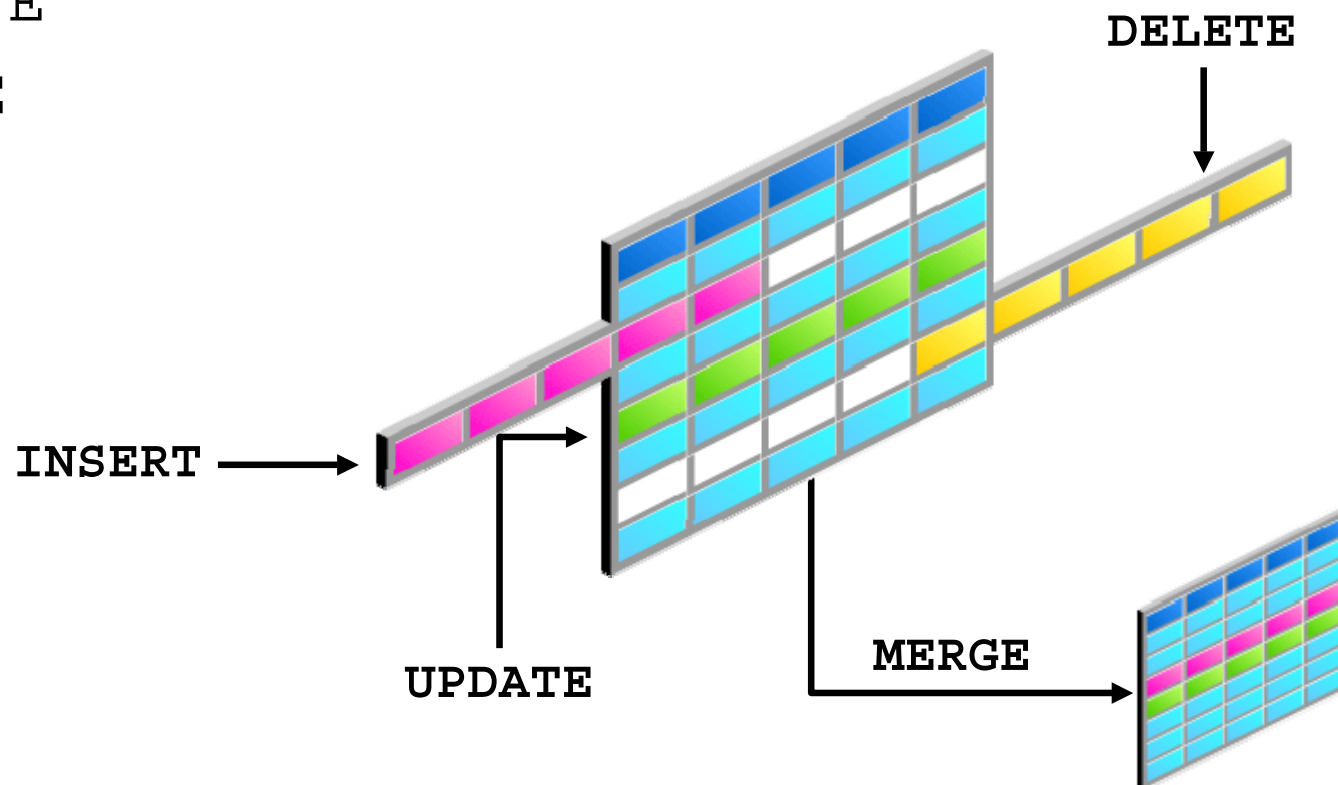
Naming Conventions

- Use a naming convention to avoid ambiguity in the `WHERE` clause.
- Avoid using database column names as identifiers.
- Syntax errors can arise because PL/SQL checks the database first for a column in the table.
- The names of local variables and formal parameters take precedence over the names of database *tables*.
- The names of database table *columns* take precedence over the names of local variables.

Using PL/SQL to Manipulate Data

Make changes to database tables by using DML commands:

- INSERT
- UPDATE
- DELETE
- MERGE



Inserting Data: Example

Add new employee information to the EMPLOYEES table.

```
BEGIN
  INSERT INTO employees
    (employee_id, first_name, last_name, email,
     hire_date, job_id, salary)
    VALUES (employees_seq.NEXTVAL, 'Ruth', 'Cores',
            'RCORES', CURRENT_DATE, 'AD_ASST', 4000);
END;
/
```

Updating Data: Example

Increase the salary of all employees who are stock clerks.

```
DECLARE
    sal_increase    employees.salary%TYPE := 800;
BEGIN
    UPDATE          employees
    SET              salary = salary + sal_increase
    WHERE            job_id = 'ST_CLERK';
END;
/
```

anonymous block completed

FIRST_NAME	SALARY
------------	--------

-----	-----
Julia	4000

Irene	3500
-------	------

James	3200
-------	------

Steven	3000
--------	------

...

Curtis	3900
--------	------

Randall	3400
---------	------

Peter	3300
-------	------

20 rows selected

Deleting Data: Example

Delete rows that belong to department 10 from the employees table.

```
DECLARE
    deptno    employees.department_id%TYPE := 10;
BEGIN
    DELETE FROM    employees
    WHERE    department_id = deptno;
END;
/
```

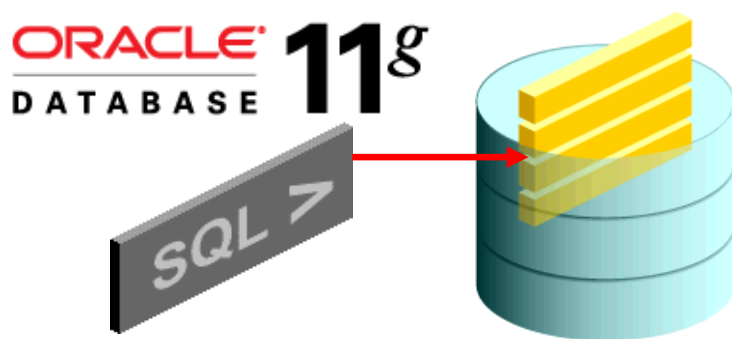
Merging Rows

Insert or update rows in the `copy_emp` table to match the `employees` table.

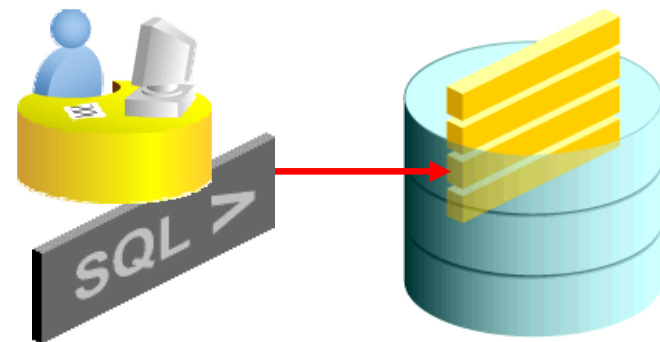
```
BEGIN
MERGE INTO copy_emp c
  USING employees e
  ON (e.employee_id = c.empno)
  WHEN MATCHED THEN
    UPDATE SET
      c.first_name      = e.first_name,
      c.last_name       = e.last_name,
      c.email           = e.email,
      . . .
  WHEN NOT MATCHED THEN
    INSERT VALUES(e.employee_id, e.first_name, e.last_name,
      . . ., e.department_id);
END;
/
```


SQL Cursor

- A cursor is a pointer to the private memory area allocated by the Oracle server. It is used to handle the result set of a `SELECT` statement.
- There are two types of cursors: Implicit and explicit.
 - **Implicit:** Created and managed internally by the Oracle server to process SQL statements
 - **Explicit:** Declared explicitly by the programmer



Implicit cursor



Explicit cursor

SQL Cursor Attributes for Implicit Cursors

Using SQL cursor attributes, you can test the outcome of your SQL statements.

SQL%FOUND	Boolean attribute that evaluates to <code>TRUE</code> if the most recent SQL statement returned at least one row
SQL%NOTFOUND	Boolean attribute that evaluates to <code>TRUE</code> if the most recent SQL statement did not return even one row
SQL%ROWCOUNT	An integer value that represents the number of rows affected by the most recent SQL statement

SQL Cursor Attributes for Implicit Cursors

Delete rows that have the specified employee ID from the employees table. Print the number of rows deleted.

Example:

```
DECLARE
  v_rows_deleted VARCHAR2(30)
  v_empno employees.employee_id%TYPE := 176;
BEGIN
  DELETE FROM employees
  WHERE employee_id = v_empno;
  v_rows_deleted := (SQL%ROWCOUNT ||
                    ' row deleted. ');
  DBMS_OUTPUT.PUT_LINE (v_rows_deleted);
END;
```

Quiz

When using the `SELECT` statement in PL/SQL, the `INTO` clause is required and queries can return one or more row.

1. True
2. False

Summary

In this lesson, you should have learned how to:

- Embed DML statements, transaction control statements, and DDL statements in PL/SQL
- Use the `INTO` clause, which is mandatory for all `SELECT` statements in PL/SQL
- Differentiate between implicit cursors and explicit cursors
- Use SQL cursor attributes to determine the outcome of SQL statements

Practice 4: Overview

This practice covers the following topics:

- Selecting data from a table
- Inserting data into a table
- Updating data in a table
- Deleting a record from a table

5

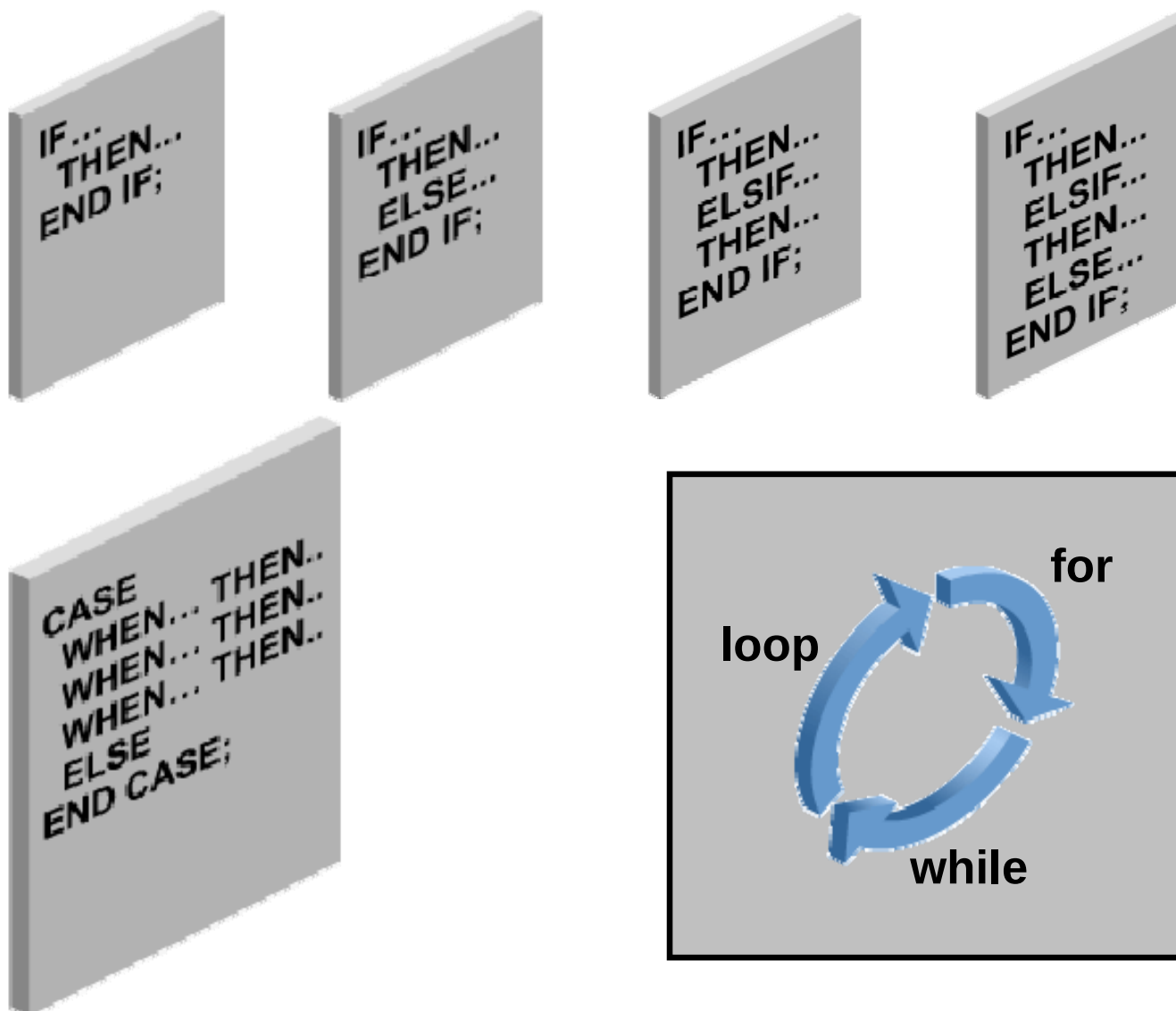
Writing Control Structures

Objectives

After completing this lesson, you should be able to do the following:

- Identify the uses and types of control structures
- Construct an `IF` statement
- Use `CASE` statements and `CASE` expressions
- Construct and identify loop statements
- Use guidelines when using conditional control structures

Controlling Flow of Execution



IF Statement

Syntax:

```
IF condition THEN  
    statements;  
[ELSIF condition THEN  
    statements;  
[ELSE  
    statements;  
END IF;
```

Simple IF Statement

```
DECLARE
  v_myage  number:=31;
BEGIN
  IF v_myage < 11
  THEN
    DBMS_OUTPUT.PUT_LINE(' I am a child ');
  END IF;
END;
/
```

anonymous block completed

IF THEN ELSE Statement

```
DECLARE
v_myage  number:=31;
BEGIN
IF v_myage < 11
  THEN
    DBMS_OUTPUT.PUT_LINE(' I am a child ');
  ELSE
    DBMS_OUTPUT.PUT_LINE(' I am not a child ');
END IF;
END;
/
```

```
anonymous block completed
I am not a child
```

IF ELIF ELSE Clause

```
DECLARE
  v_myage number:=31;
BEGIN
  IF v_myage < 11 THEN
    DBMS_OUTPUT.PUT_LINE(' I am a child ');
  ELIF v_myage < 20 THEN
    DBMS_OUTPUT.PUT_LINE(' I am young ');
  ELIF v_myage < 30 THEN
    DBMS_OUTPUT.PUT_LINE(' I am in my twenties');
  ELIF v_myage < 40 THEN
    DBMS_OUTPUT.PUT_LINE(' I am in my thirties');
  ELSE
    DBMS_OUTPUT.PUT_LINE(' I am always young ');
  END IF;
END;
/
```

```
anonymous block completed
I am in my thirties
```

NULL Value in IF Statement

```
DECLARE
  v_myage  number;
BEGIN
  IF v_myage < 11 THEN
    DBMS_OUTPUT.PUT_LINE(' I am a child ');
  ELSE
    DBMS_OUTPUT.PUT_LINE(' I am not a child ');
  END IF;
END;
/
```

```
anonymous block completed
I am not a child
```

CASE Expressions

- A CASE expression selects a result and returns it.
- To select the result, the CASE expression uses expressions. The value returned by these expressions is used to select one of several alternatives.

```
CASE selector
  WHEN expression1 THEN result1
  WHEN expression2 THEN result2
  ...
  WHEN expressionN THEN resultN
  [ELSE resultN+1]
END;
/
```

CASE Expressions: Example

```
SET VERIFY OFF
DECLARE
    v_grade CHAR(1) := UPPER('&grade');
    appraisal VARCHAR2(20);
BEGIN
    appraisal := CASE v_grade
        WHEN 'A' THEN 'Excellent'
        WHEN 'B' THEN 'Very Good'
        WHEN 'C' THEN 'Good'
        ELSE 'No such grade'
    END;
    DBMS_OUTPUT.PUT_LINE ('Grade: ' || v_grade || '
                          Appraisal ' || appraisal);
END;
/
```


Searched CASE Expressions

```
DECLARE
    v_grade CHAR(1) := UPPER('&grade');
    appraisal VARCHAR2(20);
BEGIN
    appraisal := CASE
        WHEN v_grade = 'A' THEN 'Excellent'
        WHEN v_grade IN ('B','C') THEN 'Good'
        ELSE 'No such grade'
    END;
    DBMS_OUTPUT.PUT_LINE ('Grade: ' || v_grade || '
                          Appraisal ' || appraisal);
END;
/
```

CASE Statement

```
DECLARE
    v_deptid NUMBER;
    v_deptname VARCHAR2(20);
    v_emps NUMBER;
    v_mngid NUMBER:= 108;
BEGIN
    CASE v_mngid
    WHEN 108 THEN
        SELECT department_id, department_name
        INTO v_deptid, v_deptname FROM departments
        WHERE manager_id=108;
        SELECT count(*) INTO v_emps FROM employees
        WHERE department_id=v_deptid;
    WHEN 200 THEN
        ...
    END CASE;
    DBMS_OUTPUT.PUT_LINE ('You are working in the ' || deptname ||
    ' department. There are ' || v_emps || ' employees in this
    department');
END;
/
```

Handling Nulls

When working with nulls, you can avoid some common mistakes by keeping in mind the following rules:

- Simple comparisons involving nulls always yield `NULL`.
- Applying the logical operator `NOT` to a null yields `NULL`.
- If the condition yields `NULL` in conditional control statements, its associated sequence of statements is not executed.

Logic Tables

Build a simple Boolean condition with a comparison operator.

AND	<i>TRUE</i>	<i>FALSE</i>	<i>NULL</i>	OR	<i>TRUE</i>	<i>FALSE</i>	<i>NULL</i>	NOT	
<i>TRUE</i>	TRUE	FALSE	NULL	<i>TRUE</i>	TRUE	TRUE	TRUE	<i>TRUE</i>	FALSE
<i>FALSE</i>	FALSE	FALSE	FALSE	<i>FALSE</i>	TRUE	FALSE	NULL	<i>FALSE</i>	TRUE
<i>NULL</i>	NULL	FALSE	NULL	<i>NULL</i>	TRUE	NULL	NULL	<i>NULL</i>	NULL

Boolean Conditions

What is the value of `flag` in each case?

```
flag := reorder_flag AND available_flag;
```

REORDER_FLAG	AVAILABLE_FLAG	FLAG
TRUE	TRUE	? (1)
TRUE	FALSE	? (2)
NULL	TRUE	? (3)
NULL	FALSE	? (4)

Iterative Control: LOOP Statements

- Loops repeat a statement (or sequence of statements) multiple times.
- There are three loop types:
 - Basic loop
 - FOR loop
 - WHILE loop



Basic Loops

Syntax:

```
LOOP  
    statement1;  
    . . .  
    EXIT [WHEN condition];  
END LOOP;
```

Basic Loops

Example:

```
DECLARE
  v_countryid      locations.country_id%TYPE := 'CA';
  v_loc_id         locations.location_id%TYPE;
  v_counter        NUMBER(2) := 1;
  v_new_city       locations.city%TYPE := 'Montreal';
BEGIN
  SELECT MAX(location_id) INTO v_loc_id FROM locations
  WHERE country_id = v_countryid;
  LOOP
    INSERT INTO locations(location_id, city, country_id)
    VALUES((v_loc_id + v_counter), v_new_city, v_countryid);
    v_counter := v_counter + 1;
    EXIT WHEN v_counter > 3;
  END LOOP;
END;
/
```


WHILE Loops

Syntax:

```
WHILE condition LOOP  
    statement1;  
    statement2;  
    . . .  
END LOOP;
```

Use the WHILE loop to repeat statements while a condition is TRUE.

WHILE Loops: Example

```
DECLARE
  v_countryid    locations.country_id%TYPE := 'CA';
  v_loc_id       locations.location_id%TYPE;
  v_new_city     locations.city%TYPE := 'Montreal';
  v_counter      NUMBER := 1;
BEGIN
  SELECT MAX(location_id) INTO v_loc_id FROM locations
  WHERE country_id = v_countryid;
  WHILE v_counter <= 3 LOOP
    INSERT INTO locations(location_id, city, country_id)
    VALUES((v_loc_id + v_counter), v_new_city, v_countryid);
    v_counter := v_counter + 1;
  END LOOP;
END;
/
```

FOR Loops

- Use a FOR loop to shortcut the test for the number of iterations.
- Do not declare the counter; it is declared implicitly.

```
FOR counter IN [REVERSE]
    lower_bound..upper_bound LOOP
    statement1;
    statement2;
    . . .
END LOOP;
```

FOR Loops: Example

```
DECLARE
  v_countryid    locations.country_id%TYPE := 'CA';
  v_loc_id       locations.location_id%TYPE;
  v_new_city     locations.city%TYPE := 'Montreal';
BEGIN
  SELECT MAX(location_id) INTO v_loc_id
    FROM locations
   WHERE country_id = v_countryid;
  FOR i IN 1..3 LOOP
    INSERT INTO locations(location_id, city, country_id)
      VALUES((v_loc_id + i), v_new_city, v_countryid );
  END LOOP;
END;
/
```

FOR Loops

Guidelines

- Reference the counter within the loop only; it is undefined outside the loop.
- Do not reference the counter as the target of an assignment.
- Neither loop bound should be NULL.

Guidelines for Loops

- Use the basic loop when the statements inside the loop must execute at least once.
- Use the `WHILE` loop if the condition must be evaluated at the start of each iteration.
- Use a `FOR` loop if the number of iterations is known.

Nested Loops and Labels

- You can nest loops to multiple levels.
- Use labels to distinguish between blocks and loops.
- Exit the outer loop with the `EXIT` statement that references the label.

Nested Loops and Labels

```
...
BEGIN
  <<Outer_loop>>
  LOOP
    v_counter := v_counter+1;
  EXIT WHEN v_counter>10;
    <<Inner_loop>>
    LOOP
      ...
      EXIT Outer_loop WHEN total_done = 'YES';
      -- Leave both loops
      EXIT WHEN inner_done = 'YES';
      -- Leave inner loop only
      ...
    END LOOP Inner_loop;
    ...
  END LOOP Outer_loop;
END;
/
```


PL/SQL CONTINUE Statement

- Definition
 - Adds the functionality to begin the next loop iteration
 - Provides programmers with the ability to transfer control to the next iteration of a loop
 - Uses parallel structure and semantics to the EXIT statement
- Benefits
 - Eases the programming process
 - May see a small performance improvement over the previous programming workarounds to simulate the CONTINUE statement



PL/SQL CONTINUE Statement: Example

```
DECLARE
  v_total SIMPLE_INTEGER := 0;
BEGIN
  FOR i IN 1..10 LOOP
    ① v_total := v_total + i;
      dbms_output.put_line
        ('Total is: ' || v_total);
      CONTINUE WHEN i > 5;
      v_total := v_total + i;
    ② dbms_output.put_line
        ('Out of Loop Total is:
          ' || v_total);
      END LOOP;
  END;
/
```



```
anonymous block completed
Total is: 1
Out of Loop Total is:
    2
Total is: 4
Out of Loop Total is:
    6
Total is: 9
Out of Loop Total is:
   12
Total is: 16
Out of Loop Total is:
   20
Total is: 25
Out of Loop Total is:
   30
Total is: 36
Total is: 43
Total is: 51
Total is: 60
Total is: 70
```

PL/SQL CONTINUE Statement: Example

```
DECLARE
  v_total NUMBER := 0;
BEGIN
  <<BeforeTopLoop>>
  FOR i IN 1..10 LOOP
    v_total := v_total + 1;
    dbms_output.put_line
      ('Total is: ' || v_total);
    FOR j IN 1..10 LOOP
      CONTINUE BeforeTopLoop WHEN i + j > 5;
      v_total := v_total + 1;
    END LOOP;
  END LOOP;
END two_loop;
```

```
anonymous block completed
Total is: 1
Total is: 6
Total is: 10
Total is: 13
Total is: 15
Total is: 16
Total is: 17
Total is: 18
Total is: 19
Total is: 20
```

Quiz

There are three types of loops: Basic, FOR, and WHILE.

1. True
2. False

Summary

In this lesson, you should have learned how to change the logical flow of statements by using the following control structures:

- Conditional (IF statement)
- CASE expressions and CASE statements
- Loops:
 - Basic loop
 - FOR loop
 - WHILE loop
- EXIT statement
- CONTINUE statement

Practice 5: Overview

This practice covers the following topics:

- Performing conditional actions by using `IF` statements
- Performing iterative steps by using `LOOP` structures



Working with Composite Data Types

Objectives

After completing this lesson, you should be able to do the following:

- Create user-defined PL/SQL records
- Create a record with the %ROWTYPE attribute
- Create an INDEX BY table
- Create an INDEX BY table of records
- Describe the differences among records, tables, and tables of records

Composite Data Types

- Can hold multiple values (unlike scalar types)
- Are of two types:
 - PL/SQL records
 - PL/SQL collections
 - INDEX BY tables or associative arrays
 - Nested table
 - VARRAY

Composite Data Types

- Use PL/SQL records when you want to store values of different data types but only one occurrence at a time.
- Use PL/SQL collections when you want to store values of the same data type.

PL/SQL Records

- Must contain one or more components (called *fields*) of any scalar, RECORD, or INDEX BY table data type
- Are similar to structures in most third-generation languages (including C and C++)
- Are user defined and can be a subset of a row in a table
- Treat a collection of fields as a logical unit
- Are convenient for fetching a row of data from a table for processing

%ROWTYPE Attribute

- Declare a variable according to a collection of columns in a database table or view.
- Prefix %ROWTYPE with the database table or view.
- Fields in the record take their names and data types from the columns of the table or view.

Syntax:

```
DECLARE  
    identifier reference%ROWTYPE;
```

Advantages of Using the %ROWTYPE Attribute

- The number and data types of the underlying database columns need not be known—and, in fact, might change at run time.
- The %ROWTYPE attribute is useful when retrieving a row with the `SELECT *` statement.

Creating a PL/SQL Record

Syntax:

1

```
TYPE type_name IS RECORD  
    (field_declaration [, field_declaration]...);
```

2

```
identifier    type_name;
```

field_declaration:

```
field_name {field_type | variable%TYPE  
            | table.column%TYPE | table%ROWTYPE}  
            [[NOT NULL] {:= | DEFAULT} expr]
```

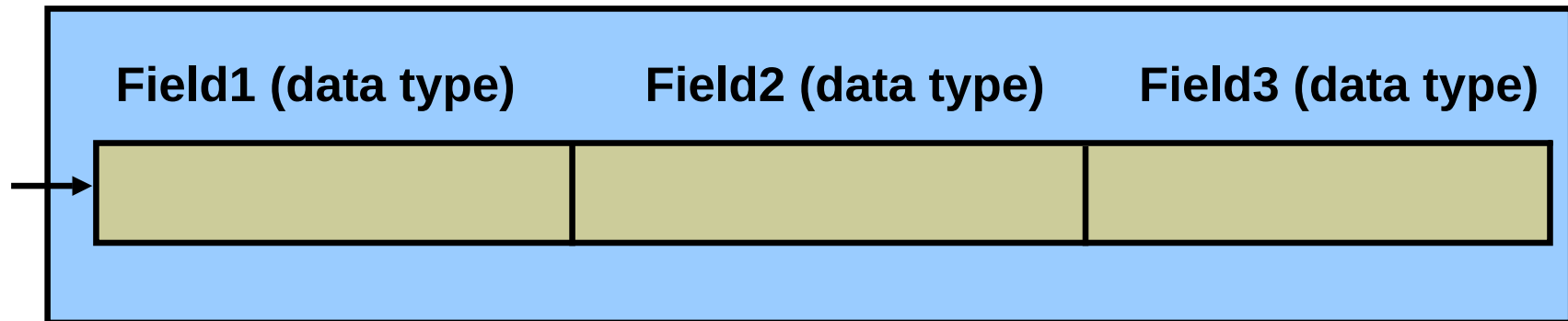
Creating a PL/SQL Record: Example

Declare variables to store the name, job, and salary of a new employee.

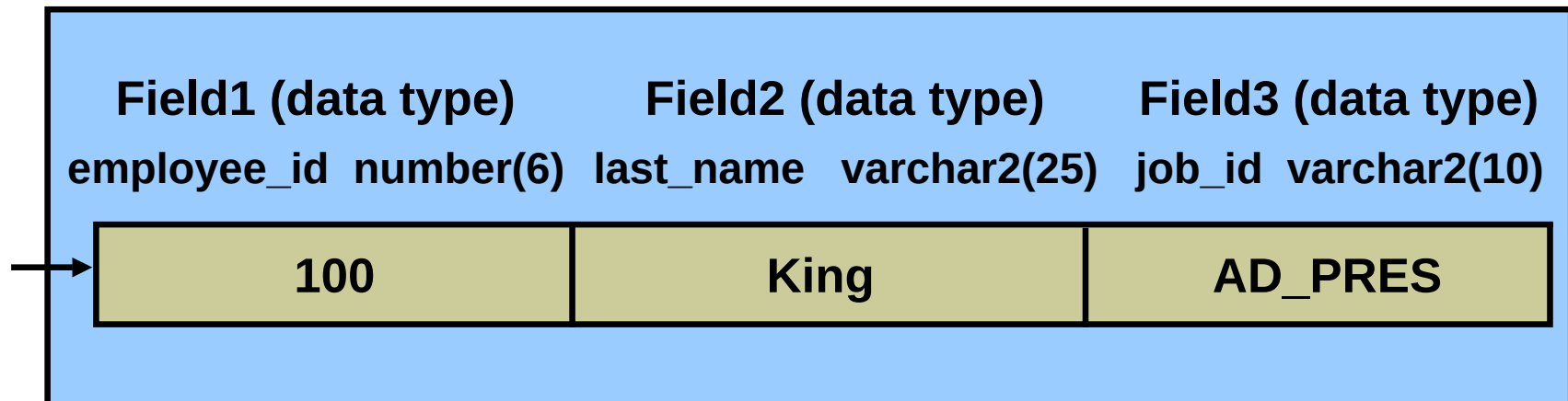
```
DECLARE
  type t_rec is record
    (v_sal number(8),
     v_minsal number(8) default 1000,
     v_hire_date employees.hire_date%type,
     v_rec1 employees%rowtype);
  v_myrec t_rec;
BEGIN
  v_myrec.v_sal := v_myrec.v_minsal + 500;
  v_myrec.v_hire_date := sysdate;
  SELECT * INTO v_myrec.v_rec1
    FROM employees WHERE employee_id = 100;
  DBMS_OUTPUT.PUT_LINE(v_myrec.v_rec1.last_name || ' ' ||
    to_char(v_myrec.v_hire_date) || ' ' || to_char(v_myrec.v_sal));
END;
```

```
anonymous block completed
King 16-FEB-09 1500
```

PL/SQL Record Structure



Example:



%ROWTYPE Attribute: Example

```
DECLARE
    v_employee_number number := 124;
    v_emp_rec      employees%ROWTYPE;
BEGIN
    SELECT * INTO v_emp_rec FROM employees
    WHERE  employee_id = v_employee_number;
    INSERT INTO retired_ems(empno, ename, job, mgr,
                           hiredate, leavedate, sal, comm, deptno)
    VALUES (v_emp_rec.employee_id, v_emp_rec.last_name,
            v_emp_rec.job_id, v_emp_rec.manager_id,
            v_emp_rec.hire_date, SYSDATE,
            v_emp_rec.salary, v_emp_rec.commission_pct,
            v_emp_rec.department_id);
END;
/
```

	 EMPNO	 ENAME	 JOB	 MGR	HIREDATE	LEAVEDATE	 SAL	 COMM	 DEPTNO
1	124	Mourgos	ST_MAN	100	16-NOV-99	16-FEB-09	5800	(null)	50

Inserting a Record by Using %ROWTYPE

```
...  
DECLARE  
    v_employee_number number:= 124;  
    v_emp_rec retired_emps%ROWTYPE;  
BEGIN  
    SELECT employee_id, last_name, job_id, manager_id,  
           hire_date, hire_date, salary, commission_pct,  
           department_id INTO v_emp_rec FROM employees  
    WHERE   employee_id = v_employee_number;  
    INSERT INTO retired_emps VALUES v_emp_rec;  
END;  
/  
SELECT * FROM retired_emps;
```

	 EMPNO	 ENAME	 JOB	 MGR	HIREDATE	LEAVEDATE	 SAL	 COMM	 DEPTNO
1	124	Mourgos	ST_MAN	100	16-NOV-99	16-NOV-99	5800	(null)	50

Updating a Row in a Table by Using a Record

```
SET VERIFY OFF
DECLARE
    v_employee_number number := 124;
    v_emp_rec retired_emps%ROWTYPE;
BEGIN
    SELECT * INTO v_emp_rec FROM retired_emps;
    v_emp_rec.leavedate:=CURRENT_DATE;
    UPDATE retired_emps SET ROW = v_emp_rec WHERE
        empno=v_employee_number;
END;
/
SELECT * FROM retired_emps;
```

	EMPNO	ENAME	JOB	MGR	HIREDATE	LEAVEDATE	SAL	COMM	DEPTNO
1	124	Mourgos	ST_MAN	100	16-NOV-99	16-FEB-09	5800	(null)	50

INDEX BY Tables or Associative Arrays

- Are PL/SQL structures with two columns:
 - Primary key of integer or string data type
 - Column of scalar or record data type
- Are unconstrained in size. However, the size depends on the values that the key data type can hold.

Creating an INDEX BY Table

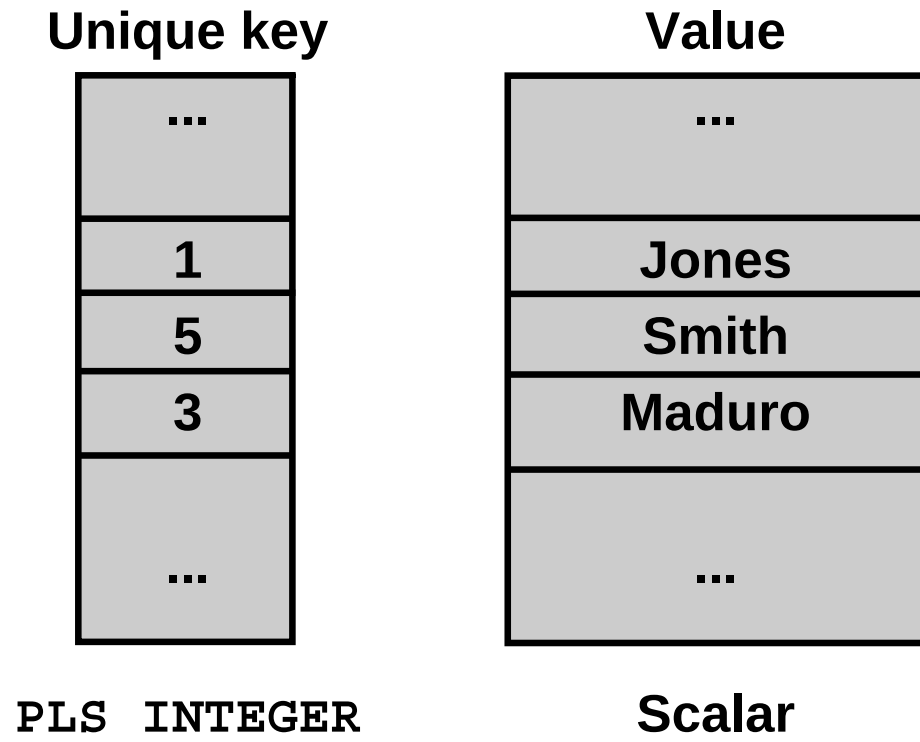
Syntax:

```
TYPE type_name IS TABLE OF
    {column_type | variable%TYPE
    | table.column%TYPE} [NOT NULL]
    | table%ROWTYPE
    [INDEX BY PLS_INTEGER | BINARY_INTEGER
    | VARCHAR2(<size>)];
identifier    type_name;
```

Declare an INDEX BY table to store the last names of employees:

```
...
TYPE ename_table_type IS TABLE OF
    employees.last_name%TYPE
    INDEX BY PLS_INTEGER;
...
ename_table ename_table_type;
```

INDEX BY Table Structure



Creating an INDEX BY Table

```
DECLARE
  TYPE ename_table_type IS TABLE OF
    employees.last_name%TYPE
    INDEX BY PLS_INTEGER;
  TYPE hiredate_table_type IS TABLE OF DATE
    INDEX BY PLS_INTEGER;
  ename_table          ename_table_type;
  hiredate_table       hiredate_table_type;
BEGIN
  ename_table(1)       := 'CAMERON';
  hiredate_table(8)    := SYSDATE + 7;
  IF ename_table.EXISTS(1) THEN
    INSERT INTO ...
    ...
END;
/
```

	ENAME	HIREDT
1	CAMERON	23-FEB-09

Using INDEX BY Table Methods

The following methods make INDEX BY tables easier to use:

- EXISTS
- COUNT
- FIRST
- LAST
- PRIOR
- NEXT
- DELETE

INDEX BY Table of Records

Define an INDEX BY table variable to hold an entire row from a table.

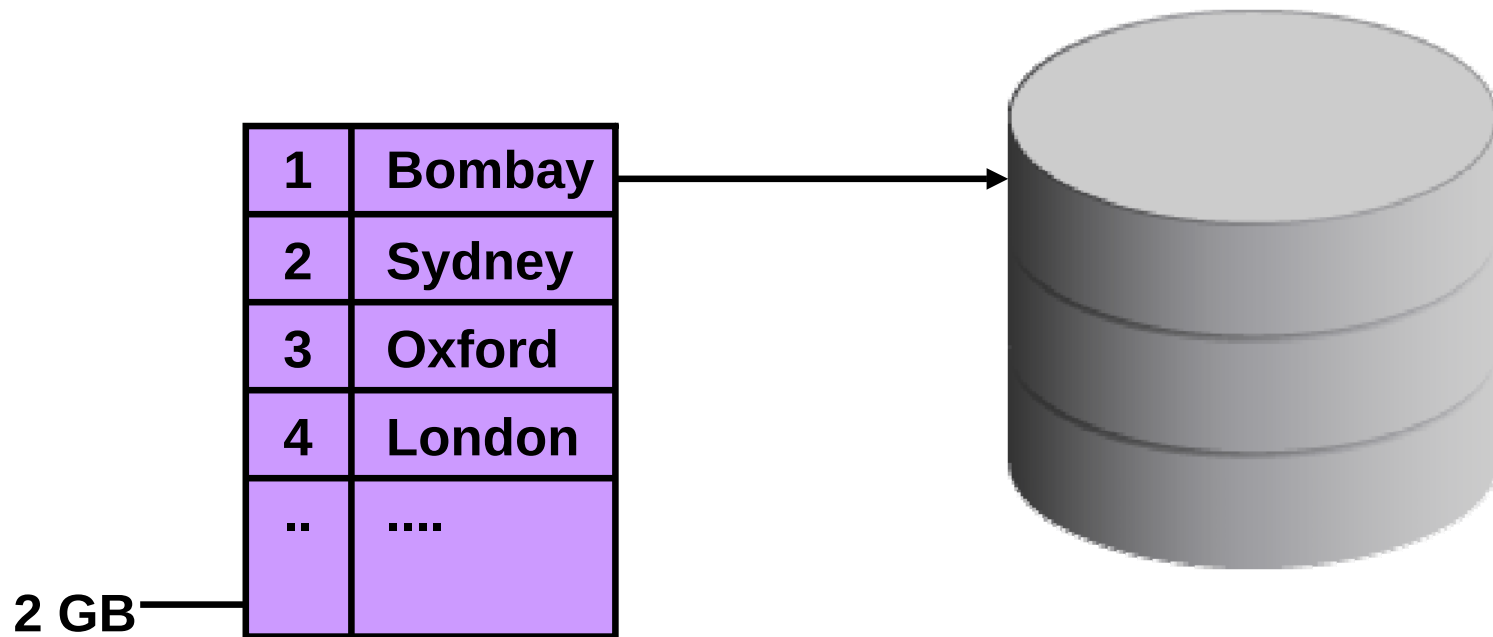
```
DECLARE
  TYPE dept_table_type IS TABLE OF
    departments%ROWTYPE
    INDEX BY VARCHAR2(20);
  dept_table dept_table_type;
  -- Each element of dept_table is a record
  . . .
```

```
anonymous block completed
Administration
```

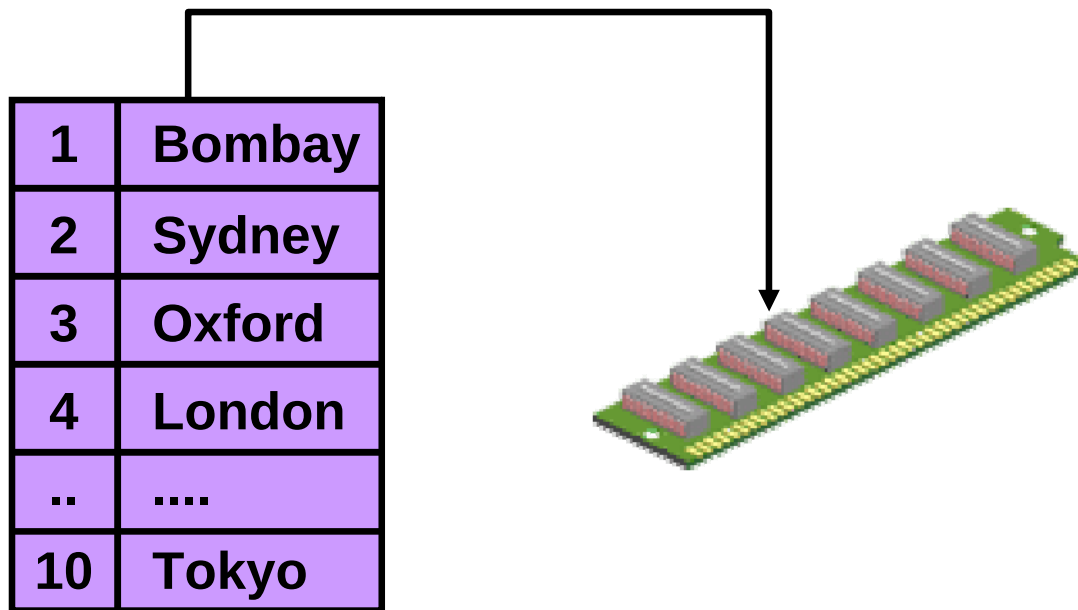
INDEX BY Table of Records: Example

```
DECLARE
    TYPE emp_table_type IS TABLE OF
        employees%ROWTYPE INDEX BY PLS_INTEGER;
    my_emp_table    emp_table_type;
    max_count       NUMBER(3) := 104;
BEGIN
    FOR i IN 100..max_count
    LOOP
        SELECT * INTO my_emp_table(i) FROM employees
        WHERE employee_id = i;
    END LOOP;
    FOR i IN my_emp_table.FIRST..my_emp_table.LAST
    LOOP
        DBMS_OUTPUT.PUT_LINE(my_emp_table(i).last_name);
    END LOOP;
END;
/
```

Nested Tables



VARRAY



Quiz

Use the %ROW attribute in the following situations:

1. When you are not sure about the structure of the underlying database table
2. When you want to retrieve an entire row from a table
3. When you want to declare a variable according to another previously declared variable or database column

Summary

In this lesson, you should have learned how to:

- Define and reference PL/SQL variables of composite data types
 - PL/SQL record
 - INDEX BY table
 - INDEX BY table of records
- Define a PL/SQL record by using the %ROWTYPE attribute

Practice 6: Overview

This practice covers the following topics:

- Declaring `INDEX BY` tables
- Processing data by using `INDEX BY` tables
- Declaring a PL/SQL record
- Processing data by using a PL/SQL record



Using Explicit Cursors

Objectives

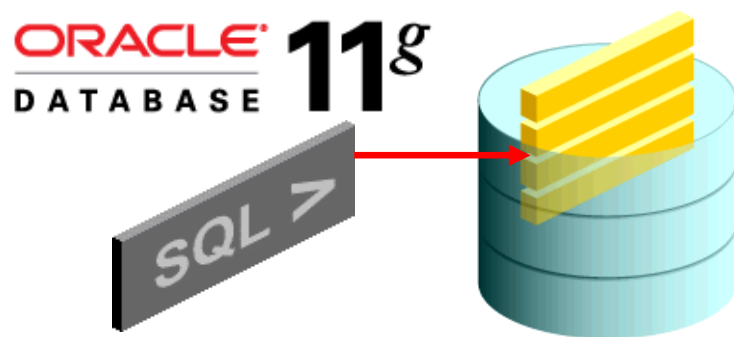
After completing this lesson, you should be able to do the following:

- Distinguish between implicit and explicit cursors
- Discuss the reasons for using explicit cursors
- Declare and control explicit cursors
- Use simple loops and cursor `FOR` loops to fetch data
- Declare and use cursors with parameters
- Lock rows with the `FOR UPDATE` clause
- Reference the current row with the `WHERE CURRENT OF` clause

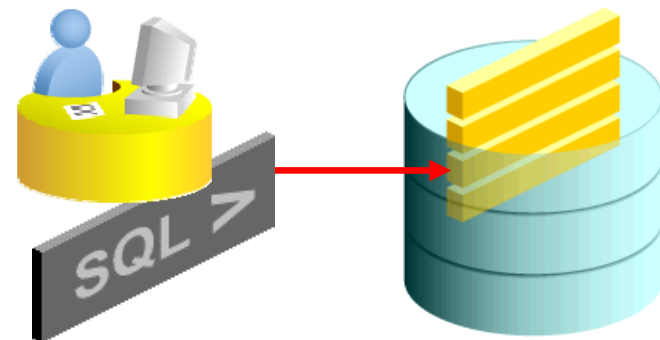
Cursors

Every SQL statement executed by the Oracle server has an associated individual cursor:

- Implicit cursors: Declared and managed by PL/SQL for all DML and PL/SQL `SELECT` statements
- Explicit cursors: Declared and managed by the programmer

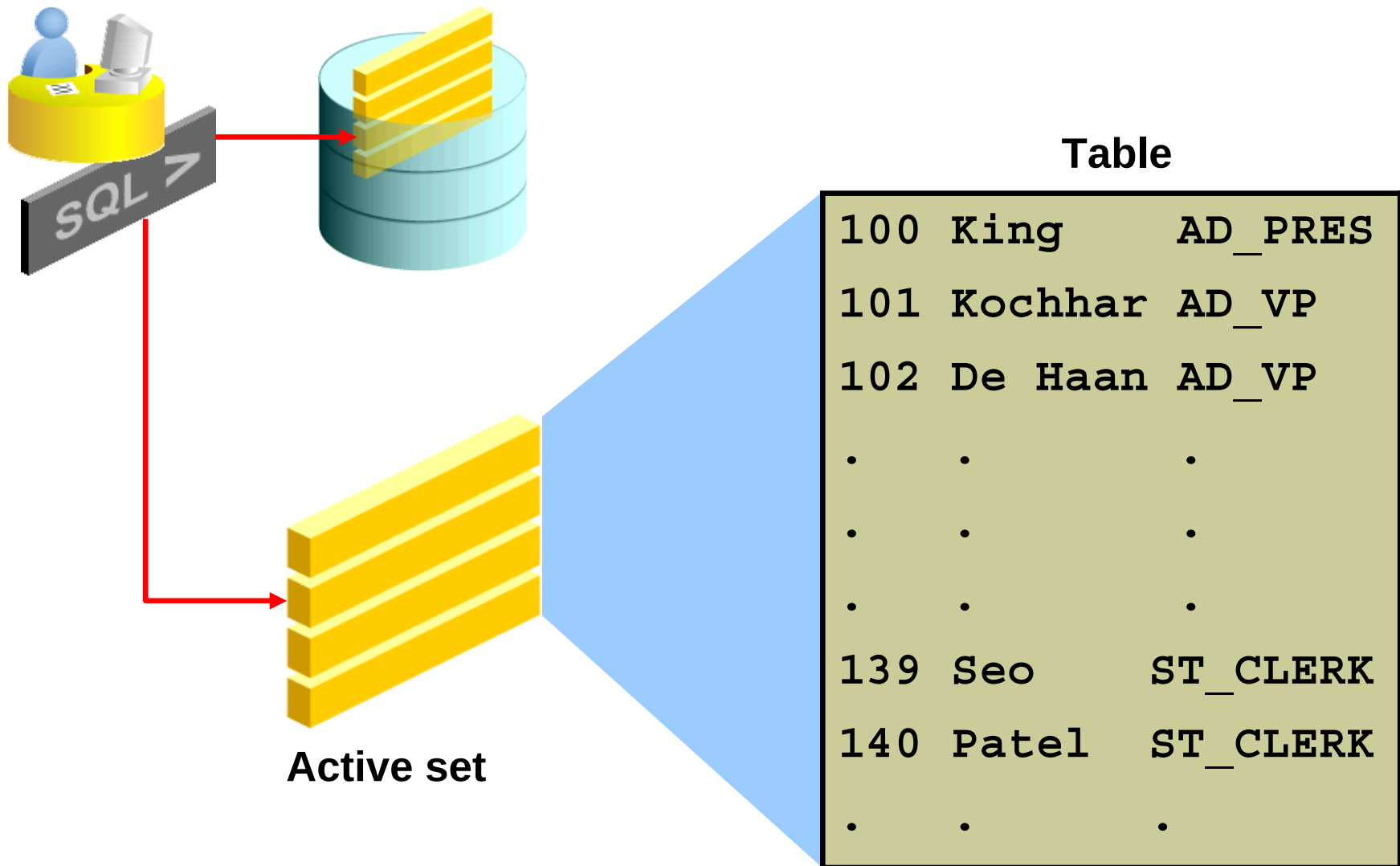


Implicit cursor

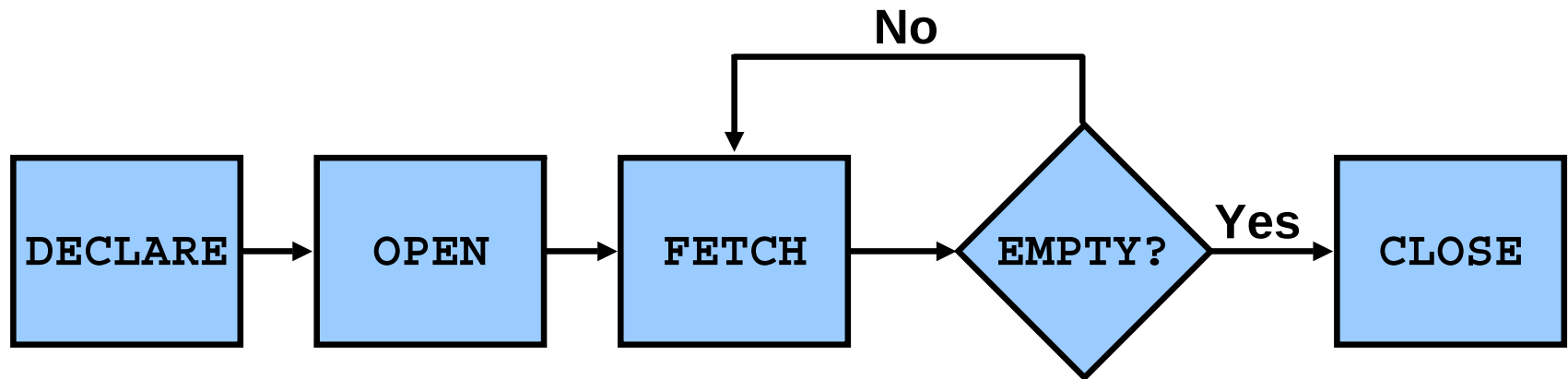


Explicit cursor

Explicit Cursor Operations



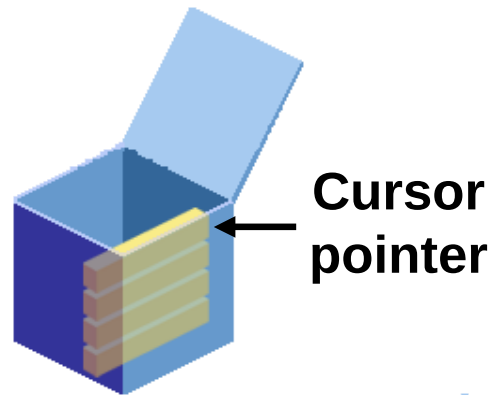
Controlling Explicit Cursors



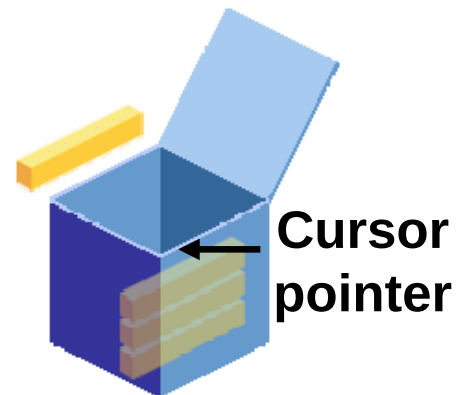
- Create a named SQL area.
- Identify the active set.
- Load the current row into variables.
- Test for existing rows.
- Release the active set.
- Return to FETCH if rows are found.

Controlling Explicit Cursors

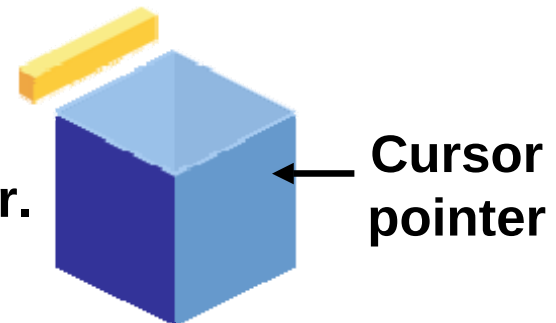
1 Open the cursor.



2 Fetch a row.



3 Close the cursor.



Declaring the Cursor

Syntax:

```
CURSOR cursor_name IS  
    select_statement;
```

Examples:

```
DECLARE  
    CURSOR c_emp_cursor IS  
        SELECT employee_id, last_name FROM employees  
        WHERE department_id =30;
```

```
DECLARE  
    v_locid NUMBER:= 1700;  
    CURSOR c_dept_cursor IS  
        SELECT * FROM departments  
        WHERE location_id = v_locid;  
    ...
```

Opening the Cursor

```
DECLARE
  CURSOR c_emp_cursor IS
    SELECT employee_id, last_name FROM employees
    WHERE department_id =30;
  ...
BEGIN
  OPEN c_emp_cursor;
```

Fetching Data from the Cursor

```
DECLARE
  CURSOR c_emp_cursor IS
    SELECT employee_id, last_name FROM employees
    WHERE department_id =30;
  v_empno employees.employee_id%TYPE;
  v_lname employees.last_name%TYPE;
BEGIN
  OPEN c_emp_cursor;
  FETCH c_emp_cursor INTO v_empno, v_lname;
  DBMS_OUTPUT.PUT_LINE( v_empno || ' ' || v_lname);
END;
/
```

```
anonymous block completed
114 Raphaely
```


Fetching Data from the Cursor

```
DECLARE
  CURSOR c_emp_cursor IS
    SELECT employee_id, last_name FROM employees
    WHERE department_id =30;
  v_empno employees.employee_id%TYPE;
  v_lname employees.last_name%TYPE;
BEGIN
  OPEN c_emp_cursor;
  LOOP
    FETCH c_emp_cursor INTO v_empno, v_lname;
    EXIT WHEN c_emp_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE( v_empno || ' ' || v_lname);
  END LOOP;
END;
/
```

Closing the Cursor

```
...  
  LOOP  
    FETCH c_emp_cursor INTO empno, lname;  
    EXIT WHEN c_emp_cursor%NOTFOUND;  
    DBMS_OUTPUT.PUT_LINE( v_empno || ' ' || v_lname);  
  END LOOP;  
  CLOSE c_emp_cursor;  
END;  
/
```

Cursors and Records

Process the rows of the active set by fetching values into a PL/SQL record.

```
DECLARE
  CURSOR c_emp_cursor IS
    SELECT employee_id, last_name FROM employees
    WHERE department_id =30;
  v_emp_record  c_emp_cursor%ROWTYPE;
BEGIN
  OPEN c_emp_cursor;
  LOOP
    FETCH c_emp_cursor INTO v_emp_record;
    EXIT WHEN c_emp_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE( v_emp_record.employee_id
                          || ' ' || v_emp_record.last_name);
  END LOOP;
  CLOSE c_emp_cursor;
END;
```

Cursor FOR Loops

Syntax:

```
FOR record_name IN cursor_name LOOP  
    statement1;  
    statement2;  
    . . .  
END LOOP;
```

- The cursor FOR loop is a shortcut to process explicit cursors.
- Implicit open, fetch, exit, and close occur.
- The record is implicitly declared.

Cursor FOR Loops

```
DECLARE
  CURSOR c_emp_cursor IS
    SELECT employee_id, last_name FROM employees
    WHERE department_id =30;
BEGIN
  FOR emp_record IN c_emp_cursor
  LOOP
    DBMS_OUTPUT.PUT_LINE( emp_record.employee_id
      || ' ' || emp_record.last_name);
  END LOOP;
END;
/
```

```
anonymous block completed
114 Raphaely
115 Khoo
116 Baida
117 Tobias
118 Himuro
119 Colmenares
```

Explicit Cursor Attributes

Use explicit cursor attributes to obtain status information about a cursor.

Attribute	Type	Description
%ISOPEN	Boolean	Evaluates to TRUE if the cursor is open
%NOTFOUND	Boolean	Evaluates to TRUE if the most recent fetch does not return a row
%FOUND	Boolean	Evaluates to TRUE if the most recent fetch returns a row; complement of %NOTFOUND
%ROWCOUNT	Number	Evaluates to the total number of rows returned so far

%ISOPEN Attribute

- Fetch rows only when the cursor is open.
- Use the %ISOPEN cursor attribute before performing a fetch to test whether the cursor is open.

Example:

```
IF NOT c_emp_cursor%ISOPEN THEN
    OPEN c_emp_cursor;
END IF;
LOOP
    FETCH c_emp_cursor...
```

%ROWCOUNT and %NOTFOUND: Example

```
DECLARE
  CURSOR c_emp_cursor IS SELECT employee_id,
    last_name FROM employees;
  v_emp_record  c_emp_cursor%ROWTYPE;
BEGIN
  OPEN c_emp_cursor;
  LOOP
    FETCH c_emp_cursor INTO v_emp_record;
    EXIT WHEN c_emp_cursor%ROWCOUNT > 10 OR
              c_emp_cursor%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE( v_emp_record.employee_id
                          || ' ' || v_emp_record.last_name);
  END LOOP;
  CLOSE c_emp_cursor;
END ; /
```

```
anonymous block completed
198 OConnell
199 Grant
200 Whalen
201 Hartstein
202 Fay
203 Mavris
204 Baer
205 Higgins
206 Gietz
100 King
```


Cursor FOR Loops Using Subqueries

There is no need to declare the cursor.

```
BEGIN
  FOR emp_record IN (SELECT employee_id, last_name
    FROM employees WHERE department_id =30)
  LOOP
    DBMS_OUTPUT.PUT_LINE( emp_record.employee_id
      || ' ' || emp_record.last_name);
  END LOOP;
END;
/
```

```
anonymous block completed
114 Raphaely
115 Khoo
116 Baida
117 Tobias
118 Himuro
119 Colmenares
```

Cursors with Parameters

Syntax:

```
CURSOR cursor_name  
    [(parameter_name datatype, ...)]  
IS  
    select_statement;
```

- Pass parameter values to a cursor when the cursor is opened and the query is executed.
- Open an explicit cursor several times with a different active set each time.

```
OPEN cursor_name(parameter_value,.....) ;
```

Cursors with Parameters

```
DECLARE
  CURSOR    c_emp_cursor (deptno NUMBER) IS
    SELECT  employee_id, last_name
    FROM    employees
    WHERE   department_id = deptno;
    ...
BEGIN
  OPEN c_emp_cursor (10);
  ...
  CLOSE c_emp_cursor;
  OPEN c_emp_cursor (20);
  ...
```

```
anonymous block completed
200 Whalen
201 Hartstein
202 Fay
```

FOR UPDATE Clause

Syntax:

```
SELECT ...  
FROM      ...  
FOR UPDATE [OF column_reference] [NOWAIT | WAIT n];
```

- Use explicit locking to deny access to other sessions for the duration of a transaction.
- Lock the rows *before* the update or delete.

WHERE CURRENT OF Clause

Syntax:

```
WHERE CURRENT OF cursor ;
```

- Use cursors to update or delete the current row.
- Include the FOR UPDATE clause in the cursor query to lock the rows first.
- Use the WHERE CURRENT OF clause to reference the current row from an explicit cursor.

```
UPDATE employees  
  SET      salary = ...  
  WHERE CURRENT OF c_emp_cursor;
```

Cursors with Subqueries: Example

```
DECLARE
  CURSOR my_cursor IS
    SELECT t1.department_id, t1.department_name,
           t2.staff
    FROM   departments t1, (SELECT department_id,
                                   COUNT(*) AS staff
                           FROM employees
                           GROUP BY department_id) t2
    WHERE  t1.department_id = t2.department_id
    AND    t2.staff >= 3;
...
```

Quiz

Implicit cursors are declared by PL/SQL implicitly for all DML and PL/SQL `SELECT` statements. The Oracle server implicitly opens a cursor to process each SQL statement that is not associated with an explicitly declared cursor.

1. True
2. False

Summary

In this lesson, you should have learned how to:

- Distinguish cursor types:
 - Implicit cursors are used for all DML statements and single-row queries.
 - Explicit cursors are used for queries of zero, one, or more rows.
- Create and handle explicit cursors
- Use simple loops and cursor `FOR` loops to handle multiple rows in the cursors
- Evaluate the cursor status by using the cursor attributes
- Use the `FOR UPDATE` and `WHERE CURRENT OF` clauses to update or delete the current fetched row

Practice 7: Overview

This practice covers the following topics:

- Declaring and using explicit cursors to query rows of a table
- Using a cursor `FOR` loop
- Applying cursor attributes to test the cursor status
- Declaring and using cursors with parameters
- Using the `FOR UPDATE` and `WHERE CURRENT OF` clauses

8

Handling Exceptions

Objectives

After completing this lesson, you should be able to do the following:

- Define PL/SQL exceptions
- Recognize unhandled exceptions
- List and use different types of PL/SQL exception handlers
- Trap unanticipated errors
- Describe the effect of exception propagation in nested blocks
- Customize PL/SQL exception messages

Example of an Exception

```
DECLARE
  v_lname VARCHAR2(15);
BEGIN
  SELECT last_name INTO v_lname
  FROM employees
  WHERE first_name='John';
  DBMS_OUTPUT.PUT_LINE ('John's last name is : '
                        || v_lname);
END;
```

Example of an Exception

```
DECLARE
    v_lname VARCHAR2(15);
BEGIN
    SELECT last_name INTO v_lname
    FROM employees
    WHERE first_name='John';
    DBMS_OUTPUT.PUT_LINE ('John's last name is : '
                          || v_lname);

EXCEPTION
    WHEN TOO_MANY_ROWS THEN

    DBMS_OUTPUT.PUT_LINE (' Your select statement
    retrieved multiple rows. Consider using a
    cursor. ');

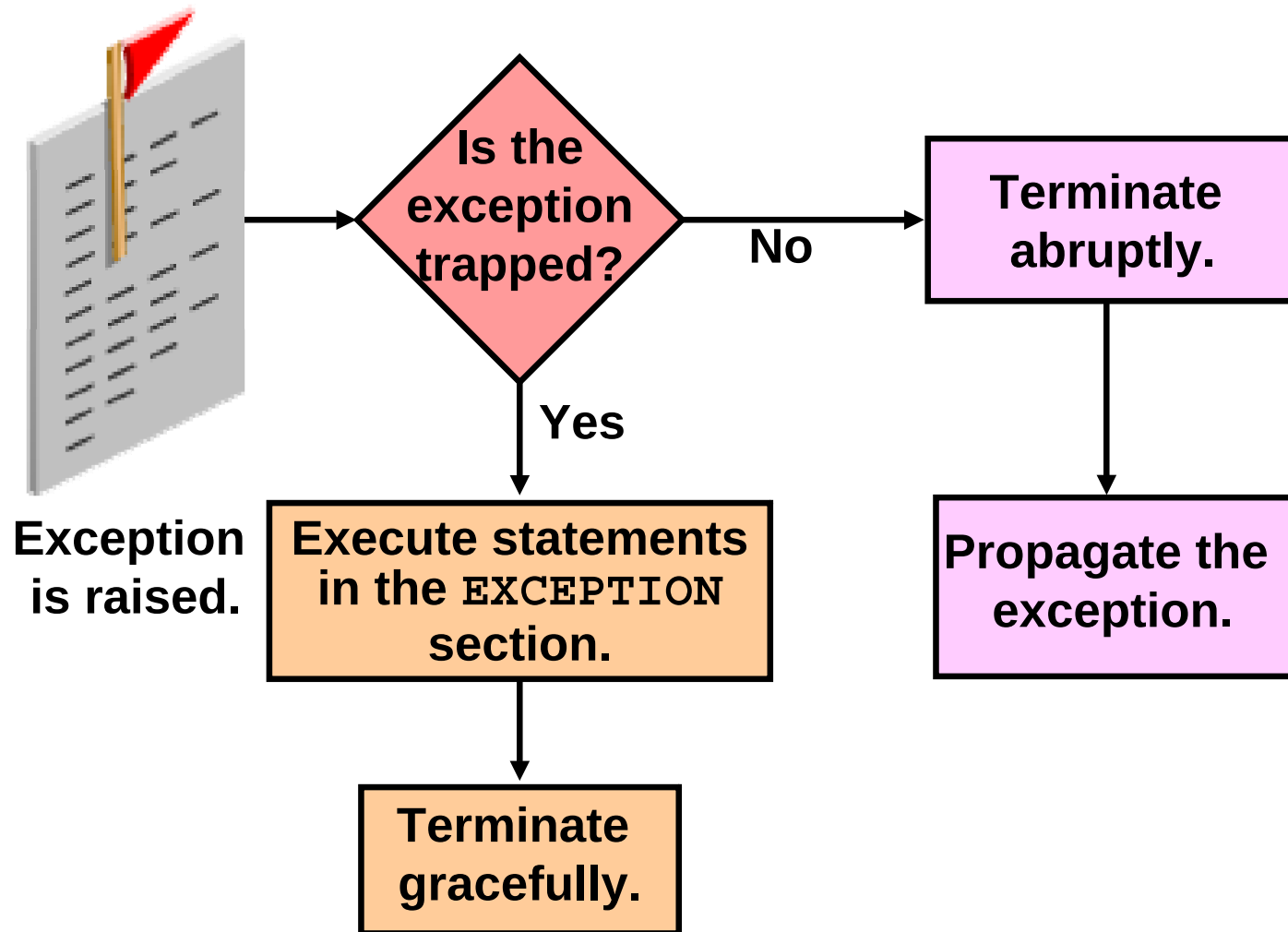
END;
/
```

```
anonymous block completed
Your select statement retrieved multiple
rows. Consider using a cursor.
```

Handling Exceptions with PL/SQL

- An exception is a PL/SQL error that is raised during program execution.
- An exception can be raised:
 - Implicitly by the Oracle server
 - Explicitly by the program
- An exception can be handled:
 - By trapping it with a handler
 - By propagating it to the calling environment

Handling Exceptions



Exception Types

- Predefined Oracle server
 - Non-predefined Oracle server
- 
- Implicitly raised**
-
- User-defined
- Explicitly raised**

Trapping Exceptions

Syntax:

```
EXCEPTION
  WHEN exception1 [OR exception2 . . .] THEN
    statement1;
    statement2;
    . . .
  [WHEN exception3 [OR exception4 . . .] THEN
    statement1;
    statement2;
    . . .]
  [WHEN OTHERS THEN
    statement1;
    statement2;
    . . .]
```

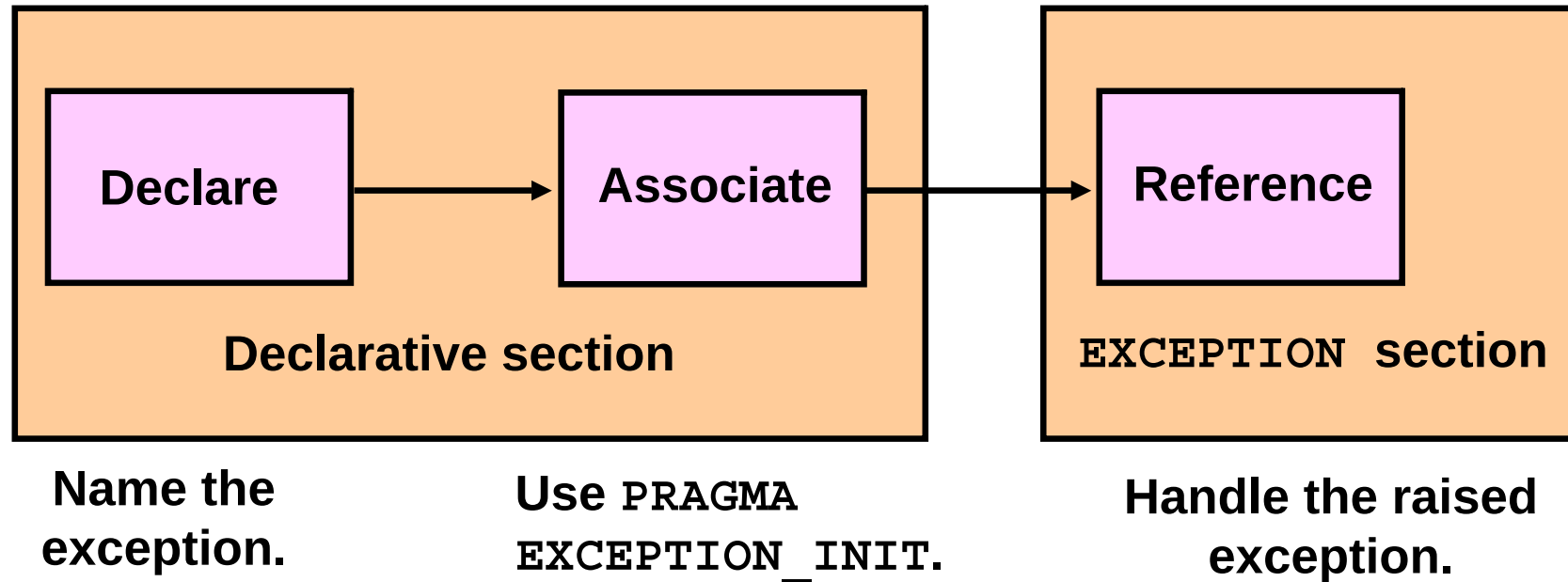
Guidelines for Trapping Exceptions

- The `EXCEPTION` keyword starts the exception-handling section.
- Several exception handlers are allowed.
- Only one handler is processed before leaving the block.
- `WHEN OTHERS` is the last clause.

Trapping Predefined Oracle Server Errors

- Reference the predefined name in the exception-handling routine.
- Sample predefined exceptions:
 - NO_DATA_FOUND
 - TOO_MANY_ROWS
 - INVALID_CURSOR
 - ZERO_DIVIDE
 - DUP_VAL_ON_INDEX

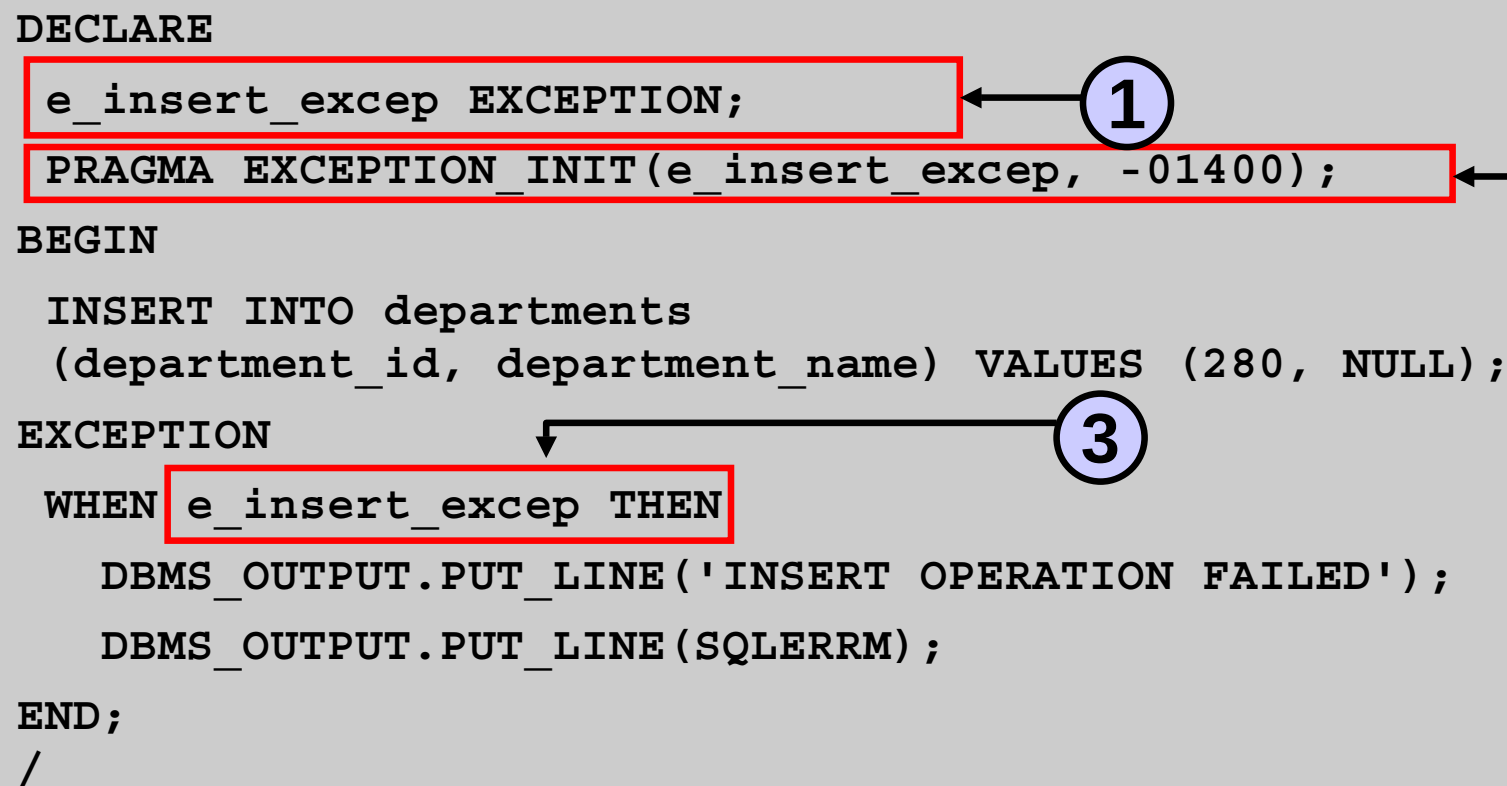
Trapping Non-Predefined Oracle Server Errors



Non-Predefined Error

To trap Oracle server error number -01400
("cannot insert NULL"):

```
DECLARE
  e_insert_excep EXCEPTION;
  PRAGMA EXCEPTION_INIT(e_insert_excep, -01400);
BEGIN
  INSERT INTO departments
    (department_id, department_name) VALUES (280, NULL);
EXCEPTION
  WHEN e_insert_excep THEN
    DBMS_OUTPUT.PUT_LINE('INSERT OPERATION FAILED');
    DBMS_OUTPUT.PUT_LINE(SQLERRM);
END;
/
```



```
anonymous block completed
INSERT OPERATION FAILED
ORA-01400: cannot insert NULL into ("ORA41"."DEPARTMENTS"."DEPARTMENT_NAME")
```

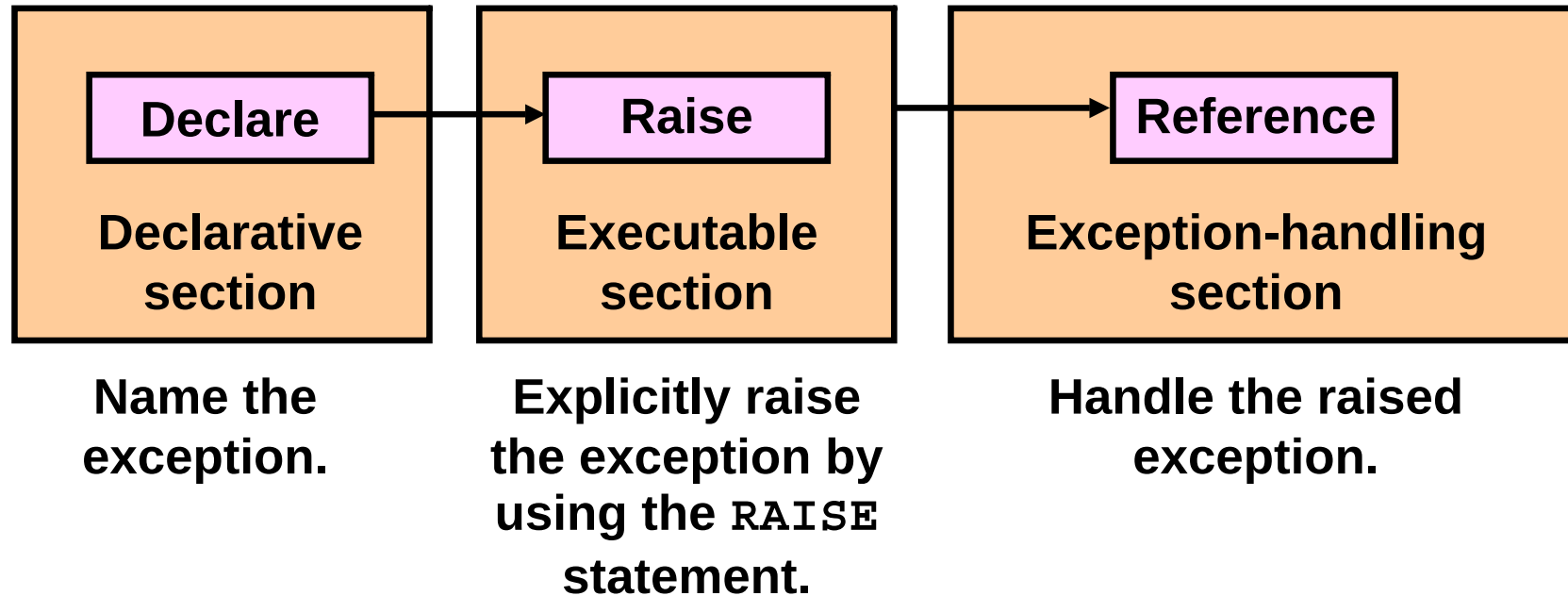
Functions for Trapping Exceptions

- `SQLCODE`: Returns the numeric value for the error code
- `SQLERRM`: Returns the message associated with the error number

Functions for Trapping Exceptions

```
DECLARE
    error_code      NUMBER;
    error_message    VARCHAR2(255);
BEGIN
    ...
EXCEPTION
    ...
    WHEN OTHERS THEN
        ROLLBACK;
        error_code := SQLCODE ;
        error_message := SQLERRM ;
        INSERT INTO errors (e_user, e_date, error_code,
            error_message) VALUES (USER,SYSDATE,error_code,
            error_message);
END;
/
```

Trapping User-Defined Exceptions



Trapping User-Defined Exceptions

```
DECLARE
  v_deptno NUMBER := 500;
  v_name VARCHAR2(20) := 'Testing';
  e_invalid_department EXCEPTION;
BEGIN
  UPDATE departments
  SET department_name = v_name
  WHERE department_id = v_deptno;
  IF SQL % NOTFOUND THEN
    RAISE e_invalid_department;
  END IF;
  COMMIT;
EXCEPTION
  WHEN e_invalid_department THEN
    DBMS_OUTPUT.PUT_LINE('No such department id.');
```

```
END;
/
```

```
anonymous block completed
No such department id.
```

Propagating Exceptions in a Subblock

Subblocks can handle an exception or pass the exception to the enclosing block.

```
DECLARE
    . . .
    e_no_rows      exception;
    e_integrity    exception;
    PRAGMA EXCEPTION_INIT (e_integrity, -2292);
BEGIN
    FOR c_record IN emp_cursor LOOP
        BEGIN
            SELECT ...
            UPDATE ...
            IF SQL%NOTFOUND THEN
                RAISE e_no_rows;
            END IF;
        END;
    END LOOP;
EXCEPTION
    WHEN e_integrity THEN ...
    WHEN e_no_rows THEN ...
END;
/
```

RAISE_APPLICATION_ERROR Procedure

Syntax:

```
raise_application_error (error_number,  
                        message[, {TRUE | FALSE}]);
```

- You can use this procedure to issue user-defined error messages from stored subprograms.
- You can report errors to your application and avoid returning unhandled exceptions.

RAISE_APPLICATION_ERROR Procedure

- Used in two different places:
 - Executable section
 - Exception section
- Returns error conditions to the user in a manner consistent with other Oracle server errors

RAISE_APPLICATION_ERROR Procedure

Executable section:

```
BEGIN
...
DELETE FROM employees
  WHERE manager_id = v_mgr;
IF SQL%NOTFOUND THEN
  RAISE_APPLICATION_ERROR(-20202,
    'This is not a valid manager');
END IF;
...
```

Exception section:

```
...
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RAISE_APPLICATION_ERROR (-20201,
      'Manager is not a valid employee.');
```

END;
/

Quiz

You can trap any error by including a corresponding handler within the exception-handling section of the PL/SQL block.

1. True
2. False

Summary

In this lesson, you should have learned how to:

- Define PL/SQL exceptions
- Add an `EXCEPTION` section to the PL/SQL block to deal with exceptions at run time
- Handle different types of exceptions:
 - Predefined exceptions
 - Non-predefined exceptions
 - User-defined exceptions
- Propagate exceptions in nested blocks and call applications

Practice 8: Overview

This practice covers the following topics:

- Handling named exceptions
- Creating and invoking user-defined exceptions



Creating Stored Procedures and Functions

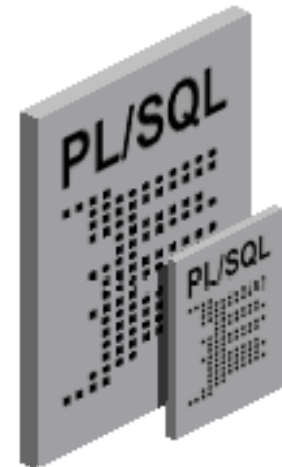
Objectives

After completing this lesson, you should be able to do the following:

- Differentiate between anonymous blocks and subprograms
- Create a simple procedure and invoke it from an anonymous block
- Create a simple function
- Create a simple function that accepts a parameter
- Differentiate between procedures and functions

Procedures and Functions

- Are named PL/SQL blocks
- Are called PL/SQL subprograms
- Have block structures similar to anonymous blocks:
 - Optional declarative section (without the `DECLARE` keyword)
 - Mandatory executable section
 - Optional section to handle exceptions



Differences Between Anonymous Blocks and Subprograms

Anonymous Blocks	Subprograms
Unnamed PL/SQL blocks	Named PL/SQL blocks
Compiled every time	Compiled only once
Not stored in the database	Stored in the database
Cannot be invoked by other applications	Named and, therefore, can be invoked by other applications
Do not return values	Subprograms called functions must return values.
Cannot take parameters	Can take parameters

Procedure: Syntax

```
CREATE [OR REPLACE] PROCEDURE procedure_name
  [(argument1 [mode1] datatype1,
    argument2 [mode2] datatype2,
    . . .)]
IS|AS
procedure_body;
```

Procedure: Example

```
...  
CREATE TABLE dept AS SELECT * FROM departments;  
CREATE PROCEDURE add_dept IS  
  v_dept_id dept.department_id%TYPE;  
  v_dept_name dept.department_name%TYPE;  
BEGIN  
  v_dept_id:=280;  
  v_dept_name:='ST-Curriculum';  
  INSERT INTO dept(department_id,department_name)  
  VALUES(v_dept_id,v_dept_name);  
  DBMS_OUTPUT.PUT_LINE('  Inserted ' || SQL%ROWCOUNT  
  || ' row ');  
END;
```

Invoking the Procedure

```
BEGIN
  add_dept;
END;
/
SELECT department_id, department_name FROM dept
WHERE department_id=280;
```

anonymous block completed

Inserted 1 row

DEPARTMENT_ID	DEPARTMENT_NAME
---------------	-----------------

-------	--

280	ST-Curriculum
-----	---------------

1 rows selected

Function: Syntax

```
CREATE [OR REPLACE] FUNCTION function_name
  [(argument1 [mode1] datatype1,
    argument2 [mode2] datatype2,
    . . .)]
RETURN datatype
IS|AS
function_body;
```


Function: Example

```
CREATE FUNCTION check_sal RETURN Boolean IS
v_dept_id employees.department_id%TYPE;
v_empno    employees.employee_id%TYPE;
v_sal      employees.salary%TYPE;
v_avg_sal  employees.salary%TYPE;
BEGIN
  v_empno:=205;
  SELECT salary,department_id INTO v_sal,v_dept_id FROM
employees
  WHERE employee_id= v_empno;
  SELECT avg(salary) INTO v_avg_sal FROM employees WHERE
department_id=v_dept_id;
  IF v_sal > v_avg_sal THEN
    RETURN TRUE;
  ELSE
    RETURN FALSE;
  END IF;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN NULL;
END;
```

Invoking the Function

```
BEGIN
  IF (check_sal IS NULL) THEN
    DBMS_OUTPUT.PUT_LINE('The function returned
      NULL due to exception');
  ELSIF (check_sal) THEN
    DBMS_OUTPUT.PUT_LINE('Salary > average');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Salary < average');
  END IF;
END;
/
```

```
anonymous block completed
Salary > average
```

Passing a Parameter to the Function

```
DROP FUNCTION check_sal;
CREATE FUNCTION check_sal(p_empno employees.employee_id%TYPE)
RETURN Boolean IS
    v_dept_id employees.department_id%TYPE;
    v_sal      employees.salary%TYPE;
    v_avg_sal  employees.salary%TYPE;
BEGIN
    SELECT salary,department_id INTO v_sal,v_dept_id FROM employees
        WHERE employee_id=p_empno;
    SELECT avg(salary) INTO v_avg_sal FROM employees
        WHERE department_id=v_dept_id;
    IF v_sal > v_avg_sal THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;
EXCEPTION
    ...
```

Invoking the Function with a Parameter

```
BEGIN
DBMS_OUTPUT.PUT_LINE('Checking for employee with id 205');
  IF (check_sal(205) IS NULL) THEN
    DBMS_OUTPUT.PUT_LINE('The function returned
      NULL due to exception');
  ELSIF (check_sal(205)) THEN
    DBMS_OUTPUT.PUT_LINE('Salary > average');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Salary < average');
  END IF;
DBMS_OUTPUT.PUT_LINE('Checking for employee with id 70');
  IF (check_sal(70) IS NULL) THEN
    DBMS_OUTPUT.PUT_LINE('The function returned
      NULL due to exception');
  ELSIF (check_sal(70)) THEN
    ...
  END IF;
END;
/
```

Quiz

Subprograms:

1. Are named PL/SQL blocks and can be invoked by other applications
2. Are compiled only once
3. Are stored in the database
4. Do not have to return values if they are functions
5. Can take parameters

Summary

In this lesson, you should have learned how to:

- Create a simple procedure
- Invoke the procedure from an anonymous block
- Create a simple function
- Create a simple function that accepts parameters
- Invoke the function from an anonymous block

Practice 9: Overview

This practice covers the following topics:

- Converting an existing anonymous block to a procedure
- Modifying the procedure to accept a parameter
- Writing an anonymous block to invoke the procedure



Using SQL Developer

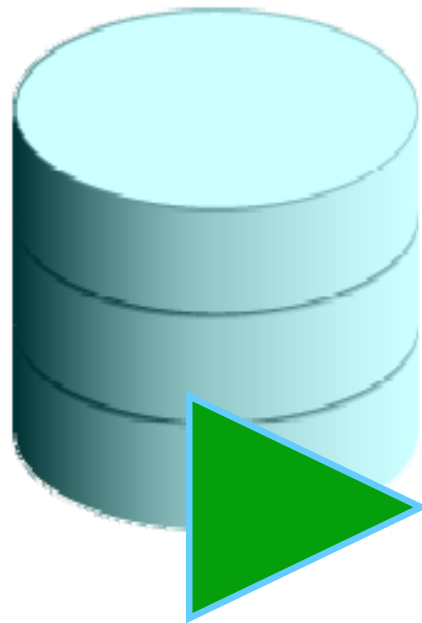
Objectives

After completing this appendix, you should be able to do the following:

- List the key features of Oracle SQL Developer
- Install Oracle SQL Developer 1.2.1
- Identify menu items of Oracle SQL Developer
- Create a database connection
- Manage database objects
- Use SQL Worksheet
- Save and Run SQL scripts
- Create and save reports
- Install and use Oracle SQL Developer 1.5.3

What Is Oracle SQL Developer?

- Oracle SQL Developer is a graphical tool that enhances productivity and simplifies database development tasks.
- You can connect to any target Oracle database schema by using standard Oracle database authentication.



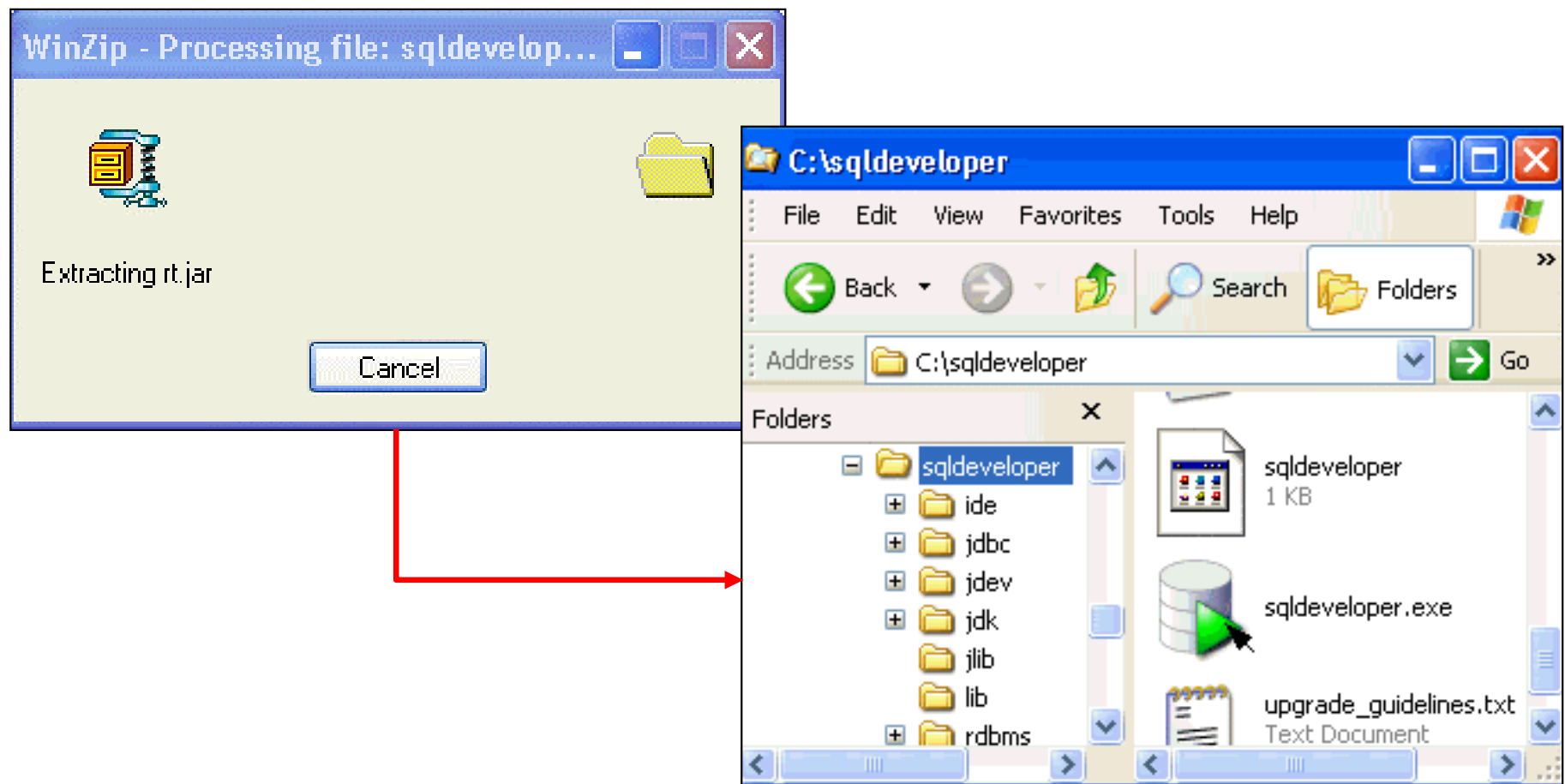
SQL Developer

Specifications of SQL Developer

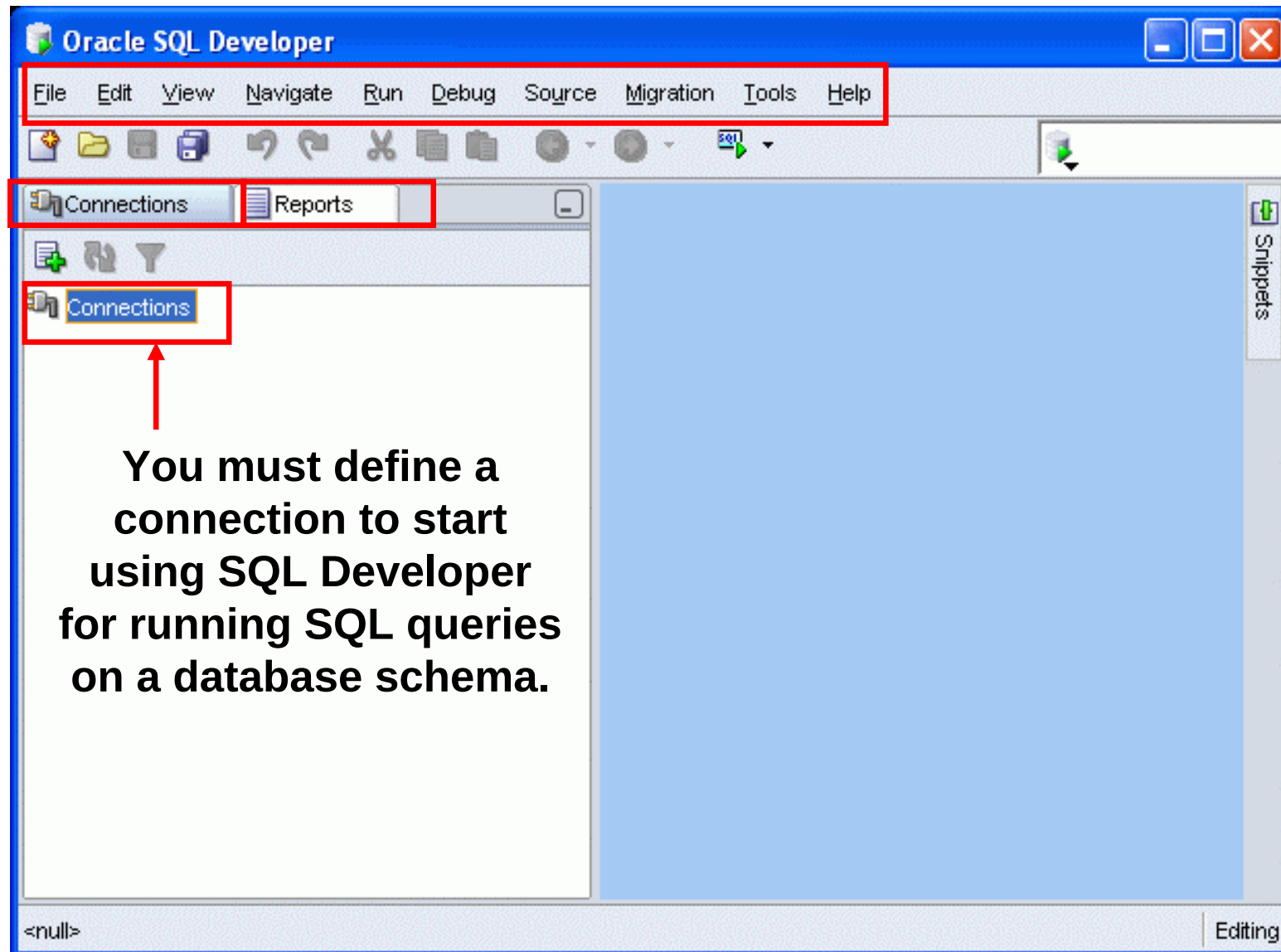
- Developed in Java
- Supports Windows, Linux, and Mac OS X platforms
- Default connectivity by using the JDBC Thin driver
- Does not require an installer
 - Unzip the downloaded SQL Developer kit and double-click `sqldeveloper.exe` to start SQL Developer.
- Connects to Oracle Database version 9.2.0.1 and later
- Freely downloadable from the following link:
 - http://www.oracle.com/technology/products/database/sql_developer/index.html
- Needs JDK 1.5 installed on your system that can be downloaded from the following link:
 - http://java.sun.com/javase/downloads/index_jdk5.jsp

Installing SQL Developer

Download the Oracle SQL Developer kit and unzip into any directory on your machine.



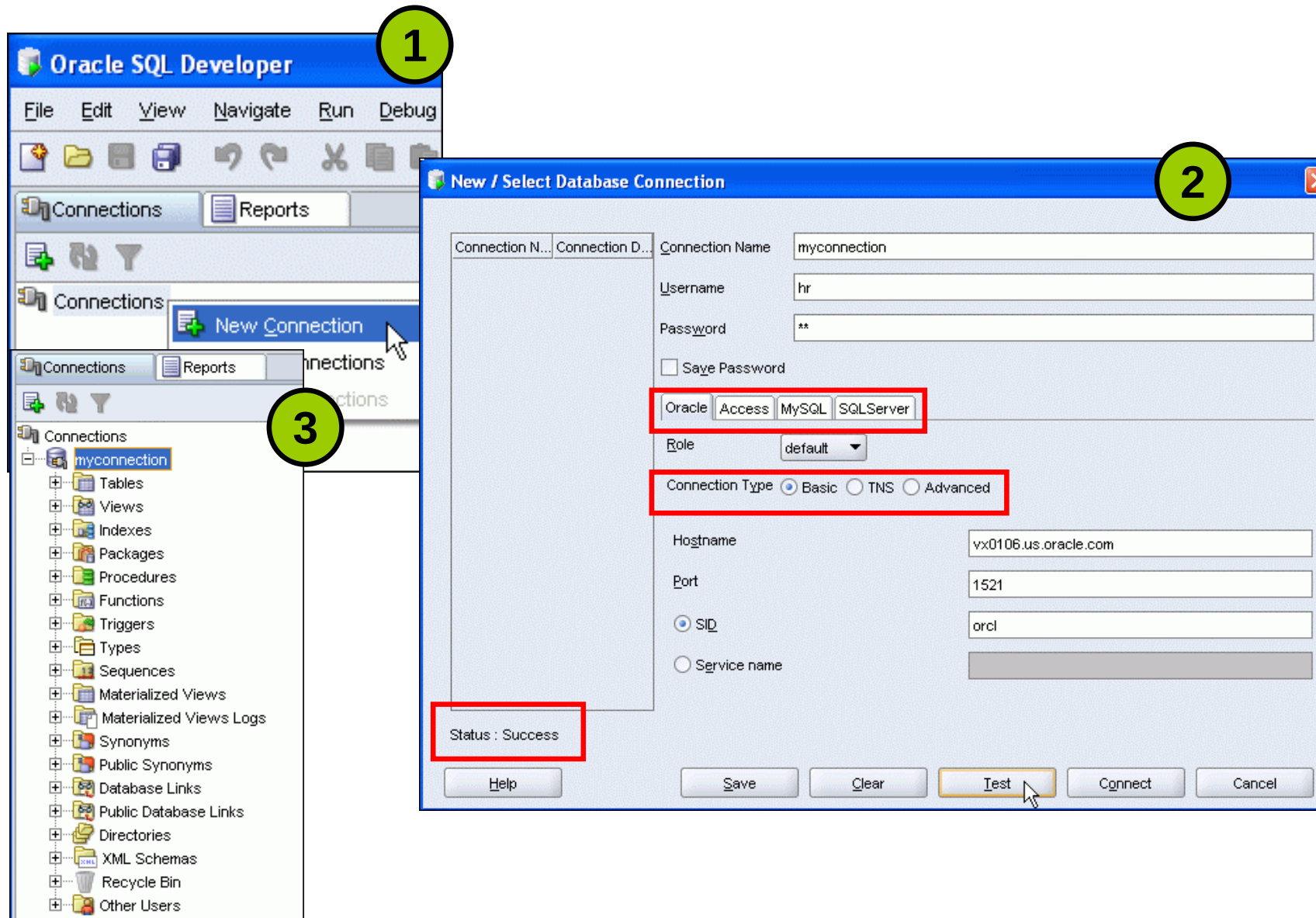
SQL Developer 1.2 Interface



Creating a Database Connection

- You must have at least one database connection to use SQL Developer.
- You can create and test connections for:
 - Multiple databases
 - Multiple schemas
- SQL Developer automatically imports any connections defined in the `tnsnames.ora` file on your system.
- You can export connections to an Extensible Markup Language (XML) file.
- Each additional database connection created is listed in the Connections Navigator hierarchy.

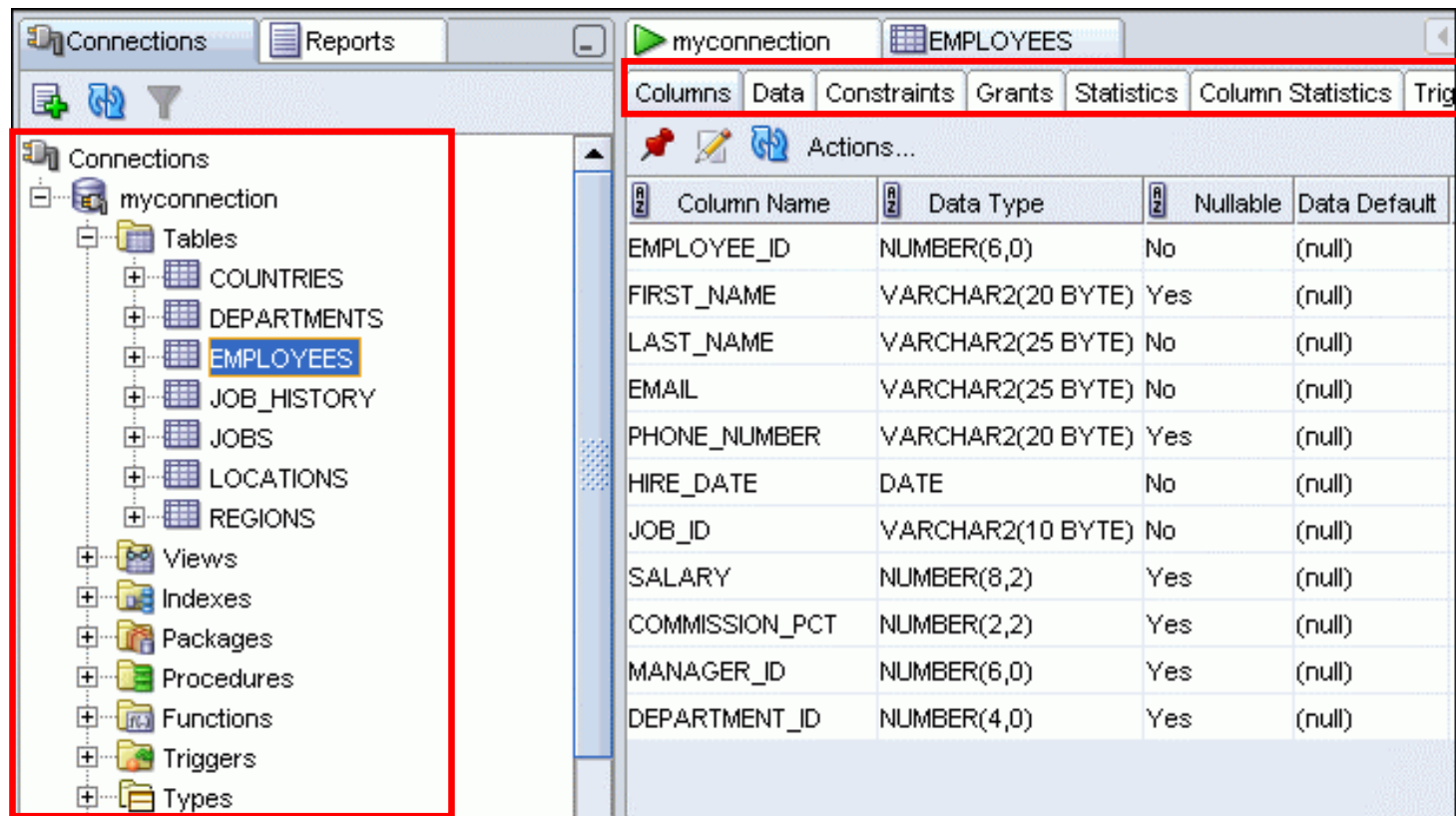
Creating a Database Connection



Browsing Database Objects

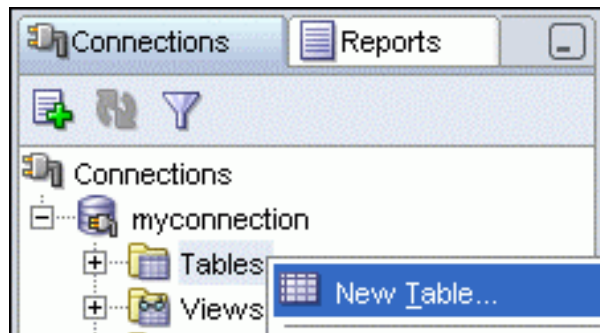
Use the Connections Navigator to:

- Browse through many objects in a database schema
- Review the definitions of objects at a glance



Creating a Schema Object

- SQL Developer supports the creation of any schema object by:
 - Executing a SQL statement in SQL Worksheet
 - Using the context menu
- Edit the objects by using an edit dialog or one of the many context-sensitive menus.
- View the data definition language (DDL) for adjustments such as creating a new object or editing an existing schema object.



Creating a New Table: Example

Create Table

Schema:

Name:

Table Type: ☒ Normal ☐ External ☐ Index Organized ☐ Temporary (Transaction) ☐ Temporary (Session)

☒ Advanced

Columns

- Primary Key
- Unique Constraints
- Foreign Keys
- Check Constraints
- Indexes
- Column Sequences
- Storage Options
- Lob Parameters
- ☒ Partitioning
 - Partition Definitions
 - Subpartition Templates
- Comment
- DDL

Columns:

- ID
- FIRST_NAME
- LAST_NAME
- RELATION
- BIRTHDATE

Column Properties

Name:

Datatype: ☒ Simple ☐ Complex

Type:

Precision:

Scale:

Default:

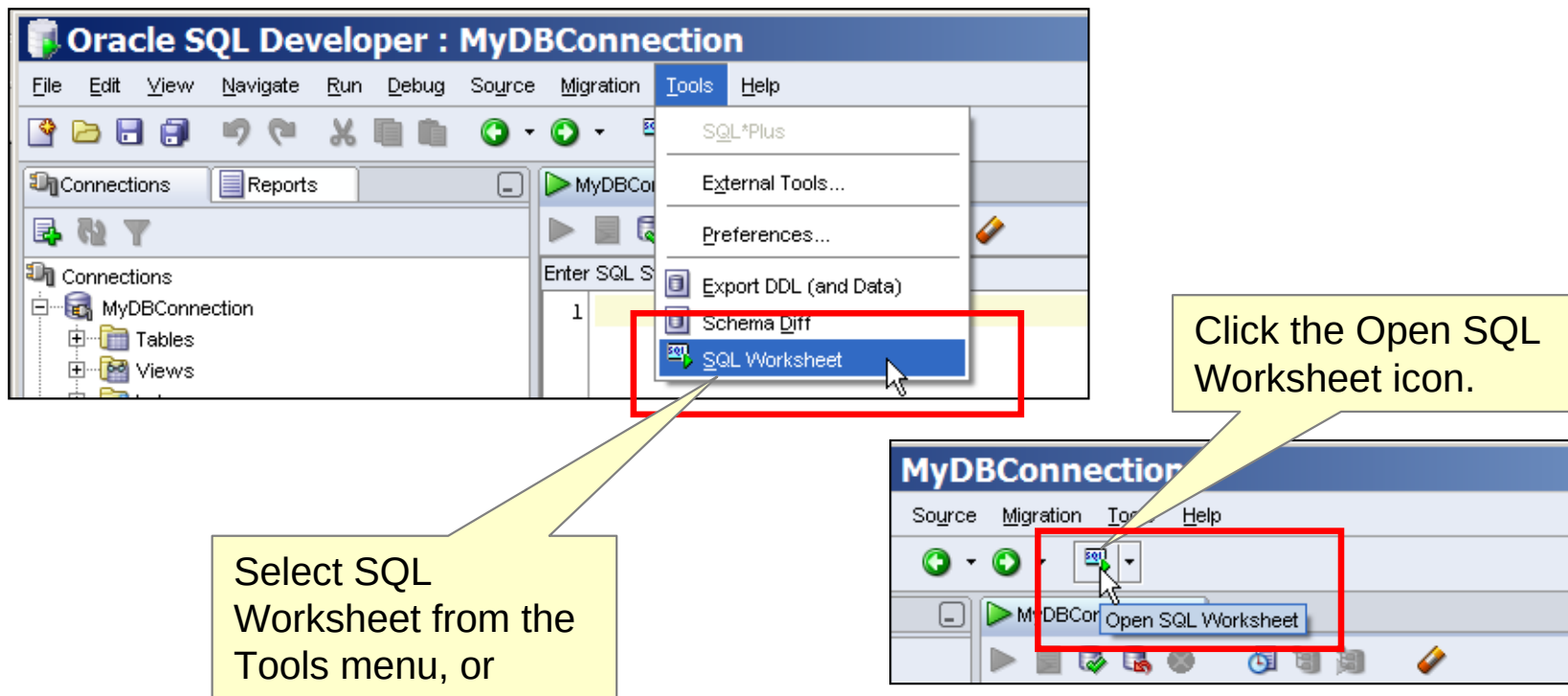
☒ Cannot be NULL

Comment:

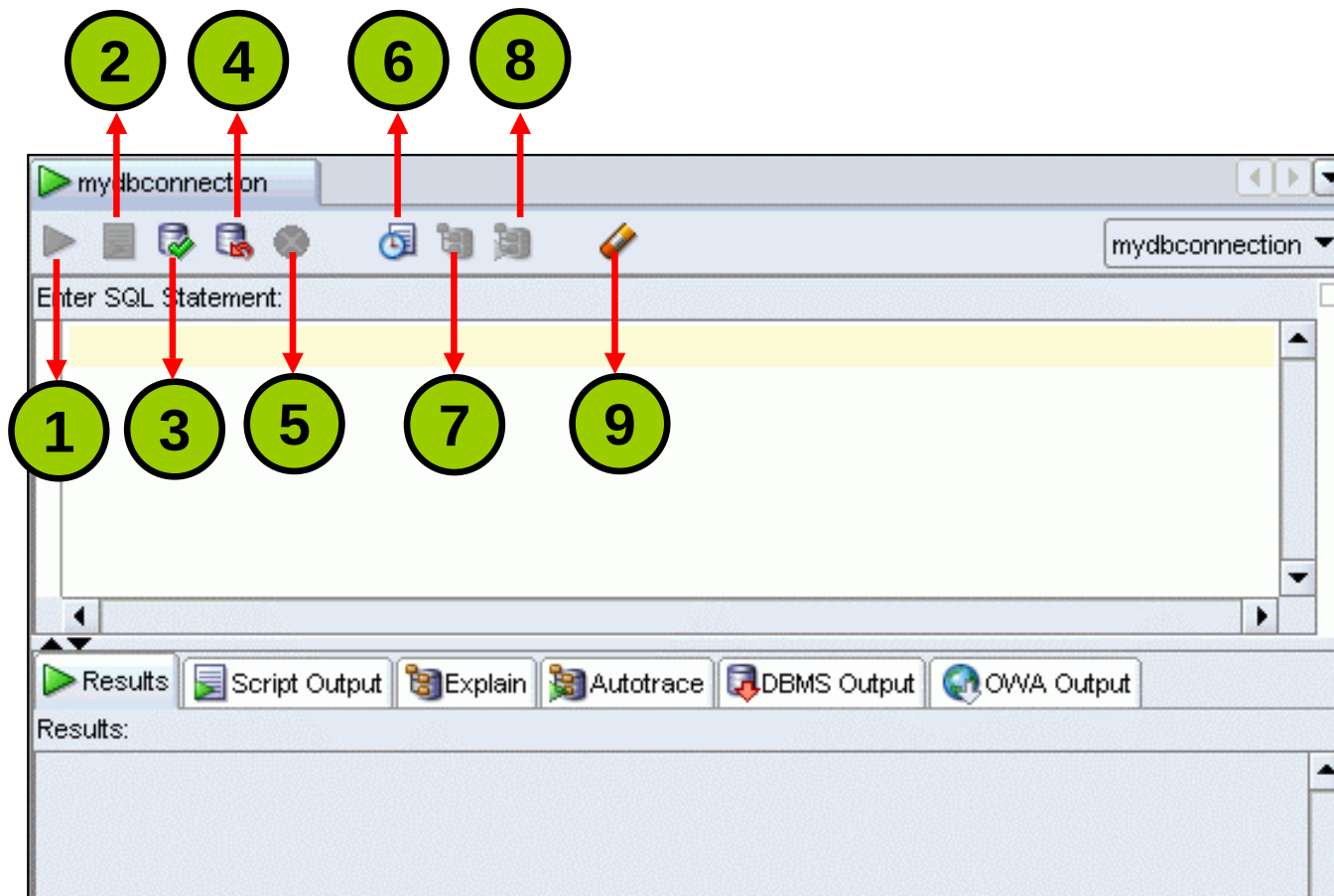
Help OK Cancel

Using the SQL Worksheet

- Use the SQL Worksheet to enter and execute SQL, PL/SQL, and SQL *Plus statements.
- Specify any actions that can be processed by the database connection associated with the worksheet.

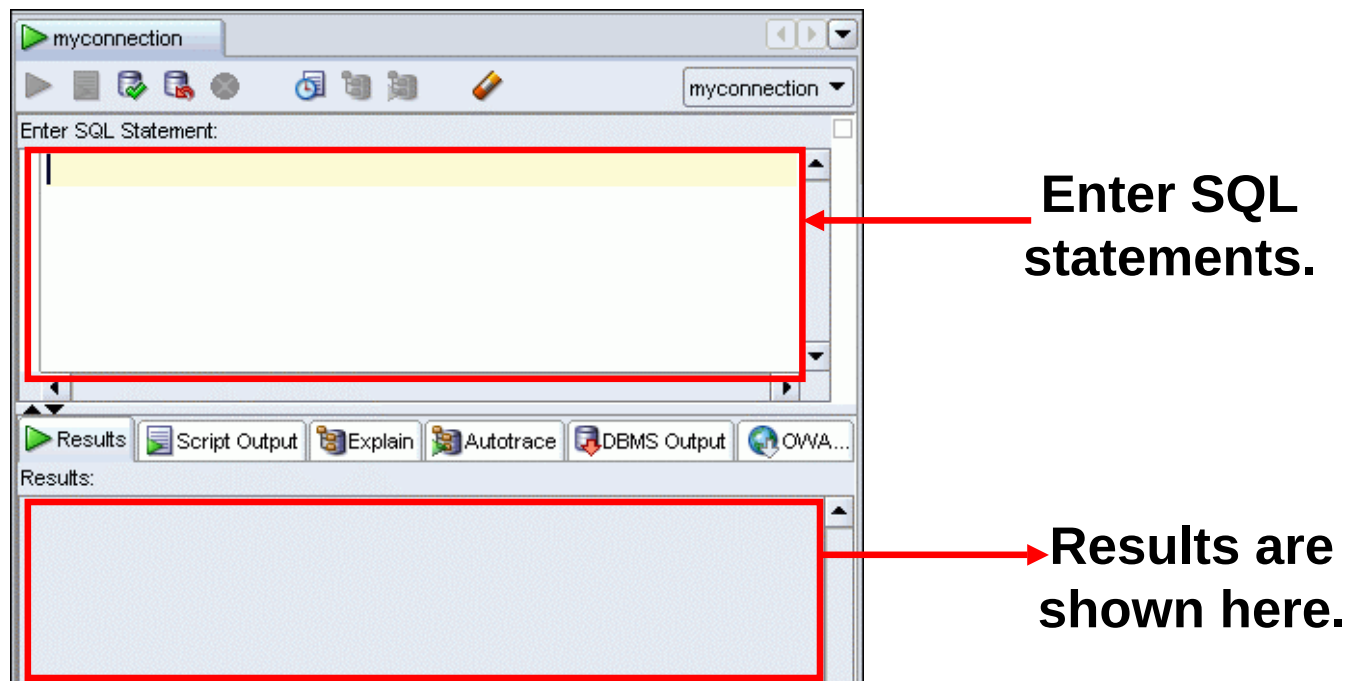


Using the SQL Worksheet



Using the SQL Worksheet

- Use the SQL Worksheet to enter and execute SQL, PL/SQL, and SQL*Plus statements.
- Specify any actions that can be processed by the database connection associated with the worksheet.



Executing SQL Statements

Use the Enter SQL Statement box to enter single or multiple SQL statements.

Use the Enter SQL Statement box to enter single or multiple SQL statements.

View the results on the Script Output tabbed page.

The screenshot shows the Oracle SQL Developer interface. At the top, a tab labeled 'MyDBConnection' is active. Below it is a toolbar with various icons, and a status bar indicating '2.03304029 seconds'. The main area is divided into two panes. The top pane, titled 'Enter SQL Statement:', contains the following SQL code:

```
1 SELECT last_name, salary
2 FROM employees
3 WHERE salary > 10000;
4
5 SELECT last_name "Name", salary*12 "Annual Salary"
6 FROM employees;
```

The bottom pane is divided into several tabs: 'Results', 'Script Output', 'Explain', 'Autotrace', 'DBMS Output', and 'OWA Output'. The 'Script Output' tab is selected and highlighted with a red box. It displays the results of the SQL execution:

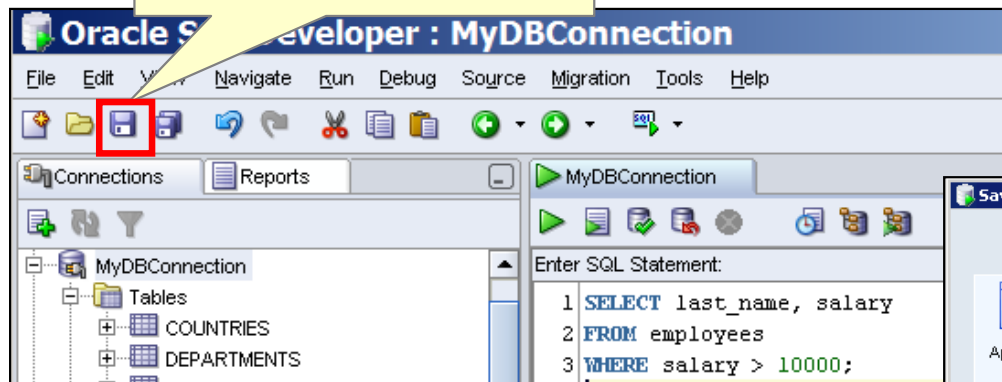
```
Ozer          11500
Abel          11000
15 rows selected

Name          Annual Salary
-----
OConnell      31200
Grant         31200
Whalen        52800
```

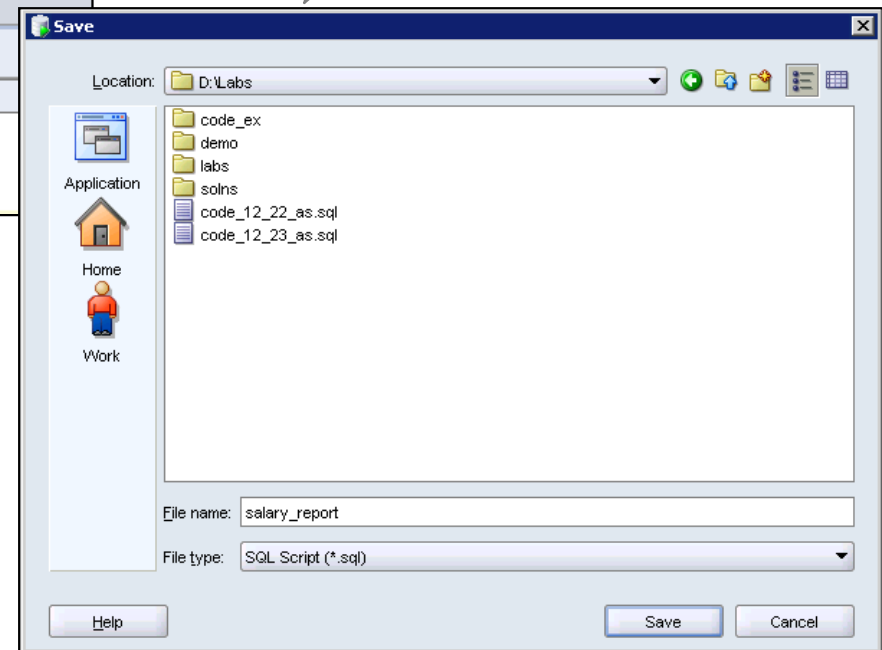
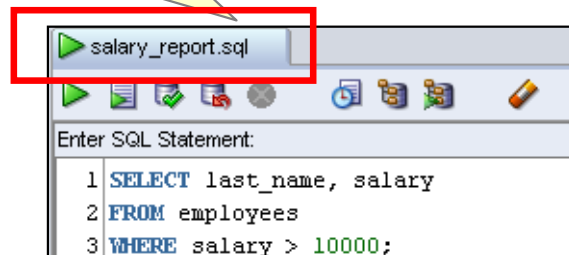
Saving SQL Scripts

Click the Save icon to save your SQL statement to a file.

Enter a file name and identify a location to save the file, and click Save.



The contents of the saved file are visible and editable in your SQL Worksheet window.



Executing Saved Script Files: Method 1

Right-click in the SQL Worksheet area, and select Open File from the shortcut menu.

Select (or navigate to) the script file that you want to open.

Click Open.

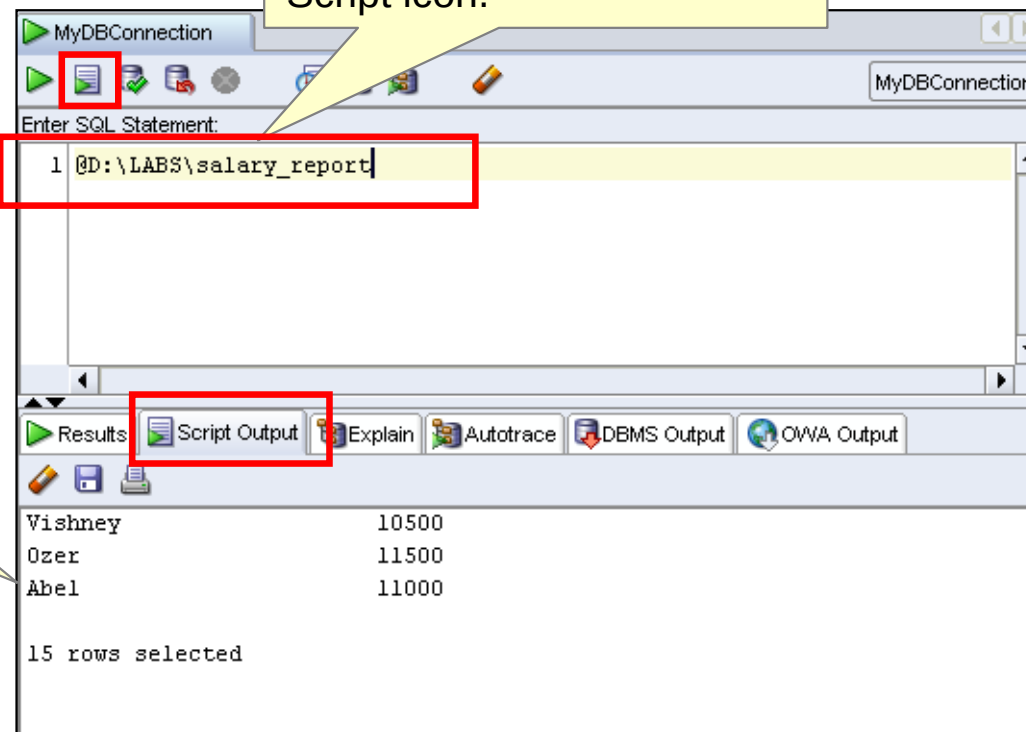
To run the code, click the Run Script (F5) icon.

```
CREATE PROCEDURE getemp IS -- header
emp_id employees.employee_id%type;
lname employees.last_name%type;
BEGIN
emp_id := 100;
SELECT last_name INTO lname
FROM EMPLOYEES
WHERE employee_id = emp_id;
DBMS_OUTPUT.PUT_LINE('Last name: '||lname);
END;
```


Executing Saved Script Files: Method 2

Use the @ command followed by the location and name of the file you want to execute, and click the Run Script icon.

The output from the script is displayed on the Script Output tabbed page.



Executing SQL Statements

Use the Enter SQL Statement box to enter single or multiple SQL statements.

The screenshot illustrates the workflow for executing SQL statements in Oracle SQL Developer. It shows three main components: the 'Enter SQL Statement' box, the 'Results' window, and the 'Script Output' window.

Enter SQL Statement Box: This box contains the SQL query: `SELECT employee_id, last_name FROM employees;`. The 'Execute' button (a green play icon) is highlighted with a red box and labeled **F9**. The 'Script' button (a document icon) is also highlighted with a red box and labeled **F5**. A red arrow points from the 'Execute' button to the 'Results' window.

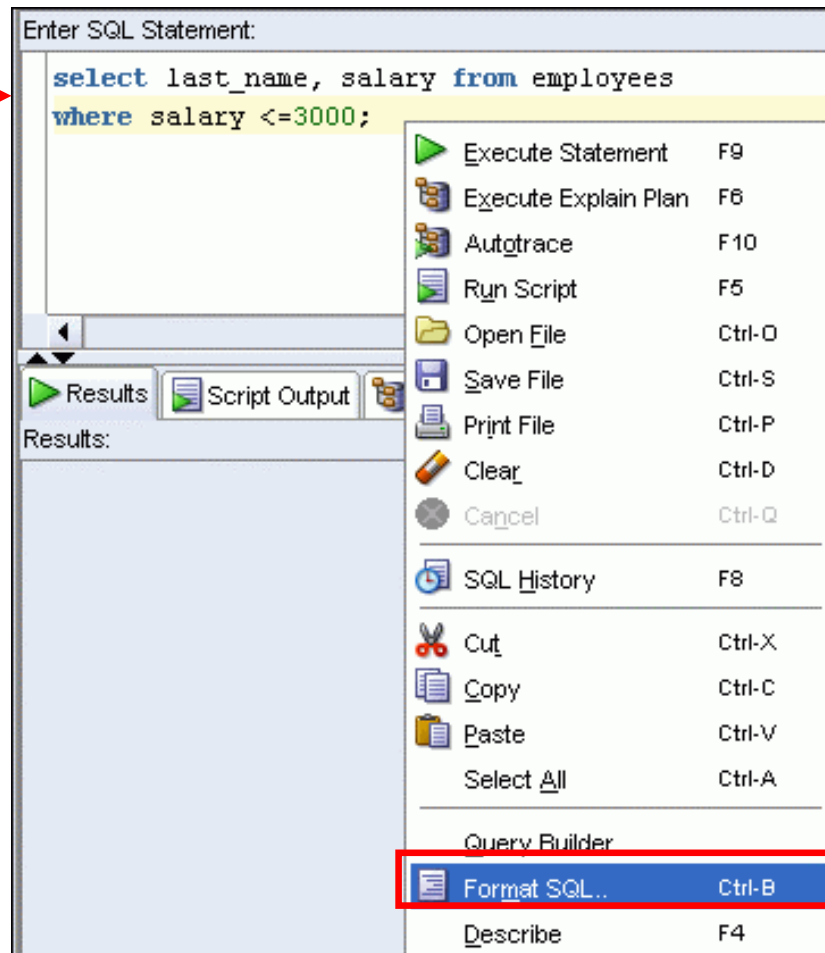
Results Window: This window displays the results of the SQL query. The 'Results' button (a green play icon) is highlighted with a red box and labeled **F9**. The results are shown in a table with columns `EMPLOYEE_ID` and `LAST_NAME`.

Script Output Window: This window displays the script output. The 'Script Output' button (a document icon) is highlighted with a red box and labeled **F5**. The output is shown in a table with columns `EMPLOYEE_ID` and `LAST_NAME`.

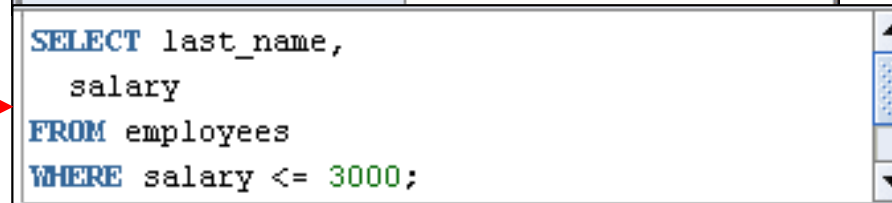
EMPLOYEE_ID	LAST_NAME
100	King
101	Kochhar
102	De Haan
103	Hunold
104	Ernst
105	Austin

Formatting the SQL Code

**Before
formatting**

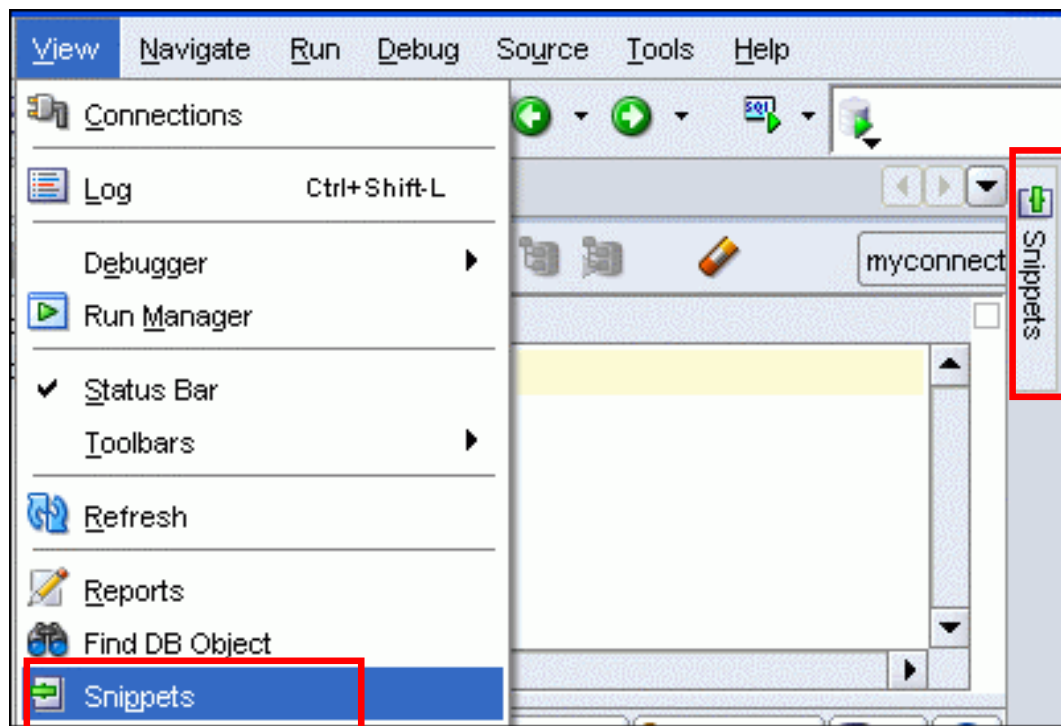


**After
formatting**

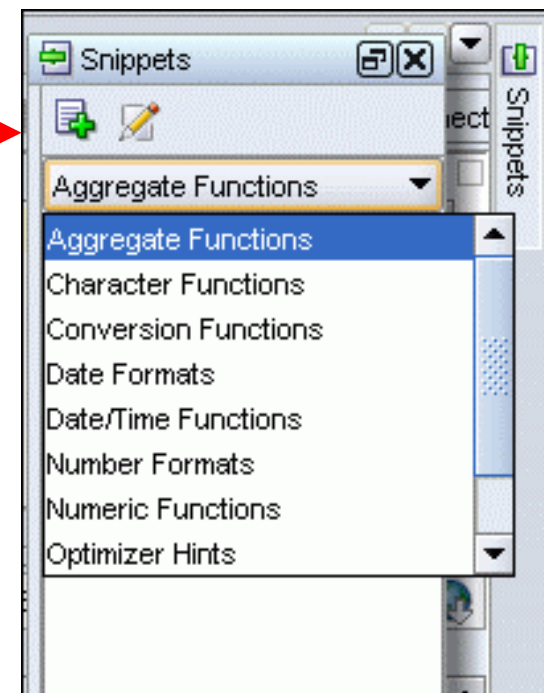


Using Snippets

Snippets are code fragments that may be just syntax or examples.

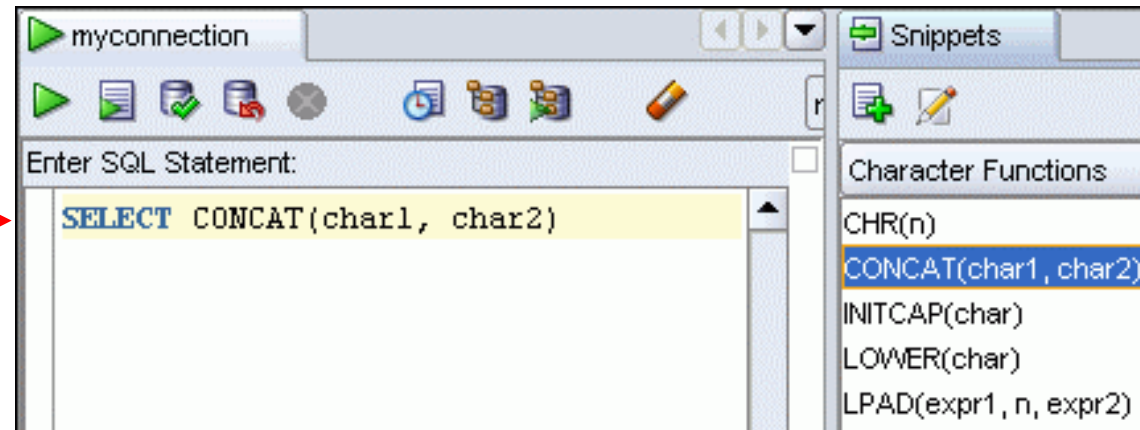


When you place your cursor here, it shows the Snippets window. From the drop-down list, you can select the functions category you want.

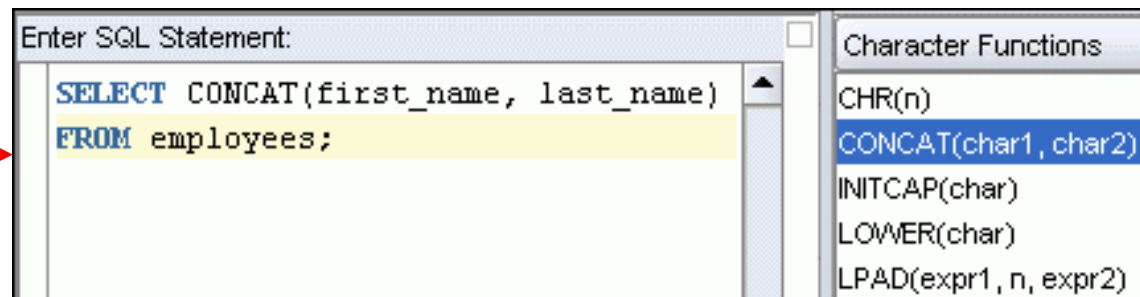


Using Snippets: Example

Inserting a snippet

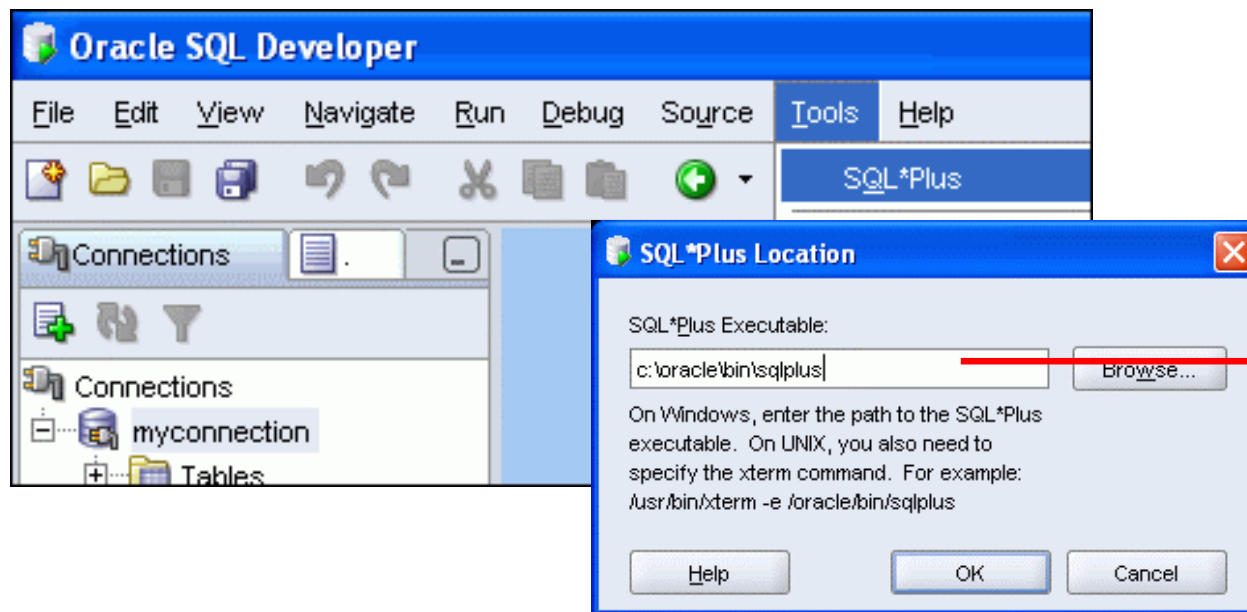


Editing the snippet



Using SQL*Plus

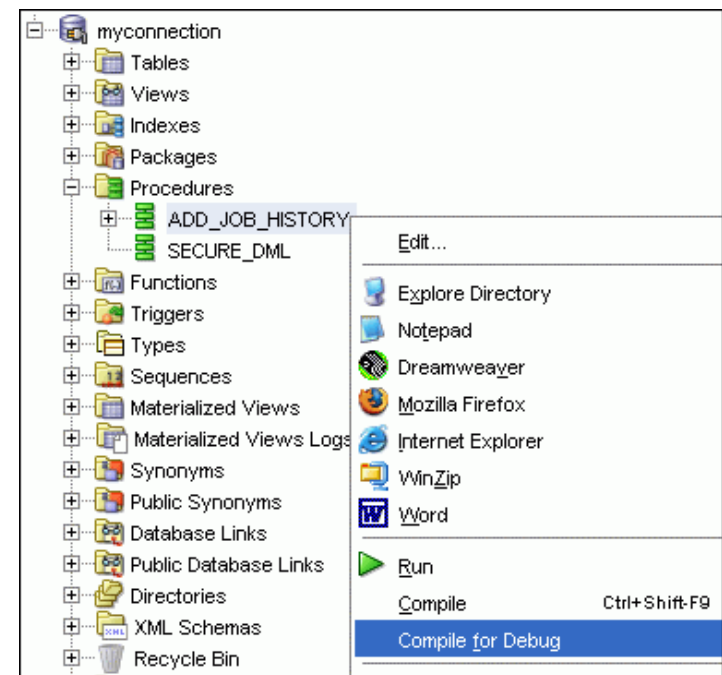
- You can invoke the SQL*Plus command-line interface from SQL Developer.
- Close all the SQL Worksheets to enable the SQL*Plus menu option.



Provide the location of the sqlplus.exe file only the first time you invoke SQL*Plus.

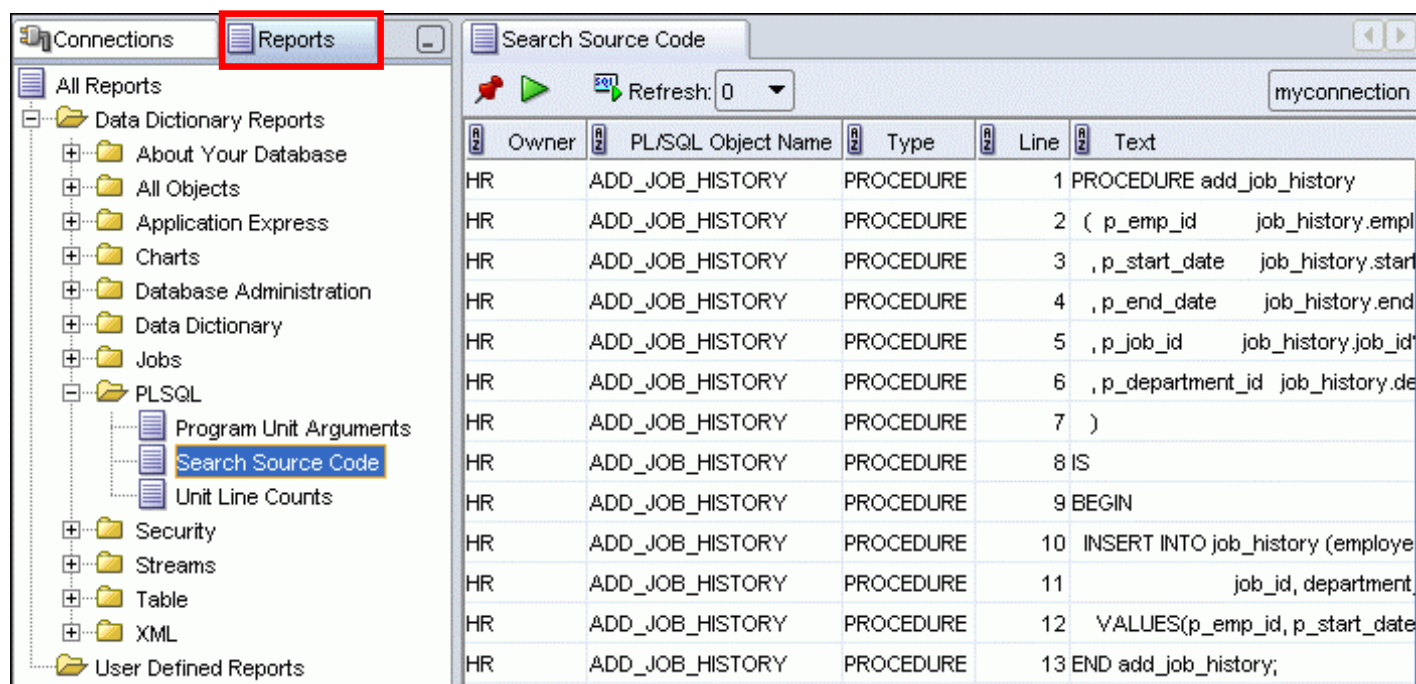
Debugging Procedures and Functions

- Use SQL Developer to debug PL/SQL functions and procedures.
- Use the Compile for Debug option to perform a PL/SQL compilation so that the procedure can be debugged.
- Use Debug menu options to set breakpoints, and to perform step into, step over tasks.



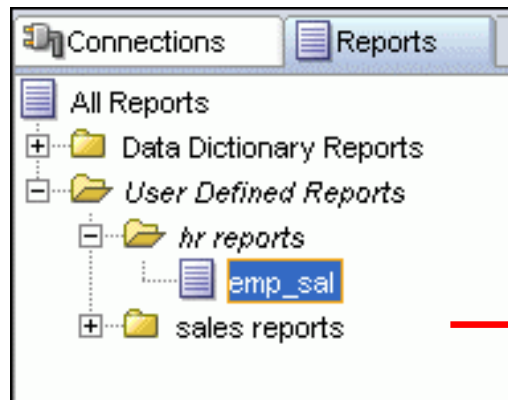
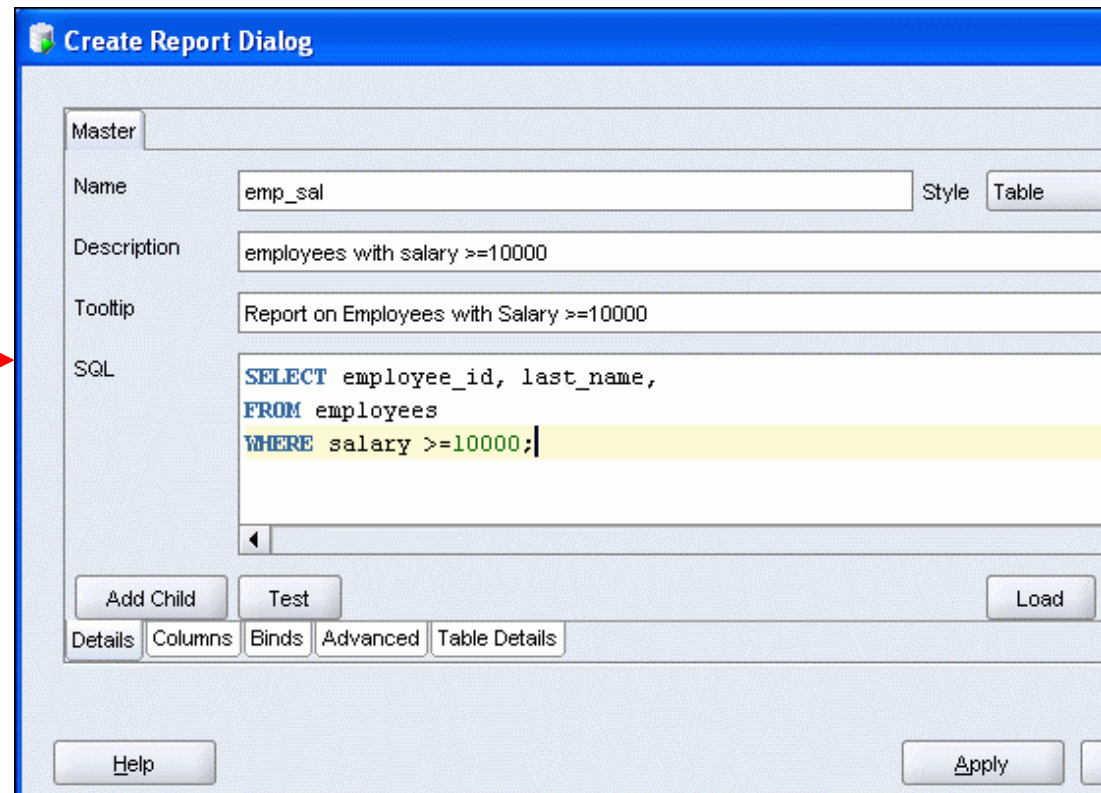
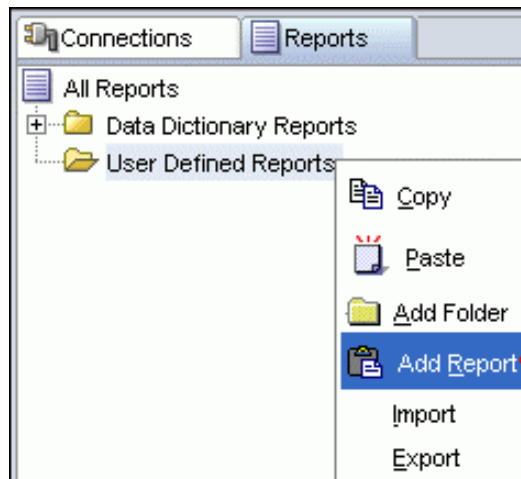
Database Reporting

SQL Developer provides a number of predefined reports about the database and its objects.



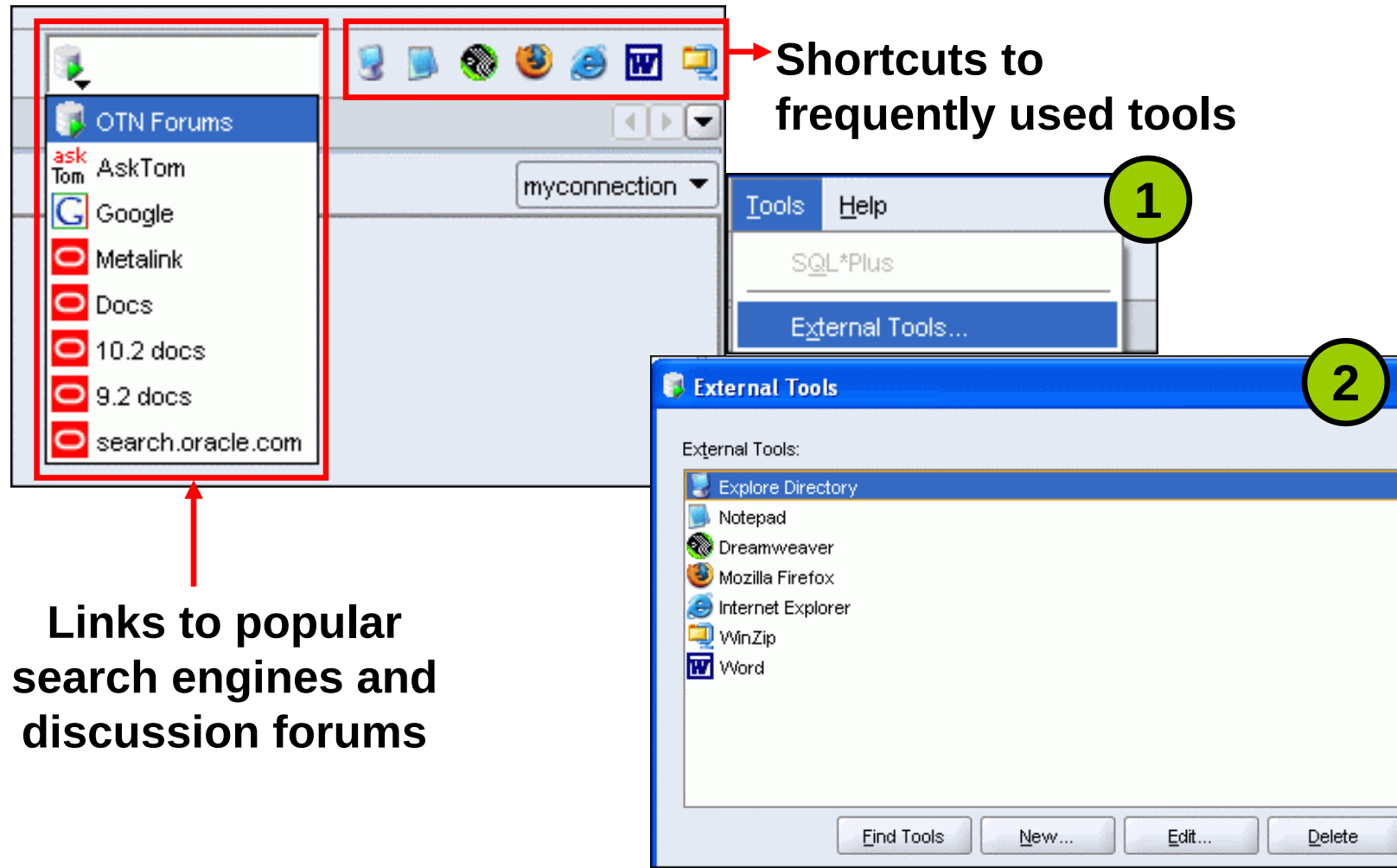
Creating a User-Defined Report

Create and save user-defined reports for repeated use.



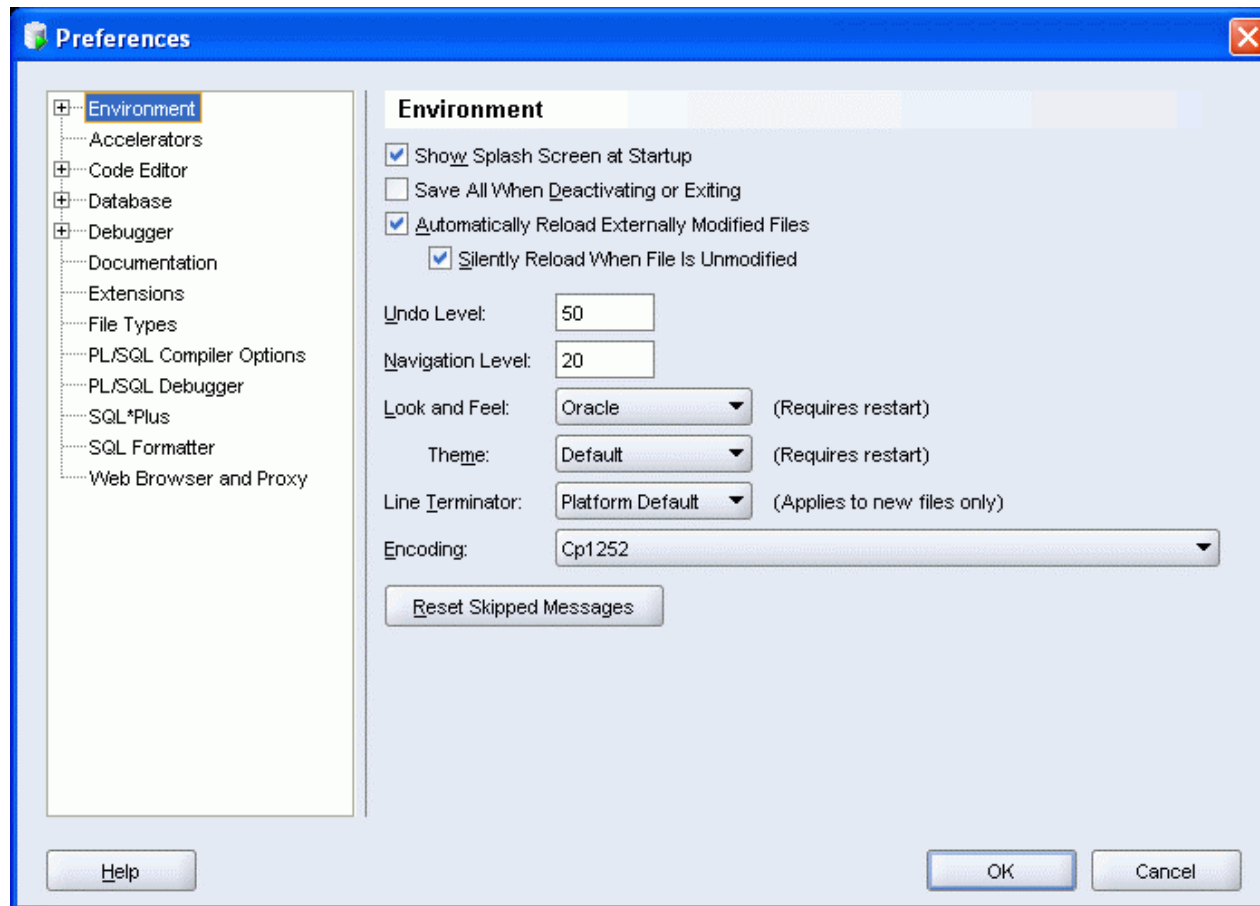
Organize reports in folders.

Search Engines and External Tools



Setting Preferences

- Customize the SQL Developer interface and environment.
- In the Tools menu, select Preferences.

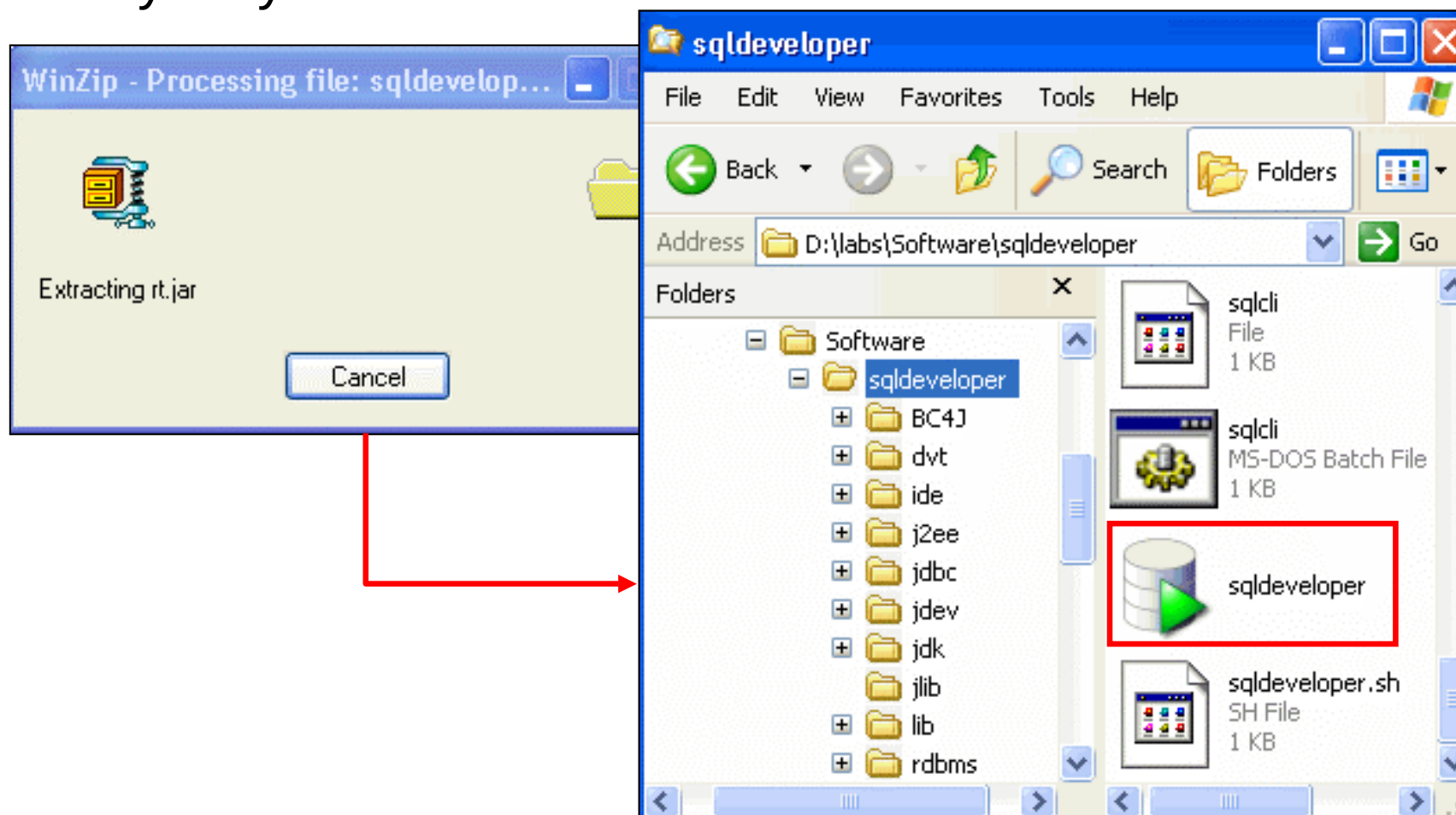


Specifications of SQL Developer 1.5.3

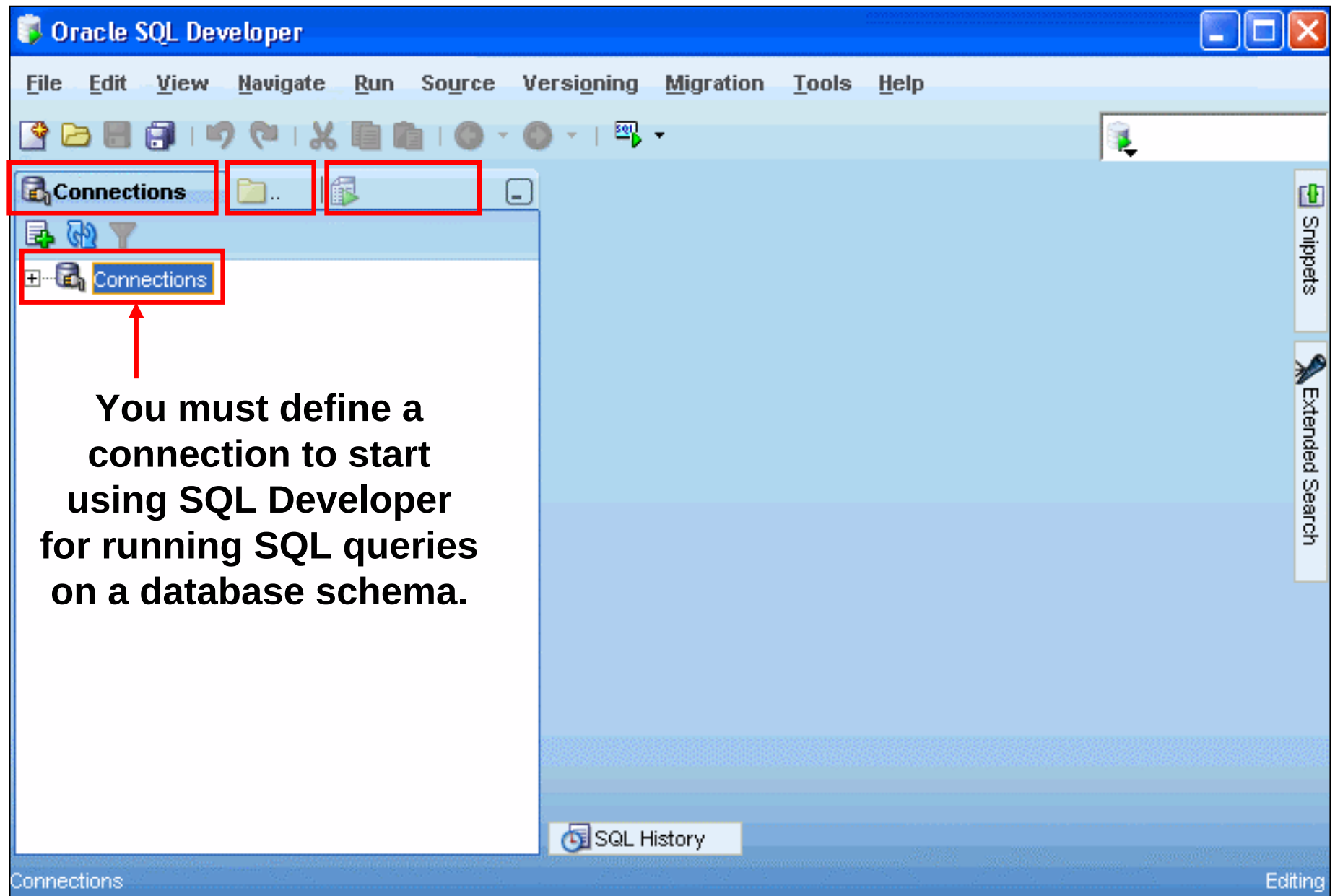
- SQL Developer 1.5.3 is the first translation release, and is a patch to Oracle SQL Developer 1.5.
- New feature list is available at:
 - http://www.oracle.com/technology/products/database/sql_developer/files/newFeatures_v15.html
- Supports Windows, Linux, and Mac OS X platforms
- To install, unzip the downloaded SQL Developer kit, which includes the required minimum JDK (JDK1.5.0_06).
- To start, double-click `sqldeveloper.exe`
- Connects to Oracle Database version 9.2.0.1 and later
- Freely downloadable from the following link:
 - http://www.oracle.com/technology/products/database/sql_developer/index.html

Installing SQL Developer 1.5.3

Download the Oracle SQL Developer kit and unzip into any directory on your machine.



SQL Developer 1.5.3 Interface



Summary

In this appendix, you should have learned how to use SQL Developer to do the following:

- Browse, create, and edit database objects
- Execute SQL statements and scripts in SQL Worksheet
- Create and save custom reports

D

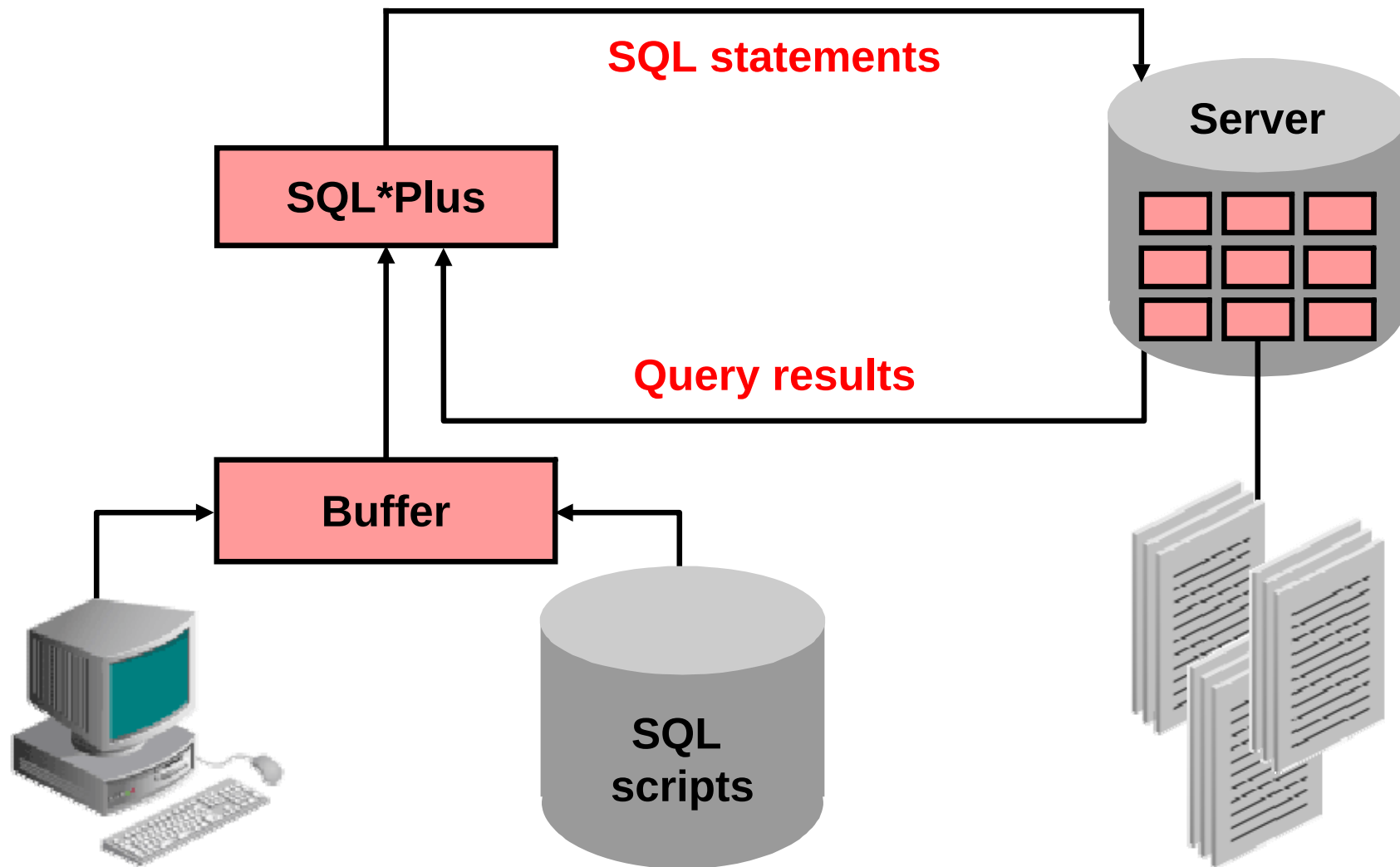
Using SQL*Plus

Objectives

After completing this appendix, you should be able to do the following:

- Log in to SQL*Plus
- Edit SQL commands
- Format output using SQL*Plus commands
- Interact with script files

SQL and SQL*Plus Interaction



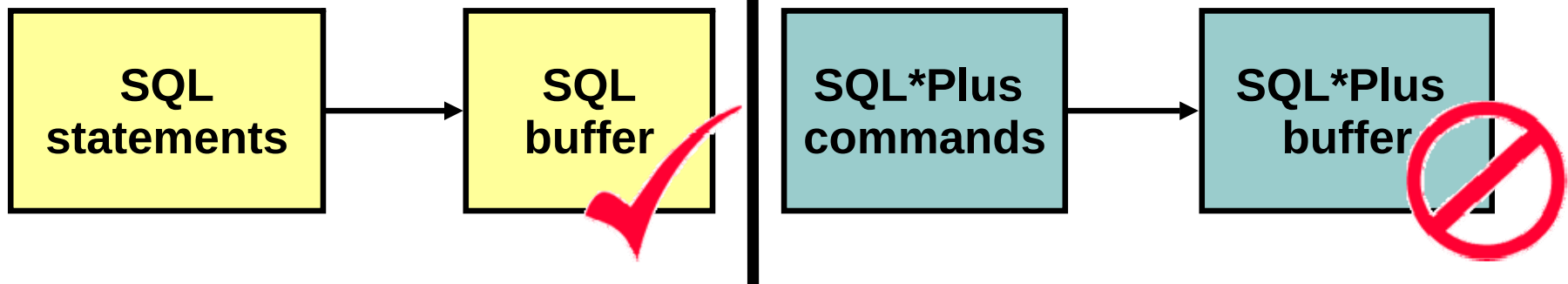
SQL Statements Versus SQL*Plus Commands

SQL

- A language
- ANSI standard
- Keywords cannot be abbreviated
- Statements manipulate data and table definitions in the database

SQL*Plus

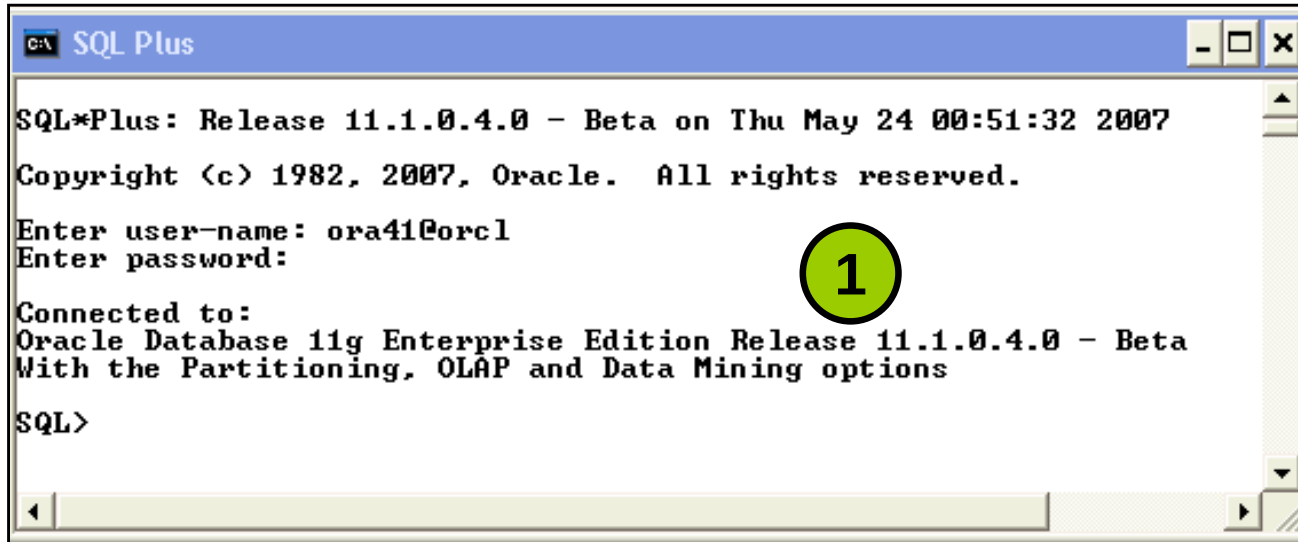
- An environment
- Oracle proprietary
- Keywords can be abbreviated
- Commands do not allow manipulation of values in the database



Overview of SQL*Plus

- Log in to SQL*Plus.
- Describe the table structure.
- Edit your SQL statement.
- Execute SQL from SQL*Plus.
- Save SQL statements to files and append SQL statements to files.
- Execute saved files.
- Load commands from file to buffer to edit.

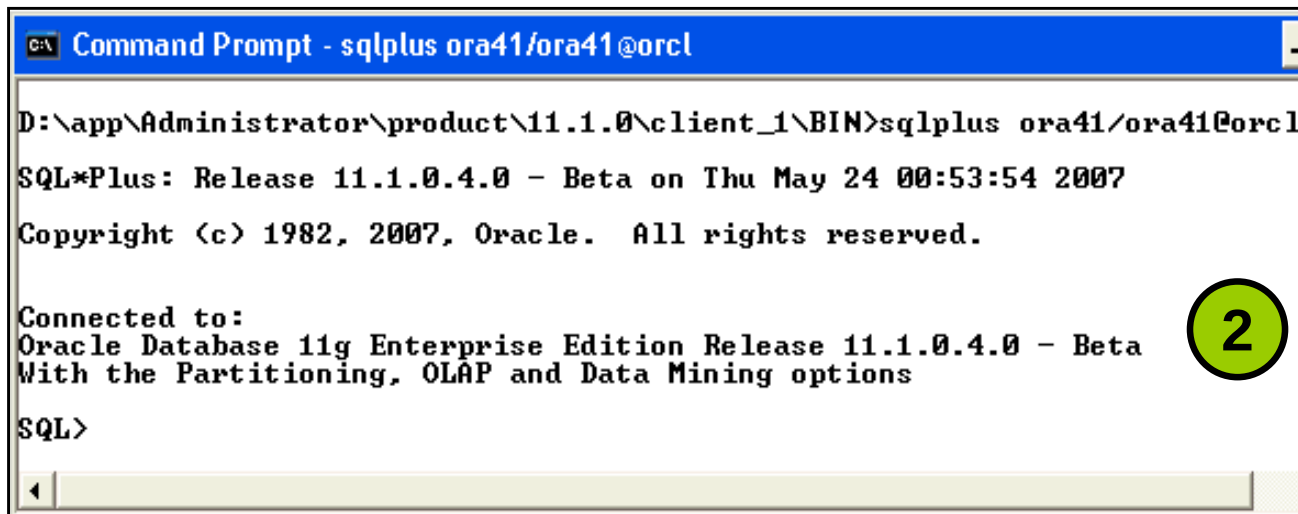
Logging In to SQL*Plus



```
C:\ SQL Plus

SQL*Plus: Release 11.1.0.4.0 - Beta on Thu May 24 00:51:32 2007
Copyright (c) 1982, 2007, Oracle. All rights reserved.
Enter user-name: ora41@orcl
Enter password:
Connected to:
Oracle Database 11g Enterprise Edition Release 11.1.0.4.0 - Beta
With the Partitioning, OLAP and Data Mining options
SQL>
```

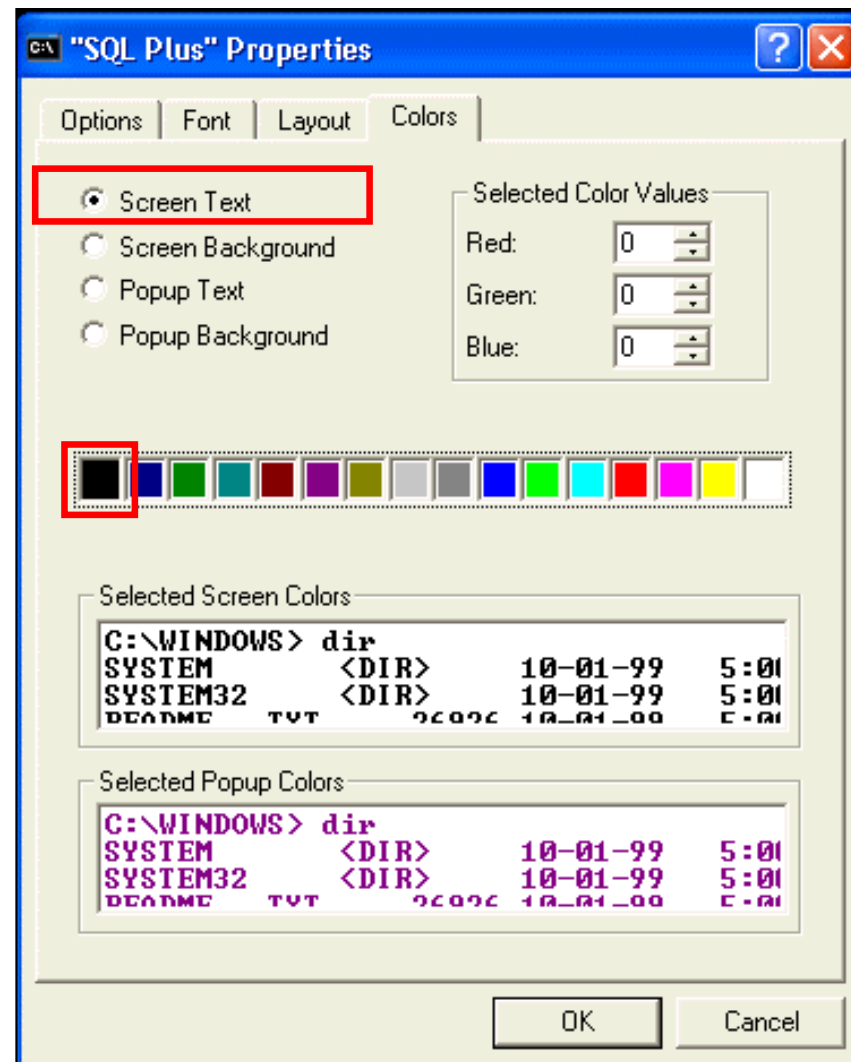
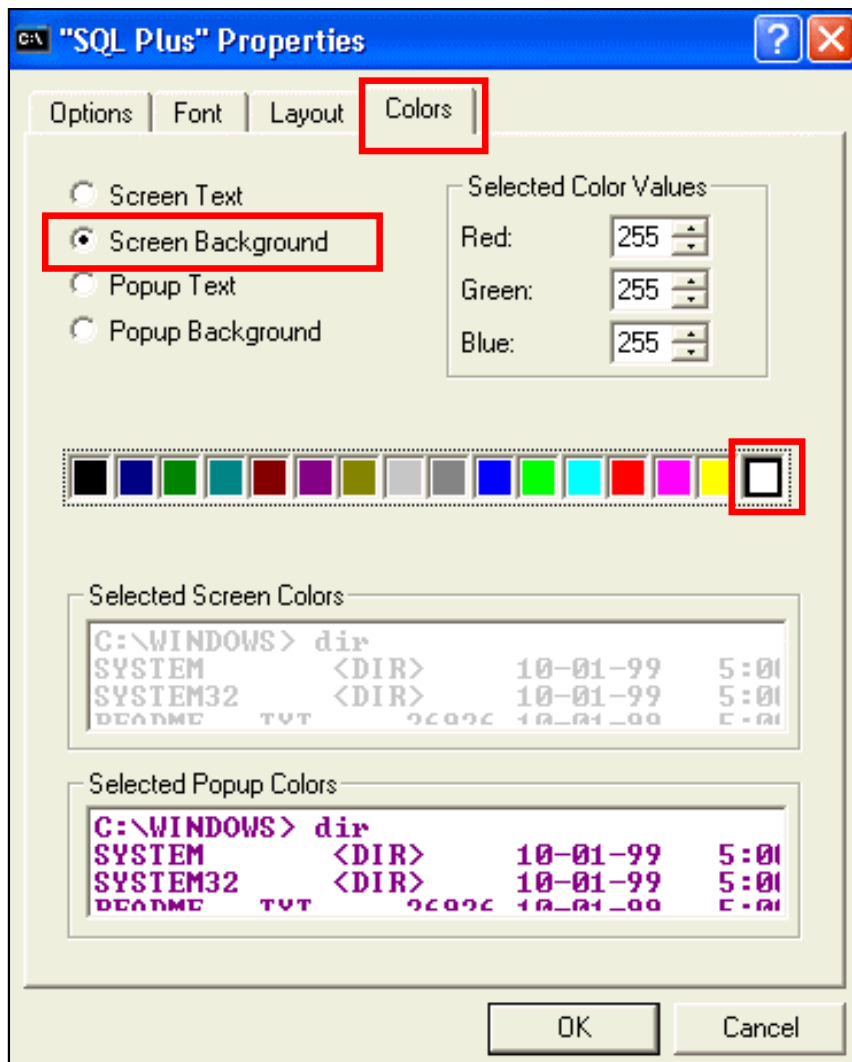
```
sqlplus [username[/password[@database]]]
```



```
C:\ Command Prompt - sqlplus ora41/ora41@orcl

D:\app\Administrator\product\11.1.0\client_1\BIN>sqlplus ora41/ora41@orcl
SQL*Plus: Release 11.1.0.4.0 - Beta on Thu May 24 00:53:54 2007
Copyright (c) 1982, 2007, Oracle. All rights reserved.
Connected to:
Oracle Database 11g Enterprise Edition Release 11.1.0.4.0 - Beta
With the Partitioning, OLAP and Data Mining options
SQL>
```

Changing the Settings of SQL*Plus Environment



Displaying Table Structure

Use the SQL*Plus DESCRIBE command to display the structure of a table:

```
DESC[RIBE]  tablename
```

Displaying Table Structure

```
DESCRIBE departments
```

Name	Null?	Type
-----	-----	-----
DEPARTMENT_ID	NOT NULL	NUMBER(4)
DEPARTMENT_NAME	NOT NULL	VARCHAR2(30)
MANAGER_ID		NUMBER(6)
LOCATION_ID		NUMBER(4)

SQL*Plus Editing Commands

- `A[PPEND] text`
- `C[HANGE] / old / new`
- `C[HANGE] / text /`
- `CL[EAR] BUFF[ER]`
- `DEL`
- `DEL n`
- `DEL m n`

SQL*Plus Editing Commands

- I [NPUT]
- I [NPUT] *text*
- L [IST]
- L [IST] *n*
- L [IST] *m n*
- R [UN]
- *n*
- *n text*
- 0 *text*

Using LIST, n, and APPEND

LIST

```
1  SELECT last_name  
2* FROM    employees
```

1

```
1* SELECT last_name
```

A , job_id

```
1* SELECT last_name, job_id
```

LIST

```
1  SELECT last_name, job_id  
2* FROM    employees
```

Using the CHANGE Command

```
LIST  
1* SELECT * from employees
```

```
c/employees/departments  
1* SELECT * from departments
```

```
LIST  
1* SELECT * from departments
```

SQL*Plus File Commands

- `SAVE filename`
- `GET filename`
- `START filename`
- `@ filename`
- `EDIT filename`
- `SPOOL filename`
- `EXIT`

Using the SAVE, START, and EDIT Commands

```
LIST
```

```
1  SELECT last_name, manager_id, department_id
2* FROM employees
```

```
SAVE my_query
```

```
Created file my_query
```

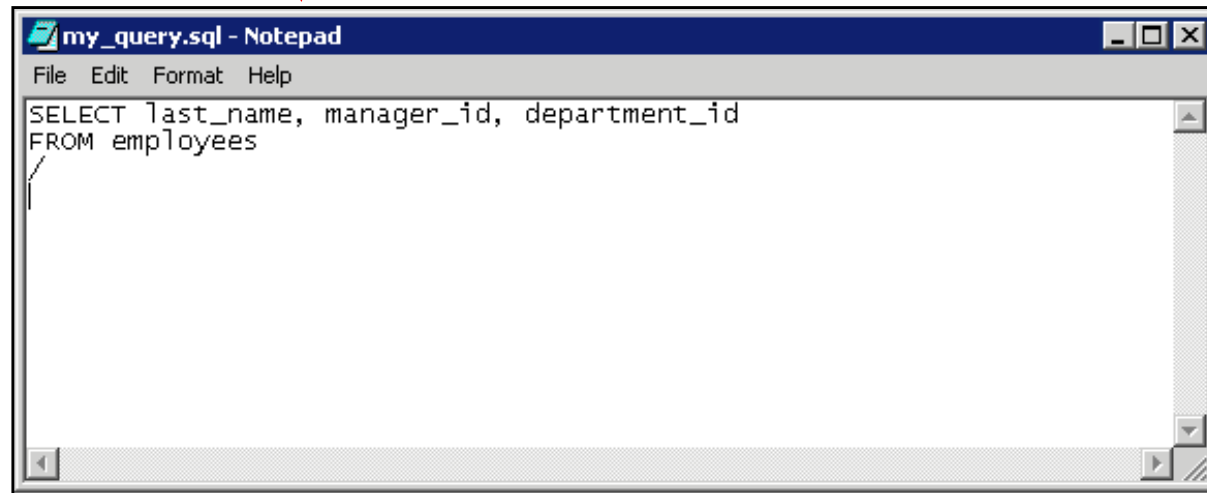
```
START my_query
```

LAST_NAME	MANAGER_ID	DEPARTMENT_ID
King		90
Kochhar	100	90
...		

107 rows selected.

Using the SAVE, START, and EDIT Commands

EDIT my_query



SERVEROUTPUT Command

- Use the SET SERVEROUT [PUT] command to control whether to display the output of stored procedures or PL/SQL blocks in SQL*Plus.
- The DBMS_OUTPUT line length limit is increased from 255 bytes to 32767 bytes.
- The default size is now unlimited.
- Resources are not preallocated when SERVEROUTPUT is set.
- Because there is no performance penalty, use UNLIMITED unless you want to conserve physical memory.

```
SET SERVEROUT [PUT] {ON | OFF} [SIZE {n | UNL[IMITED]}]  
[FOR [MAT] {WRA[PPED] | WOR[D_WAPPED] | TRU[NCATED] }]
```


Using the SQL*Plus SPOOL Command

```
SPO[OL] [file_name[.ext] [CRE[ATE] | REP[LACE] |  
APP[END]] | OFF | OUT]
```

Option	Description
file_name[.ext]	Spools output to the specified file name
CRE[ATE]	Creates a new file with the name specified
REP[LACE]	Replaces the contents of an existing file. If the file does not exist, REPLACE creates the file.
APP[END]	Adds the contents of the buffer to the end of the file you specify
OFF	Stops spooling
OUT	Stops spooling and sends the file to your computer's standard (default) printer

Using the AUTOTRACE Command

- The AUTOTRACE command displays a report after the successful execution of SQL DML statements such as SELECT, INSERT, UPDATE, or DELETE.
- The report can now include execution statistics and the query execution path.

```
SET AUTOT[RACE] {ON | OFF | TRACE[ONLY]} [EXP[LAIN]]  
[STAT[ISTICS]]
```

```
SET AUTOTRACE ON
```

```
-- The AUTOTRACE report includes both the optimizer  
-- execution path and the SQL statement execution  
-- statistics.
```

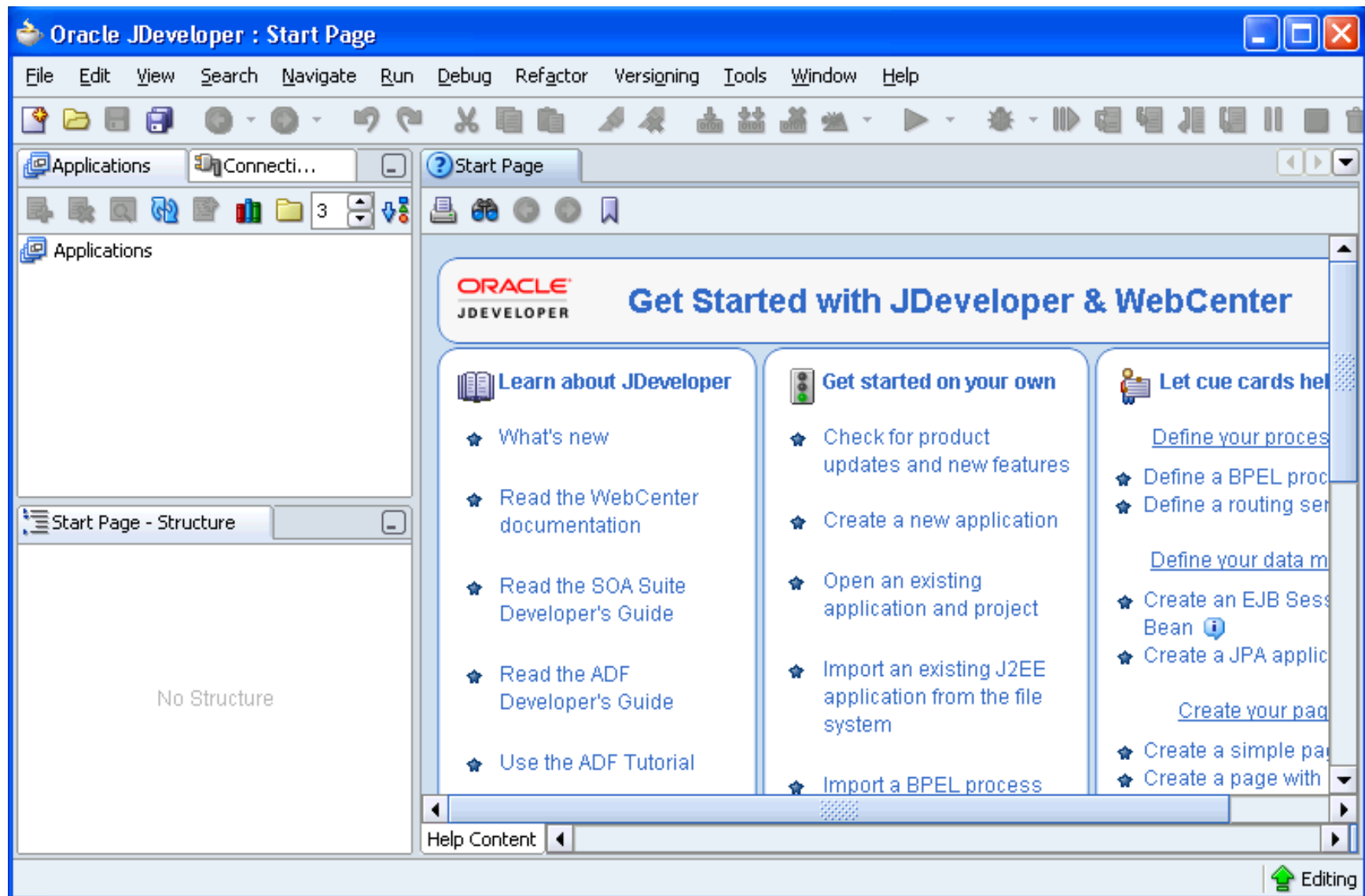
Summary

In this appendix, you should have learned how to use SQL*Plus as an environment to do the following:

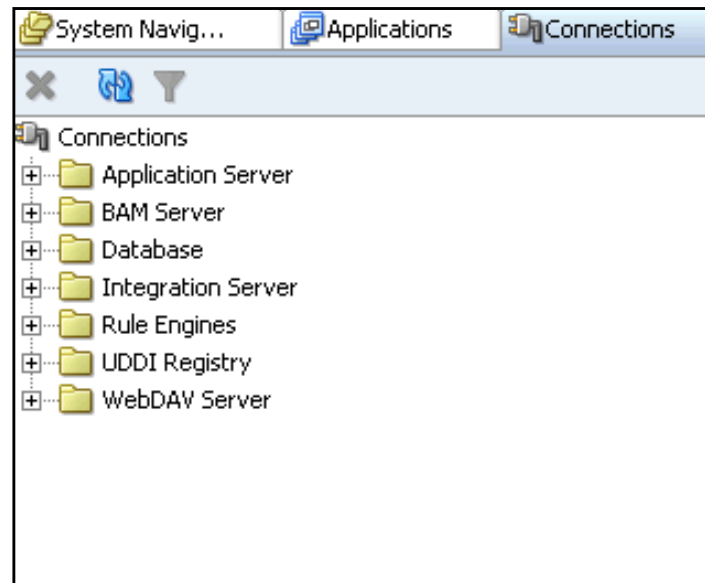
- Execute SQL statements
- Edit SQL statements
- Format output
- Interact with script files

Using JDeveloper

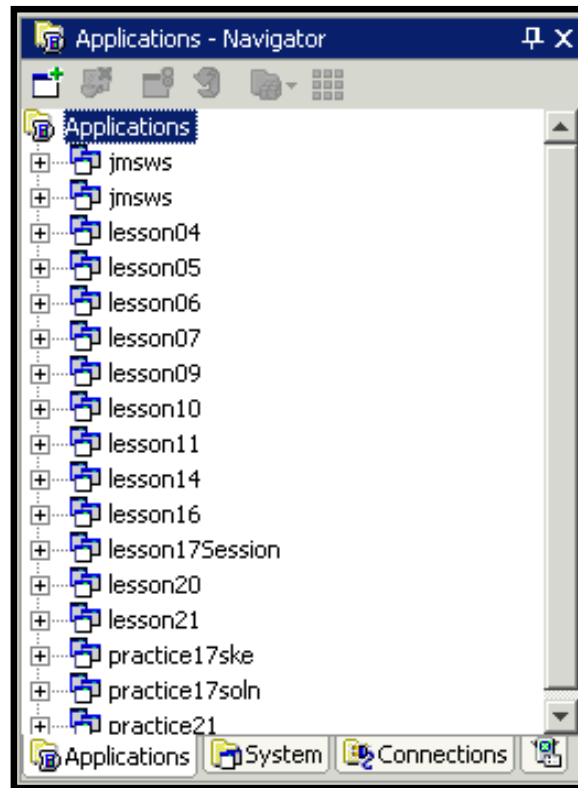
Oracle JDeveloper



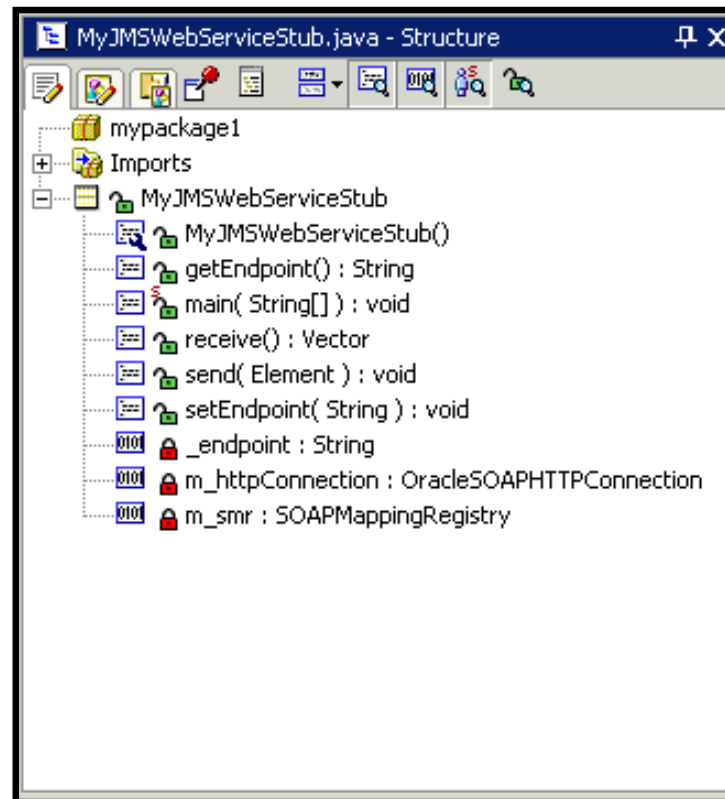
Connection Navigator



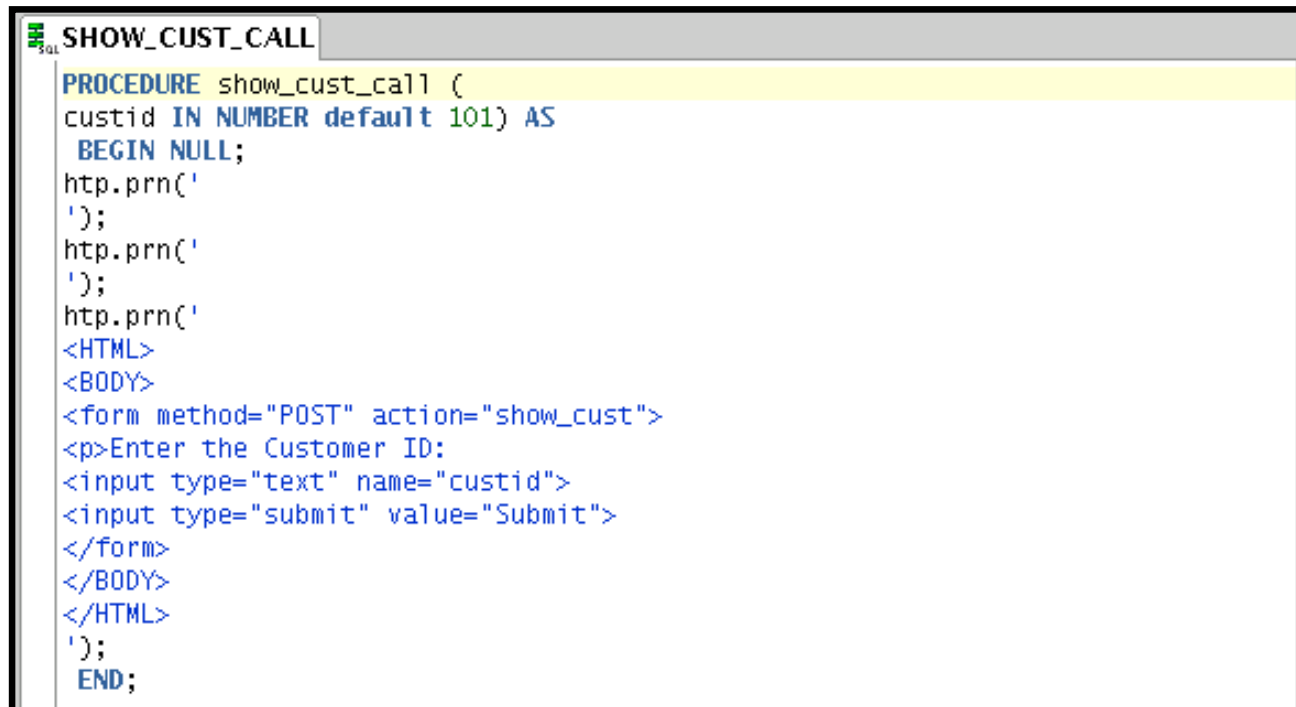
Applications - Navigator



Structure Window



Editor Window

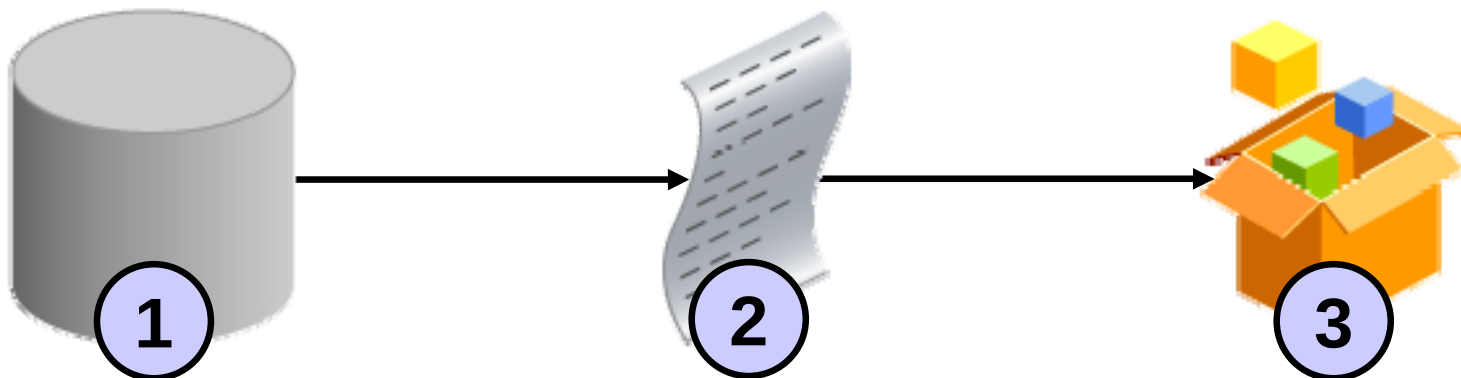


```
SQL SHOW_CUST_CALL
PROCEDURE show_cust_call (
custid IN NUMBER default 101) AS
BEGIN NULL;
http.prn('
');
http.prn('
');
http.prn('
<HTML>
<BODY>
<form method="POST" action="show_cust">
<p>Enter the Customer ID:
<input type="text" name="custid">
<input type="submit" value="Submit">
</form>
</BODY>
</HTML>
');
END;
```

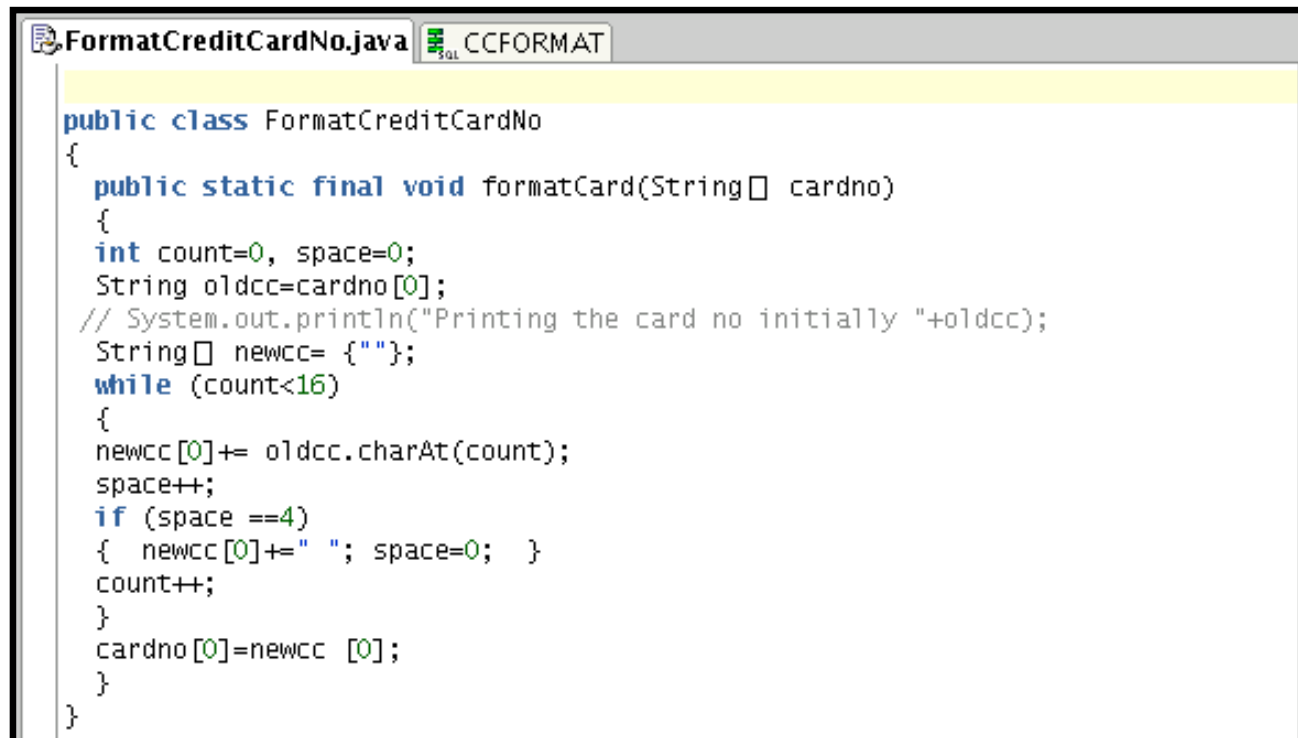
Deploying Java Stored Procedures

Before deploying Java stored procedures, perform the following steps:

1. Create a database connection.
2. Create a deployment profile.
3. Deploy the objects.



Publishing Java to PL/SQL



```
FormatCreditCardNo.java  CCFORMAT

public class FormatCreditCardNo
{
    public static final void formatCard(String[] cardno)
    {
        int count=0, space=0;
        String oldcc=cardno[0];
        // System.out.println("Printing the card no initially "+oldcc);
        String[] newcc= {" "};
        while (count<16)
        {
            newcc[0]+= oldcc.charAt(count);
            space++;
            if (space ==4)
            { newcc[0]+=" "; space=0; }
            count++;
        }
        cardno[0]=newcc [0];
    }
}
```



```
FormatCreditCardNo.java  CCFORMAT

PROCEDURE ccformat (x IN OUT varchar2)
AS LANGUAGE JAVA
NAME 'FormatCreditCardNo.formatCard(java.lang.String[])';
```

Creating Program Units

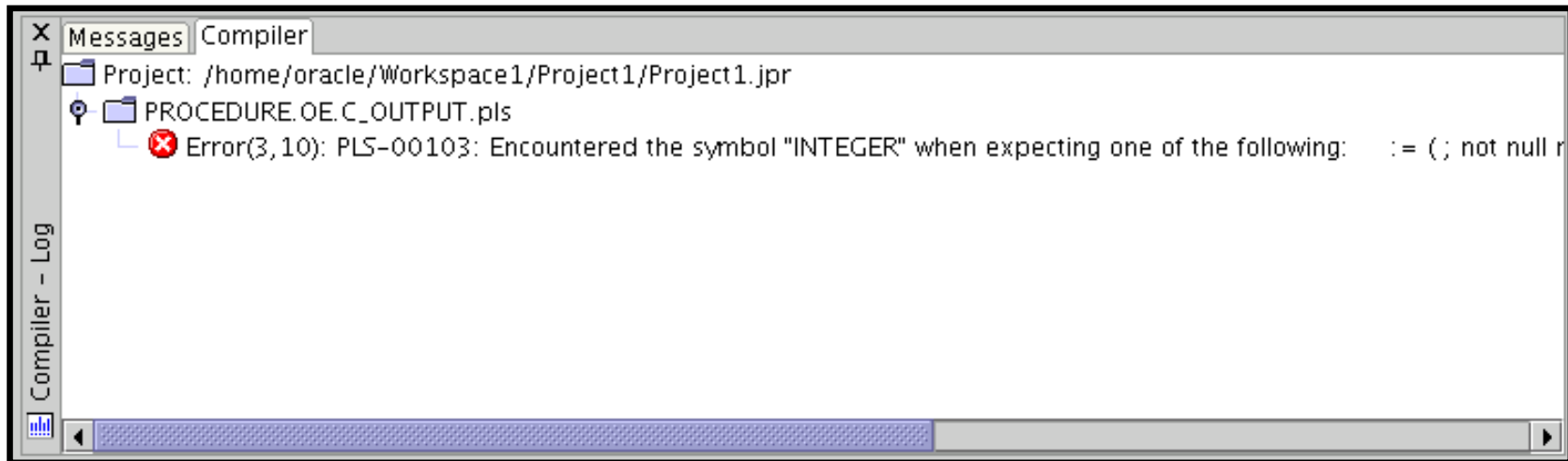


The screenshot shows a window titled 'TEST_JDEV' with a tab icon. The code inside is as follows:

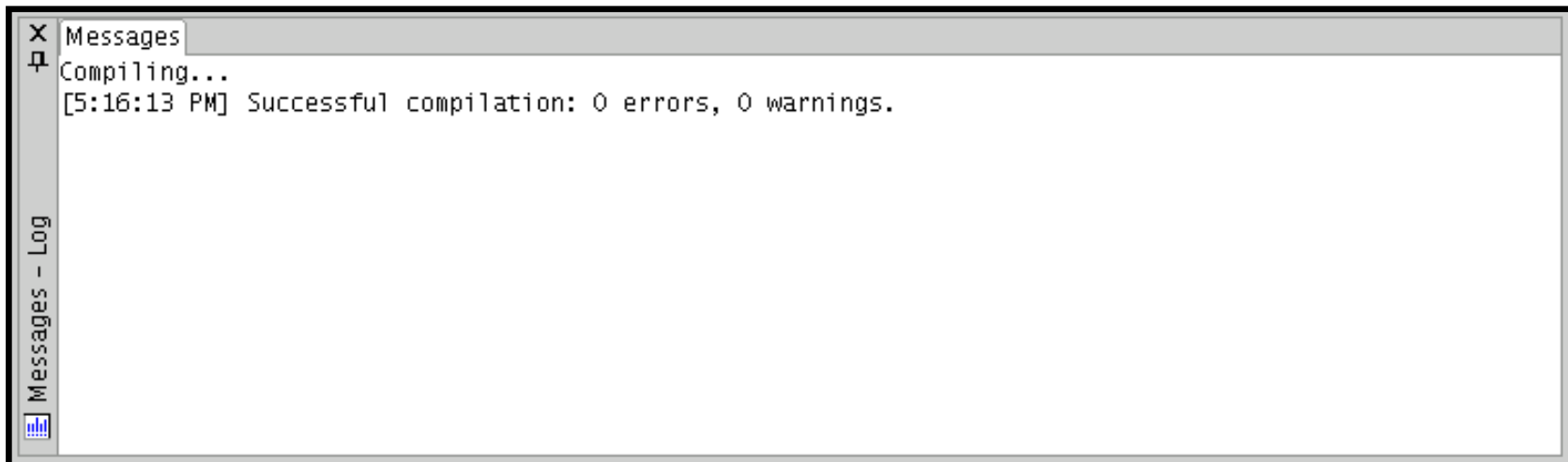
```
FUNCTION "TEST_JDEV" RETURN VARCHAR2  
AS  
BEGIN  
    RETURN(' ');  
END;
```

Skeleton of the function

Compiling

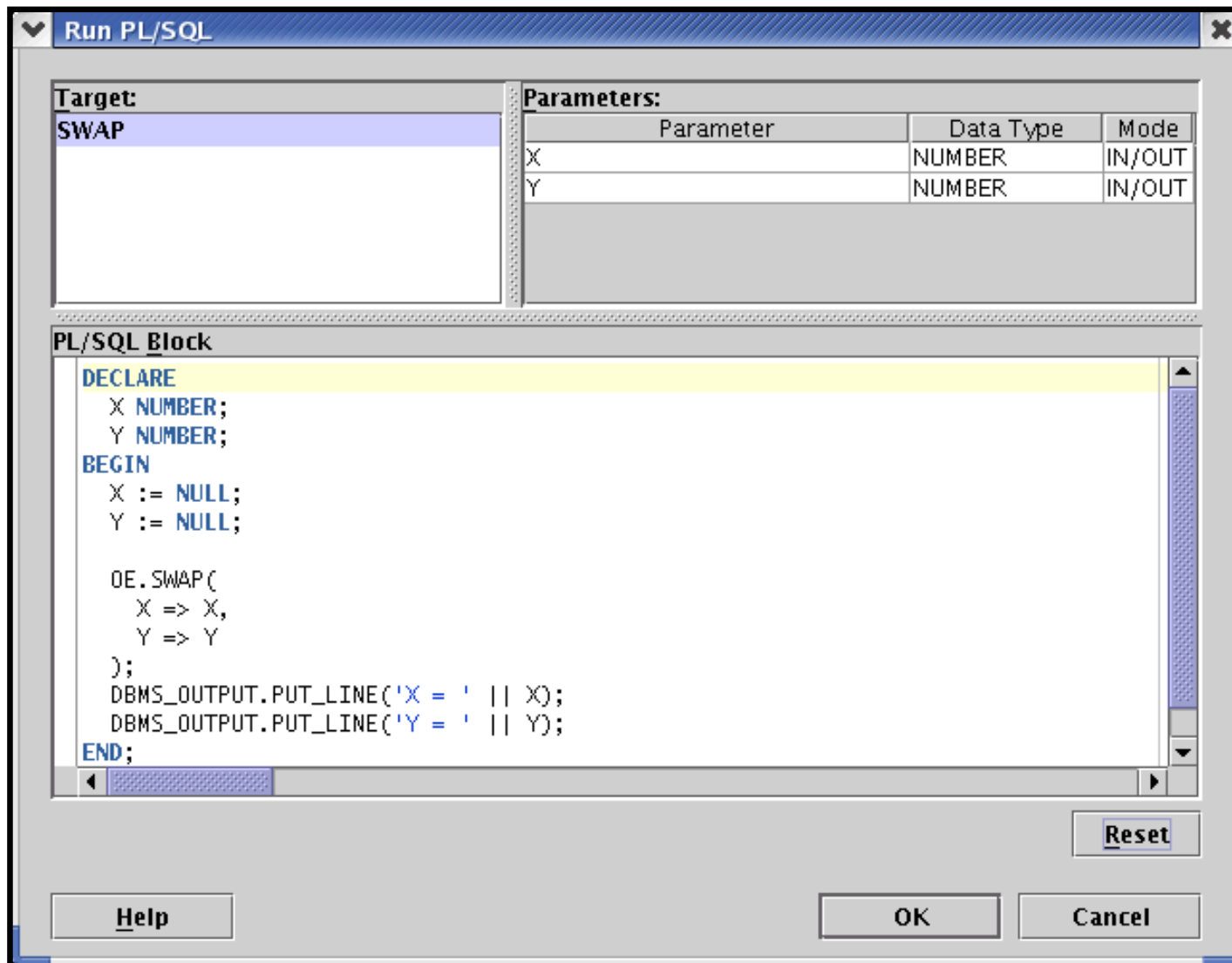


Compilation with errors

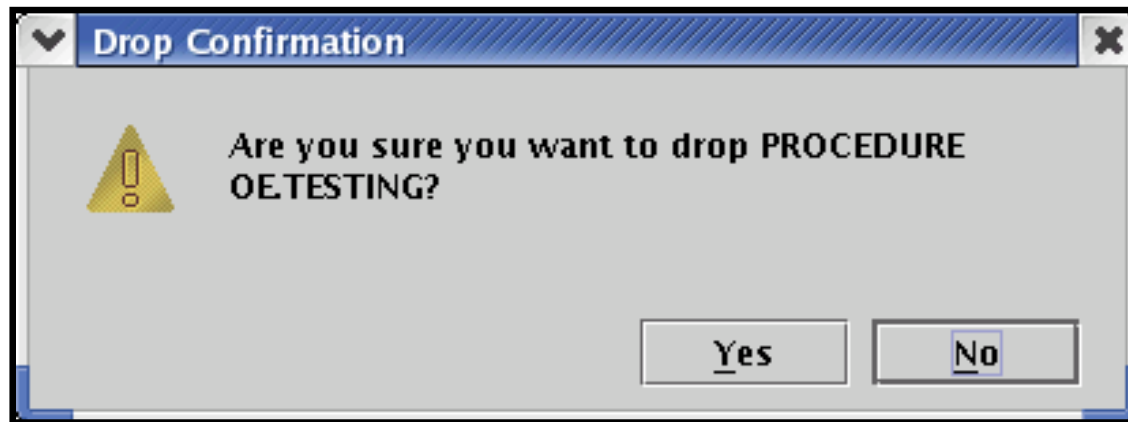


Compilation without errors

Running a Program Unit



Dropping a Program Unit



Debugging PL/SQL Programs

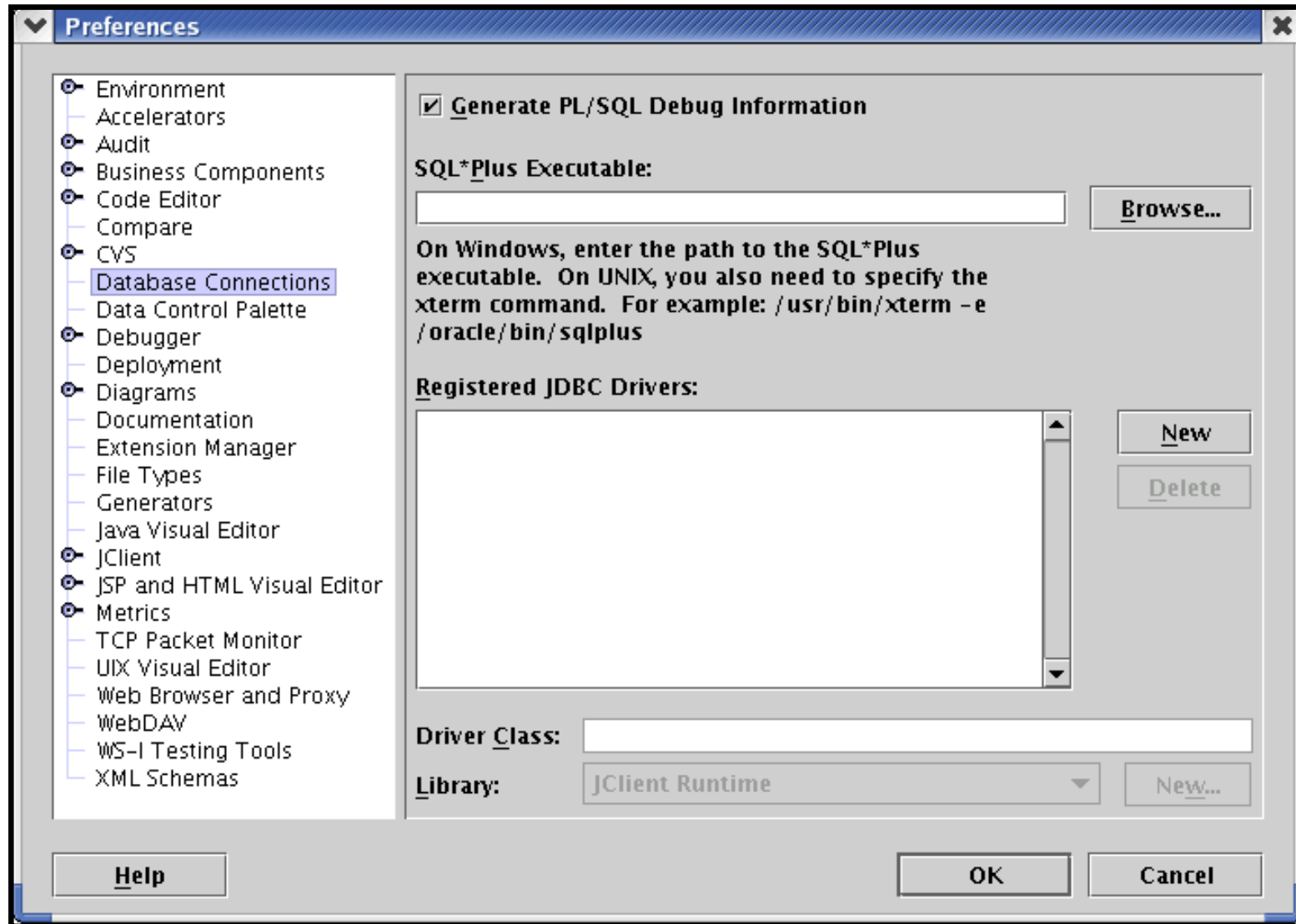
JDeveloper support two types of debugging:

- Local
- Remote

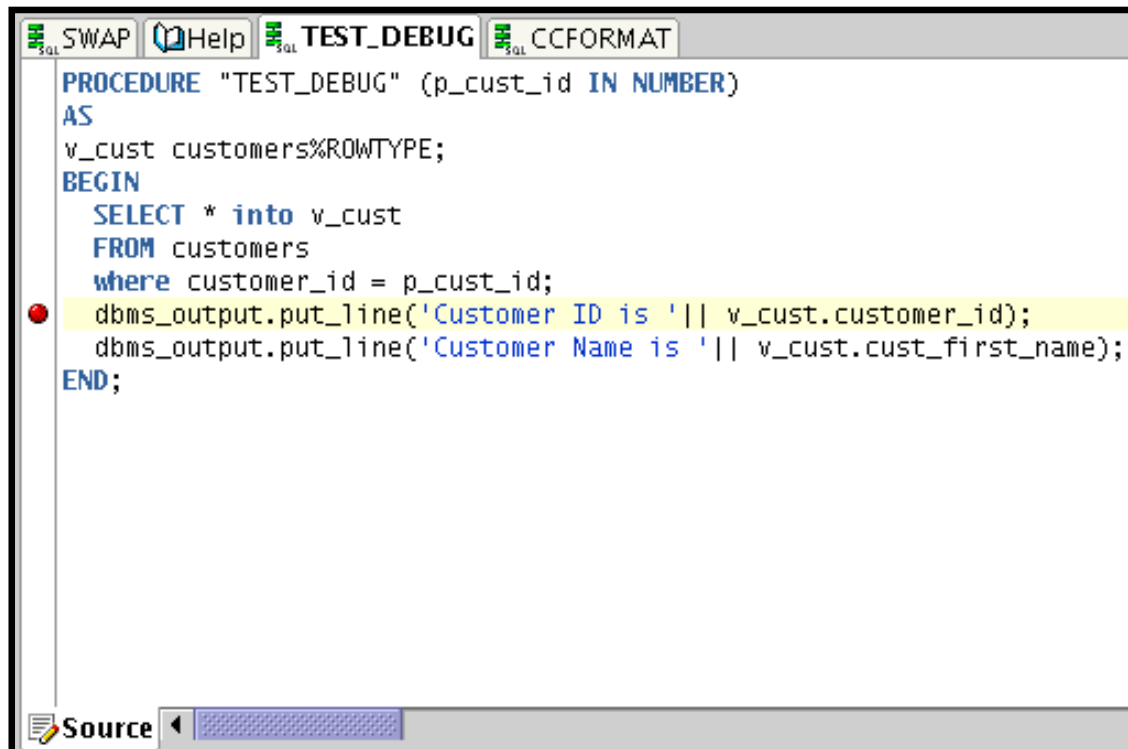
You need the following privileges to perform PL/SQL debugging:

- `DEBUG ANY PROCEDURE`
- `DEBUG CONNECT SESSION`

Debugging PL/SQL Programs



Setting Breakpoints



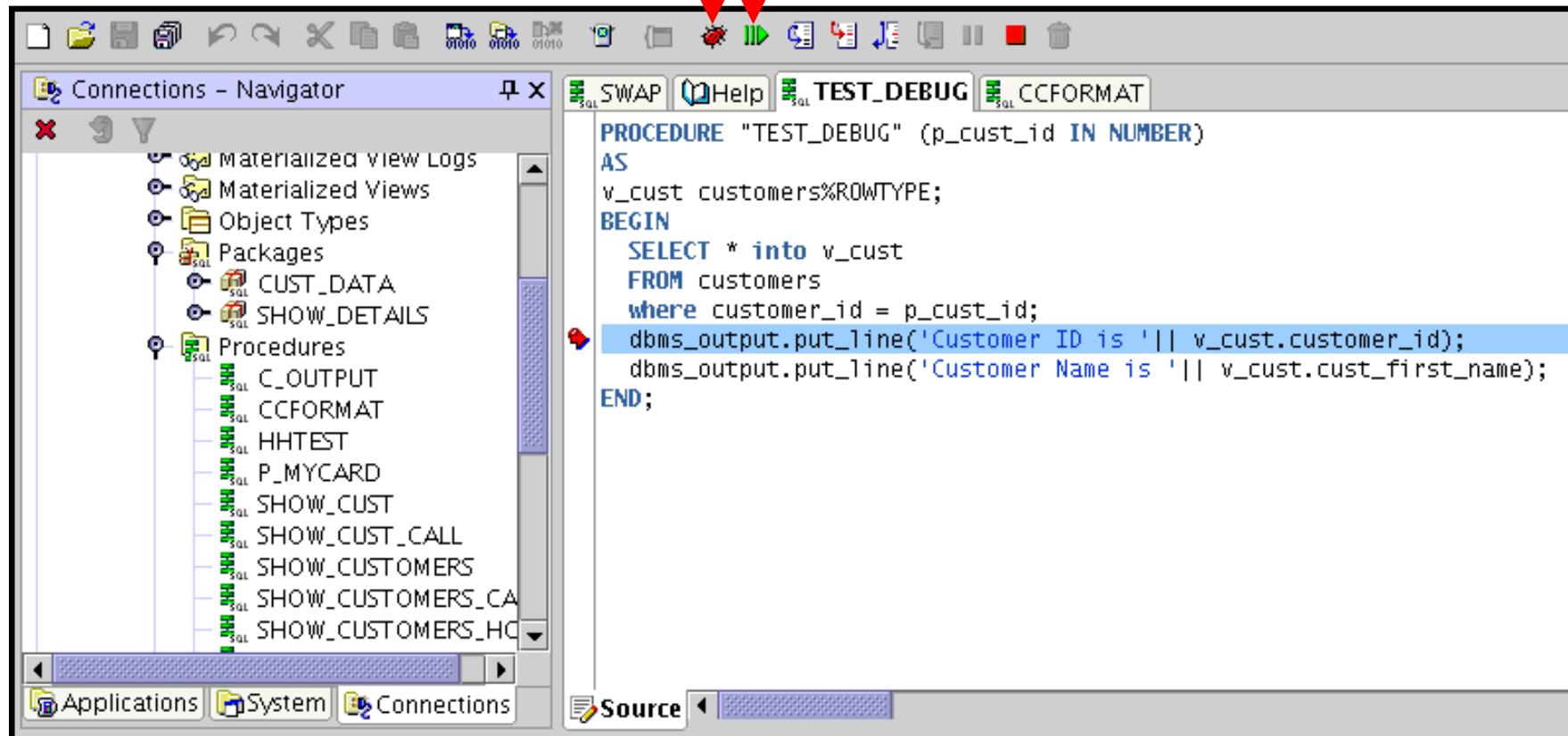
The screenshot shows a window titled "TEST_DEBUG" with a menu bar containing "SWAP", "Help", "TEST_DEBUG", and "CCFORMAT". The main text area contains the following PL/SQL code:

```
PROCEDURE "TEST_DEBUG" (p_cust_id IN NUMBER)
AS
v_cust customers%ROWTYPE;
BEGIN
  SELECT * into v_cust
  FROM customers
  where customer_id = p_cust_id;
  dbms_output.put_line('Customer ID is ' || v_cust.customer_id);
  dbms_output.put_line('Customer Name is ' || v_cust.cust_first_name);
END;
```

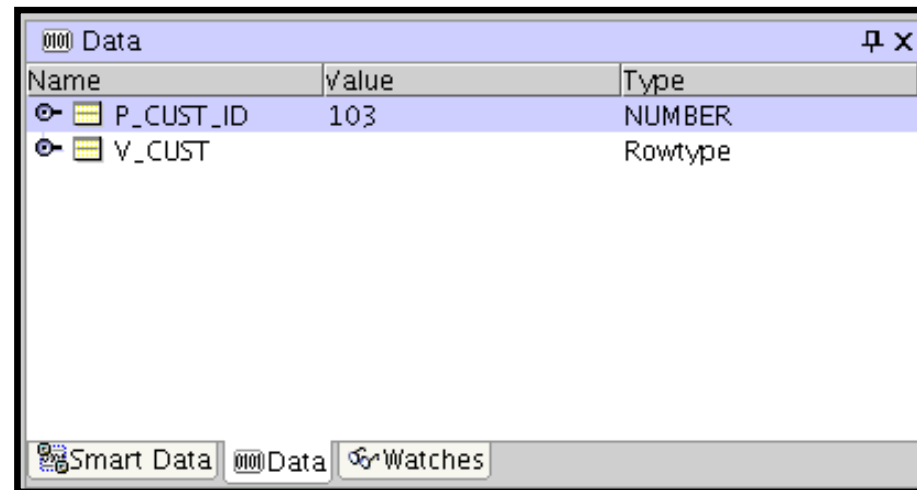
A red circular breakpoint icon is positioned to the left of the line containing the first `dbms_output.put_line` statement. The text of this line is highlighted in yellow. At the bottom of the window, there is a "Source" tab and a horizontal scrollbar.

Stepping Through Code

Debug   Resume

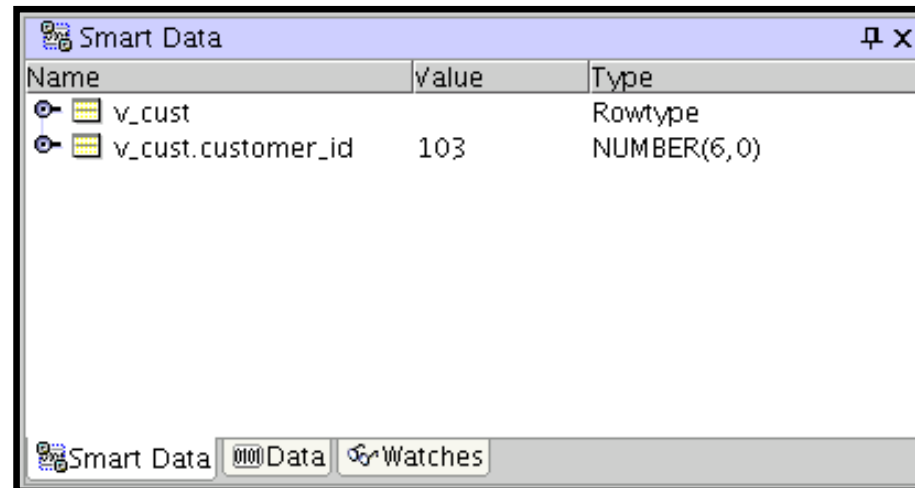


Examining and Modifying Variables



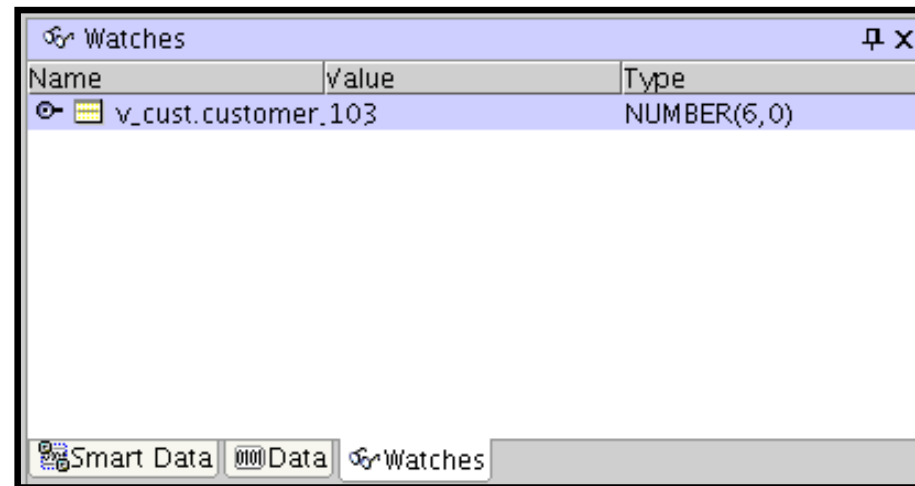
Data window

Examining and Modifying Variables



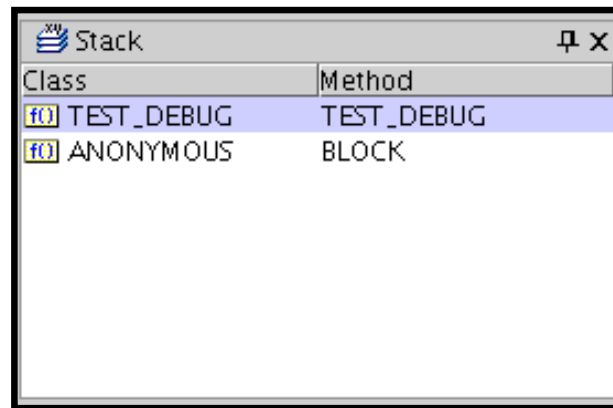
Smart Data window

Examining and Modifying Variables



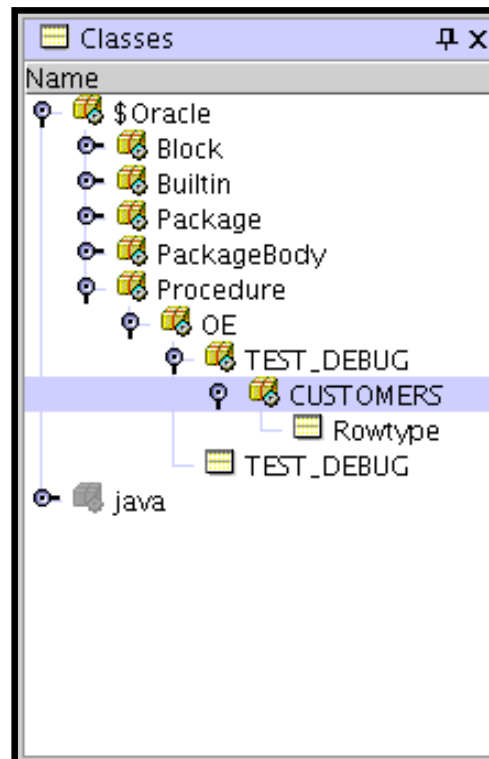
Watches window

Examining and Modifying Variables



Stack window

Examining and Modifying Variables



Classes window

F

REF Cursors

Cursor Variables

- Cursor variables are like C or Pascal pointers, which hold the memory location (address) of an item instead of the item itself.
- In PL/SQL, a pointer is declared as `REF X`, where `REF` is short for `REFERENCE` and `X` stands for a class of objects.
- A cursor variable has the data type `REF CURSOR`.
- A cursor is static, but a cursor variable is dynamic.
- Cursor variables give you more flexibility.

Using Cursor Variables

- You can use cursor variables to pass query result sets between PL/SQL stored subprograms and various clients.
- PL/SQL can share a pointer to the query work area in which the result set is stored.
- You can pass the value of a cursor variable freely from one scope to another.
- You can reduce network traffic by having a PL/SQL block open (or close) several host cursor variables in a single round-trip.

Defining REF CURSOR Types

Define a REF CURSOR type:

```
Define a REF CURSOR type  
TYPE ref_type_name IS REF CURSOR [RETURN return_type];
```

Declare a cursor variable of that type:

```
ref_cv ref_type_name;
```

Example:

```
DECLARE  
TYPE DeptCurTyp IS REF CURSOR RETURN  
departments%ROWTYPE;  
dept_cv DeptCurTyp;
```

Using the OPEN-FOR, FETCH, and CLOSE Statements

- The OPEN-FOR statement associates a cursor variable with a multirow query, executes the query, identifies the result set, and positions the cursor to point to the first row of the result set.
- The FETCH statement returns a row from the result set of a multirow query, assigns the values of select-list items to corresponding variables or fields in the INTO clause, increments the count kept by %ROWCOUNT, and advances the cursor to the next row.
- The CLOSE statement disables a cursor variable.

Example of Fetching

```
DECLARE
    TYPE EmpCurTyp IS REF CURSOR;
    emp_cv      EmpCurTyp;
    emp_rec     employees%ROWTYPE;
    sql_stmt    VARCHAR2(200);
    my_job      VARCHAR2(10) := 'ST_CLERK';
BEGIN
    sql_stmt := 'SELECT * FROM employees
                WHERE job_id = :j';
    OPEN emp_cv FOR sql_stmt USING my_job;
    LOOP
        FETCH emp_cv INTO emp_rec;
        EXIT WHEN emp_cv%NOTFOUND;
        -- process record
    END LOOP;
    CLOSE emp_cv;
END;
/
```